

DATENGETRIEBENE OPTIMIERUNG VON PID-REGLERPARAMETERN IM LTI- BEREICH MITTELS MACHINE LEARNING

Bachelorarbeit

Author: Niklas Keßel

Matrikelnummer: 1274327

Betreut durch: Prof. Dr. Ing. André Edelmann & Dipl.-Ing. Sameh Ben Nablia

Abgabedatum: 25.09.2025

Frankfurt University of Applied Sciences,

Fb2: Informatik und Ingenieurwissenschaften,

Studiengang: BioVerfahrenstechnik (Bachelor of Engineering)

Beschluss des Prüfungsausschusses Maschinenbau zur Eigenständigkeitserklärung zur selbstständigen Erstellung einer studentischen Arbeit

Der Prüfungsausschuss Maschinenbau weist die Studierenden explizit darauf hin, dass sie ab 15. Mai 2024 verpflichtet sind, ihrer studentischen Prüfungsarbeit (wie z.B. semesterbegleitende Projektarbeit, Hausarbeit, Schwerpunktprojekt, Teamprojekt, Praxisprojekt, Bachelorarbeit, Wissenschaftlichem Projekt, Masterarbeit, ...) die folgende Eidesstattliche Erklärung beizufügen:

Eidesstattliche Erklärung

Hiermit versichere ich das Folgende:

- 1. Die vorgelegte Prüfungsleistung habe ich selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel und Quellen angefertigt.*
- 2. Alle Stellen, die wörtlich oder sinngemäß aus anderen Werken oder von anderen Personen übernommen wurden (z.B. fremde Texte, Arbeiten, Materialien, Abbildungen, Tabellen, Gedankengänge, Gesprächsnotizen etc.), sind kenntlich gemacht, indem meine Ausführungen eindeutige und vollständige Verweise und Zitate auf die entsprechenden Urheber und Quellen enthalten.*
- 3. Inhalte aus eigenen Berichten oder Arbeiten wurden nicht übernommen, oder sie sind kenntlich gemacht, indem meine Ausführungen eindeutige und vollständige Verweise und Zitate auf mich als Urheber und Quelle enthalten.*
- 4. Alle weiteren Inhalte der vorgelegten Prüfungsarbeit ohne entsprechende Verweise und Zitate stammen im urheberrechtlichen Sinn von mir. Abbildungen und Tabellen, die im urheberrechtlichen Sinne von mir stammen, sind mit [Eigene Darstellung] gekennzeichnet*
- 5. Ich kann sämtliche im Quellenverzeichnis aufgeführte Quellen in ausgedruckter Form dem Referenten oder dem Korreferenten, dem Prüfungsausschuss oder seinem Vorsitzenden vorlegen und werde dies unverzüglich tun, sofern ich dazu aufgefordert werde.*
- 6. Die Prüfungsleistung war weder vollständig noch in wesentlichen Teilen Gegenstand eines anderen Prüfungsverfahrens an dieser oder einer anderen Hochschule.*
7. ☐ *Ich habe weder KI-gestützte Schreibwerkzeuge und/oder andere KI-Anwendungen verwendet noch wurden von KI-Anwendungen generierte Texte, Textpassagen, Abbildungen oder Tabellen in die schriftliche Prüfungsarbeit eingefügt. **

ODER

Bitte wenden

Seite 2 zum Beschluss des Prüfungsausschusses zur Eigenständigkeitserklärung

☒ Ich habe nach Rücksprache mit dem Erstbetreuer der Prüfungsarbeit zur Erstellung der Prüfungsarbeit KI-gestützte Schreibwerkzeuge und/oder andere KI-Anwendungen verwendet. Bei deren Einsatz habe ich steuernd gearbeitet. Ich versichere, dass mein eigener gestalterischer Einfluss in der Arbeit überwiegt. Sämtliche von mir verwendeten KI-Anwendungen habe ich als Hilfsmittel mit deren Produktnamen und Bezugsquelle (URL) angegeben und eine Übersicht der im Rahmen der Prüfungsarbeit genutzten Funktionen vollständig aufgeführt. Die von KI-gestützten Schreibwerkzeugen oder anderen KI-Anwendungen generierten Texte, Textpassagen, Abbildungen oder Tabellen, die übernommen wurden, habe ich gekennzeichnet. *

* BITTE auswählen

Mir ist bekannt, dass ein Täuschungsversuch nach § 17 ABPO vorliegt, wenn sich eine der vorstehenden Versicherungen als unrichtig erweist. Mir ist ferner bekannt, dass die von mir eingereichte elektronische Version meiner Arbeit verwendet werden kann, um die Arbeit auf mögliche Plagiatsanteile oder KI-generierte Inhalte zu überprüfen

< eigenhändige Unterschrift >

Frankfurt am Main, den xx.xx.20xx

Zur Kenntnis genommen:

Niklas Keßel

1274327

Vorname/Name

Matrikelnummer

04.08.2025

Datum / Unterschrift



Danksagung

An dieser Stelle möchte ich mich herzlich bei all jenen bedanken, die mich während der Anfertigung dieser Bachelorarbeit unterstützt und maßgeblich zum Gelingen beigetragen haben.

Mein besonderer Dank gilt meinen Betreuern, Herrn Prof. Dr.-Ing. André Edelmann & Frau Dipl.-Ing. Sameh Ben Nablia, für die Vergabe dieses hochinteressanten Themas an der Schnittstelle von Regelungstechnik und maschinellem Lernen. Ich danke meinen beiden Betreuern für die durchgehend hervorragende Begleitung. Ihre fachkundige Anleitung, die anregenden Diskussionen und insbesondere ihre tatkräftige Unterstützung während der Versuchsreihen im Labor waren für das Gelingen dieser Arbeit entscheidend.

Contents

1 Abstract	1
2 Theoretische Grundlagen.....	2
2.1 Regelungstechnische Grundlagen	2
2.1.1 Übertragungsfunktionen und Regelstrecken	2
2.1.2 Stabilität und Dynamik in Regelkreisen	3
2.2 PID-Regler	4
2.2.1 Struktur und Wirkungsweise (P-, I-, D-Anteil)	4
2.2.2 Einfluss der Parameter K_p , K_i und K_d	4
2.3 Klassische Tuning-Methoden.....	5
2.3.1 Ziegler-Nichols-Verfahren	5
2.3.2 Chien-Hrones-Reswick-Methode	5
2.3.3 Simulink PID-Autotuning.....	6
2.3.4 Limitierungen klassischer Tuningverfahren	6
2.4 Performanzmetriken in der Regelungstechnik.....	7
3 Grundlagen des Machine Learning	9
3.1 Einführung in Machine Learning.....	9
3.1.1 Grundprinzipien und Paradigmen	9
3.1.2 Relevanz für die Regelungstechnik	10
3.2 Supervised Learning für Regelungstechnik.....	12
3.2.1 Lernprinzip	12
3.2.2 Anwendungsfälle in der PID-Optimierung	13
3.3 Auswahl relevanter Modelle	14
3.3.1 Decision Tree Methoden (Random Forrest / XGBoost)	14
3.3.2 Symbolic Regression	16
3.3.3 Gaussian Processes und Deep Gaussian Processes	17
3.4 Evaluierung von ML-Modellen.....	18
3.4.1 Statistische Gütemaße (R^2 , MAE, RMSE)	18
3.4.2 Visualisierung durch Vorhersage-vs.-Wahrheit-Analysen.....	19
4 Methoden und Implementierung.....	26
4.1 Gesamtarchitektur der Machine Learning pipeline	26
4.2 Generierung synthetischer Datensätze	28
4.2.1 Modellierung von Regelstrecken in MATLAB/Simulink.....	28
4.2.2 Definition der Parameterbereiche.....	29
4.2.3 Automatisierte Berechnung von Leistungsmetriken.....	30
4.3 Datenaufbereitung und Lernprozess	30

4.4 Surrogat- und Klassifikationsmodelle.....	31
4.4.1 Vorhersage von Reglerparametern aus Streckenparametern.....	31
4.4.2 Vorhersage von Leistungskenngrößen aus Strecken- und Reglerparametern	32
4.5 Optimierungsmodell mit nutzerdefinierten Präferenzen	34
5 Ergebnisse	38
5.1 Vergleich der Machine Learning Modelle	38
5.2 Vergleich mit klassischen Tuning-Methoden.....	41
5.2.1 Evaluierung anhand synthetischer Strecken in Simulink	41
5.2.2 Validierung an realen Regelstrecken	47
6 Diskussion	58
6.1 Interpretation der Ergebnisse im Vergleich zu klassischen Verfahren	58
6.2 Bewertung der Stärken und Schwächen des ML-Ansatzes	62
6.3 Grenzen der Generalisierbarkeit	64
7 Ausblick.....	64
8 Literaturverzeichnis	66
9 Anhang	67

Abbildungsverzeichnis

Abbildung 1: Decision Tree (Eigendarstellung)	15
Abbildung 2: Vorhersage vs Wahrheit ideal (Eigendarstellung)	20
Abbildung 3: Vorhersage vs Wahrheit große Streuung (Eigendarstellung)	21
Abbildung 4: Vorhersage vs Wahrheit mit Konfidenzintervall (Eigendarstellung)	22
Abbildung 5: Vorhersage vs Wahrheit mit Konfidenzintervall große Streuung (Eigendarstellung)	23
Abbildung 6: Vorhersage vs Wahrheit nach Regelparameter (Eigendarstellung)	24
Abbildung 7: Confusion Matrix (Eigendarstellung)	25
Abbildung 8: Pipeline (Eigendarstellung)	27
Abbildung 9: Simulink Modell Synthetische Generierung (Eigendarstellung)	29
Abbildung 10: Vorhersage von Reglerparametern aus Streckenparametern (Eigendarstellung)	32
Abbildung 11: Vorhersage von Leistungskenngrößen (Eigendarstellung)	33
Abbildung 12: Optimierungsmodell Nutzereingabe (Eigendarstellung)	35
Abbildung 13: Optimierungsmodell Output (Eigendarstellung)	35
Abbildung 14: Stepresponse Synthetische Strecke1 (Eigendarstellung)	43
Abbildung 15: Stepresponse Synthetische Strecke2 (Eigendarstellung)	44
Abbildung 16: Stepresponse Synthetische Strecke3 (Eigendarstellung)	46
Abbildung 17: Stepresponse Synthetische Strecke instabil (Eigendarstellung)	47
Abbildung 18: Stepresponse reale Strecke1 (Eigendarstellung)	48
Abbildung 19: Stepresponse Plot ML Strecke 1 (Eigendarstellung)	49
Abbildung 20: Stepresponse reale Strecke3 (Eigendarstellung)	50
Abbildung 21: Simulink Darstellung Temperaturreglung (Eigendarstellung mithilfe von Dipl.-Ing. Sameh Ben Nablia)	53
Abbildung 22: ML Temperaturreglung mit Störung Z1 Z2 (Eigendarstellung)	54
Abbildung 23: ZN Temperaturreglung mit Störung Z1 Z2 (Eigendarstellung)	55
Abbildung 24: Chr20% Temperaturreglung mit Störung Z1 Z2 (Eigendarstellung)	56
Abbildung 25: Chr0% Temperaturreglung mit Störung Z1 Z2 (Eigendarstellung)	57

Tabellenverzeichnis

Tabelle 1: Symbolic Regression Gleichungen	17
Tabelle 2: Vergleich der Modellgüte (R^2 , MAE, RMSE) von XGBoost, DGP und Symbolic Regression über verschiedene Streckentypen	39
Tabelle 3: DGP-Performance in unterschiedlichen Parameterregionen	40
Tabelle 4: Vergleich Leistungsparameter1	43
Tabelle 5: Vergleich Leistungsparameter2	44
Tabelle 6: Vergleich Leistungsparameter3	46
Tabelle 7: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 1 ..	49
Tabelle 8: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 2 ..	51
Tabelle 9: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 3 ..	52
Tabelle 10: Vergleich CHR/ML Synthetische Strecken	59
Tabelle 11: Vergleich CHR/ML reale Strecken	60

Abkürzungsverzeichnis

Abkürzung / Symbol	Ausgeschriebene Version / Bedeutung
AI/KI	Artificial Intelligence / Künstliche Intelligenz
C	Grenze / Constrain
CHR	Chien-Hrones-Reswick
CSV	Comma-Separated Values
D	Differentiell (Anteil des PID-Reglers)
DGP	Deep Gaussian Process
GP	Gaussian Process
I	Integral (Anteil des PID-Reglers)
ISE	Integrated Square Error (Integriertes quadratisches Fehlermaß)
IT ₁	Integrierer mit Verzögerung 1. Ordnung
K	Prozessverstärkung
LTI	Linear Time-Invariant (Linear zeitinvariant)
MAE	Mean Absolute Error (Mittlerer absoluter Fehler)
ML	Machine Learning (Maschinelles Lernen)
OS	Überschwingung / Overshoot
n	Anzahl der Datenpunkte
P	Proportional (Anteil des PID-Reglers)
PID	Proportional-Integral-Differential (Regler)
PSO	Particle Swarm Optimization
PT1	Proportionalglied 1. Ordnung (Verzögerungsglied)
PT2	Proportionalglied 2. Ordnung (Verzögerungsglied)
R ²	Bestimmtheitsmaß
RMSE	Root Mean Squared Error (Wurzel des mittleren quadratischen Fehlers)
RT	Anstiegszeit / Rise Time
Sim	Simulation
SSE	Steady-State Error (Stationärer Fehler)
ST	Einschwingzeit / Settling Time
T1, T2	Zeitkonstanten
u(t)	Zeitabhängiges Eingangssignal (Stellgröße)
w	Gewichtung / weight
XGBoost	Extreme Gradient Boosting
y _i	Tatsächlicher (wahrer) Wert
$\hat{y}_i$ (y-Dach)	Vorhergesagter Wert
$\bar{y}$ (y-quer)	Mittelwert aller tatsächlichen Werte
y(t)	Zeitabhängiges Ausgangssignal (Regelgröße)
ZN	Ziegler-Nichols
Z1, Z2	Störungsgröße

Glossar

In der gesamten Arbeit wird zur Darstellung von Dezimalzahlen der Punkt anstelle des im Deutschen üblichen Kommas verwendet. Diese Schreibweise entspricht dem internationalen wissenschaftlichen Standard sowie der Konvention in den für die machine learning relevanten Programmiersprachen und Software-Bibliotheken.

1 Abstract

Die klassische Parametrierung von PID-Reglern erfolgt in der Praxis überwiegend über heuristische Methoden wie Ziegler-Nichols (ZN) oder Chien-Hrones-Reswick (CHR). Diese Verfahren bieten einfache Einstellregeln, stoßen jedoch insbesondere bei nichtlinearen Prozessen, Unsicherheiten oder komplexen Dynamiken an ihre Grenzen. Diese Arbeit verfolgt das Ziel, Machine Learning Modelle zu entwickeln und systematisch hinsichtlich ihrer Eignung zur Optimierung von PID-Parametern zu untersuchen die sowohl Flexibilität gegenüber unterschiedlichen Prozessstrecken als auch eine interaktive Einbindung von Nutzerpräferenzen ermöglichen.

Hierzu wurde zunächst ein Datensatz synthetischer Teststrecken mithilfe von Matlab Simulink generiert, auf dessen Basis verschiedene Regressionsmodelle trainiert wurden. Die resultierenden Modelle sagen anhand von Streckenparametern passende Regelparameter voraus und resultierende Leistungskenngößen. Über eine interaktive Softwareoberfläche können Nutzer durch Gewichtung der Metriken und Definition von Grenzwerten gezielt Einfluss auf die Kostenfunktion der Optimierung nehmen.

Die Ergebnisse zeigen, dass das ML-basierte Optimierungsverfahren in Simulationsumgebungen mit klassischen Methoden vergleichbar ist und in vielen Szenarien zu einer ausgewogeneren und teilweise besseren Regelung führt. Besonders im Hinblick auf Robustheit gegenüber Störgrößen und die Flexibilität bei variierenden Prozessparametern erweist sich der Ansatz als vorteilhaft. Validierungen an realen Teststrecken bestätigen die Praxistauglichkeit und zeigen, dass die Modelle in vielen unterschiedlichen Umgebungen stabile Regler entwerfen kann.

Die Arbeit demonstriert damit, dass datengetriebene Verfahren ein vielversprechendes Werkzeug für die moderne Regelungstechnik darstellen und klassische Tuning-Methoden nicht nur ergänzen, sondern in vielen Anwendungsfällen auch übertreffen können.

2 Theoretische Grundlagen

In diesem Kapitel werden die wesentlichen Konzepte der Regelungstechnik kurz vorgestellt, die für das Verständnis von PID-Reglern und deren Optimierung notwendig sind. Dazu gehören die Beschreibung von Regelstrecken und Übertragungsfunktionen, grundlegende Stabilitäts- und Dynamikbetrachtungen sowie die Struktur, Funktionsweise und Parameterwirkung von PID-Reglern. Außerdem werden klassische Tuning-Methoden und relevante Performanzmetriken erläutert, die als Grundlage für die spätere Optimierung mittels Machine Learning dienen.

2.1 Regelungstechnische Grundlagen

Lineare zeitinvariante Systeme (LTI) bilden die Grundlage der klassischen Regelungstechnik. Sie zeichnen sich durch das Superpositionsprinzip sowie zeitinvariante Systemdynamiken aus und erlauben damit eine konsistente Beschreibung im Zeit- und Frequenzbereich. Die Charakterisierung erfolgt wahlweise über Differentialgleichungen, Impuls-, Sprungantworten oder im Laplace-Bereich durch Übertragungsfunktionen. Aufgrund dieser Eigenschaften lassen sich wesentliche Konzepte wie Stabilität, Frequenzgang und Reglerentwurf auf eine systematische Weise behandeln. Auch wenn reale Prozesse häufig nichtlinear oder zeitvariant sind, können sie im relevanten Arbeitspunkt durch LTI-Modelle hinreichend genau approximiert werden [\[1\]\[2\]](#).

2.1.1 Übertragungsfunktionen und Regelstrecken

Die Beschreibung von Regelstrecken erfolgt in der klassischen Regelungstechnik meist über Übertragungsfunktionen. Diese fassen das Eingang-Ausgangs-Verhalten eines Systems kompakt zusammen und ermöglichen eine direkte Analyse von Stabilität und Dynamik. Typische Modelle sind Standardglieder wie IT_N , P , PT_N und Totzeitglieder, die das Verhalten vieler technischer Prozesse näherungsweise beschreiben [\[1\]\[2\]](#).

Ein PT_1 -Glied stellt ein einfaches System erster Ordnung mit einer charakteristischen Verzögerung dar. Es zeigt ein glattes, nicht oszillierendes Verhalten und wird häufig zur Modellierung von Prozessen mit trägem Reaktionsverhalten genutzt. Das PT_2 -Glied erweitert

diese Beschreibung auf Systeme zweiter Ordnung, die abhängig von der Dämpfung sowohl ein sanftes Einschwingen als auch ein oszillierendes Verhalten aufweisen können. Oszillierende Regelglieder treten insbesondere bei schwach gedämpften Prozessen auf und sind durch periodische Schwankungen in der Systemantwort erkennbar. Ein System mit IT_1 -Verhalten zeichnet sich durch einen integrierenden Charakter aus, dessen dynamische Antwort zusätzlich durch eine Verzögerung erster Ordnung geprägt ist, wie es bei speichernden Prozessen, etwa in der Füllstandsreglung, auftritt [\[1\]\[2\]](#).

Diese Standardglieder ermöglichen eine kompakte und praxisnahe Beschreibung realer Prozesse und bilden damit die Grundlage für den Reglerentwurf.

2.1.2 Stabilität und Dynamik in Regelkreisen

Die Stabilität eines geschlossenen Regelkreises ist eine zentrale Voraussetzung für dessen Einsatz in der Praxis. Ein System gilt als stabil, wenn es nach einem Eingangssignal in einen stationären Endzustand zurückkehrt. Dieser Endzustand muss jedoch nicht zwingend mit der Führungsgröße übereinstimmen; ein bleibender stationärer Fehler ist möglich. Die Genauigkeit beschreibt dagegen, wie nah der stationäre Endwert an der gewünschten Führungsgröße liegt und wird typischerweise durch die Reglerstruktur, insbesondere den Einsatz von Integrationsanteilen, beeinflusst [\[1\]\[2\]](#).

Neben Stabilität und Genauigkeit ist die Dynamik entscheidend für die Bewertung der Regelgüte. Wichtige Aspekte sind Anstiegszeit, Ausregelzeit und Überschwingen, die das Verhalten des Systems auf Einflüsse beschreiben. Eine gute Regelung vereint schnelle Reaktionen mit minimalem Überschwingen und hoher Genauigkeit [\[1\]\[2\]](#).

2.2 PID-Regler

2.2.1 Struktur und Wirkungsweise (P-, I-, D-Anteil)

Der PID-Regler ist das am weitesten verbreitete Regelverfahren in der industriellen Praxis und kombiniert drei Anteile, um Stabilität, Geschwindigkeit und Genauigkeit einer Regelung zu gewährleisten [\[1\]\[2\]](#).

Der proportionale Anteil (P) reagiert auf die aktuelle Abweichung zwischen Soll- und Istwert und liefert damit einen unmittelbaren Blick in die Gegenwart des Systems. Er sorgt für eine schnelle Reaktion auf Änderungen, kann aber allein oft keinen bleibenden stationären Fehler korrigieren [\[3\]](#).

Der integrale Anteil (I) summiert vergangene Abweichungen auf und berücksichtigt damit die Historie des Systems. Durch die Korrektur vergangener Fehler eliminiert er den stationären Fehler und verbessert die Genauigkeit der Regelung [\[3\]](#).

Der differenzielle Anteil (D) reagiert auf die Änderungsrate des Fehlers und ermöglicht eine Vorschau auf die zukünftige Entwicklung. Er dämpft Überschwingen und sorgt für eine feinfühligere Anpassung an schnelle Änderungen, wodurch die Dynamik des Systems verbessert wird [\[3\]](#).

Die Kombination dieser drei Anteile erlaubt es dem PID-Regler, ein ausgewogenes Verhalten zwischen Geschwindigkeit, Stabilität und Genauigkeit zu erreichen. Die richtige Abstimmung von P-, I- und D-Anteil ist dabei entscheidend für die Performance der Regelung [\[3\]](#).

2.2.2 Einfluss der Parameter K_p , K_i und K_d

Die Regelgüte eines PID-Reglers wird maßgeblich durch die Abstimmung seiner Parameter bestimmt. Der Proportionalbeiwert K_p steuert die unmittelbare Reaktion auf aktuelle Abweichungen: Ein höherer Wert erhöht die Geschwindigkeit, kann jedoch zu größerem Überschwingen führen, während ein niedrigerer Wert die Reaktion sanfter, aber träger macht. Der Integralbeiwert K_i bestimmt, wie stark vergangene Fehler in die Regelung einfließen und damit der stationäre Fehler korrigiert wird; ein zu großer K_i -Anteil kann das System instabil oder oszillierend machen, während ein zu kleiner Anteil zu bleibenden Abweichungen führt.

Der Differentialbeiwert K_d beeinflusst die Vorschau auf zukünftige Änderungen und dient der Dämpfung von Überschwingen und Nachschwingern; eine zu starke Dämpfung kann die Reaktionsgeschwindigkeit verringern, während ein fehlender oder zu schwacher D-Anteil die durch den P-Anteil verursachten Schwingungen nicht ausreichend dämpft. Das Zusammenspiel dieser drei Parameter ermöglicht es, Geschwindigkeit, Stabilität und Genauigkeit gezielt auszubalancieren, wobei die optimale Kombination stets vom jeweiligen Prozessverhalten und den spezifischen Anforderungen an die Regelung abhängt [\[3\]](#).

2.3 Klassische Tuning-Methoden

2.3.1 Ziegler-Nichols-Verfahren

Das Ziegler-Nichols-Verfahren ist eine klassische Methode zur Parametrierung von PID-Reglern und basiert auf experimentellen Kennwerten der Regelstrecke. Es existieren zwei gängige Varianten: die Reaktionskurvenmethode, bei der die offene Regelstrecke auf einen Sprung getestet wird und aus der gemessenen Verzögerung und Zeitkonstanten die PID-Parameter abgeleitet werden, sowie die Kritische-Koeffizienten-Methode, bei der der Proportionalbeiwert im geschlossenen Regelkreis so lange erhöht wird, bis das System gerade noch stabil oszilliert, und aus der kritischen Verstärkung und Schwingungsperiode die Parameter berechnet werden. Beide Ansätze ermöglichen eine schnelle und einfache Anpassung des Reglers, berücksichtigen jedoch keine spezifischen Anforderungen an Genauigkeit oder Störverhalten und liefern daher nur Näherungswerte, die gegebenenfalls durch Feinabstimmung verbessert werden müssen [\[1\]\[2\]](#).

2.3.2 Chien-Hrones-Reswick-Methode

Die Chien-Hrones-Reswick-Methode (CHR) ist eine Weiterentwicklung der klassischen Ziegler-Nichols-Ansätze und wurde entwickelt, um das Überschwingen gezielt zu beeinflussen. Im Gegensatz zum ZN-Verfahren basiert CHR häufig auf der Analyse der Sprungantwort der Regelstrecke über das Wendetangentenverfahren, bei dem charakteristische Punkte der Kurve, wie Steigung und Wendepunkt, zur Berechnung der PID-Parameter genutzt werden. Auf diese Weise lassen sich die Parameter so einstellen, dass entweder ein aggressives Ansprechverhalten oder ein möglichst überschwingungsarmes (0%, 20% Überschwingung)

Verhalten erzielt wird. Die Methode ermöglicht damit eine gezieltere Abstimmung von Geschwindigkeit, Stabilität und Genauigkeit und bietet insbesondere für industrielle Prozesse eine praxisnahe Alternative zu den klassisch aggressiveren ZN-Einstellungen [\[1\]\[2\]](#).

2.3.3 Simulink PID-Autotuning

Das Simulink PID-Autotuning bietet eine automatisierte Möglichkeit, PID-Parameter direkt aus der Modell- oder Messdatenanalyse zu bestimmen. Dabei wird die Regelstrecke auf eine definierte Eingangsänderung, meist einen Sprung, angeregt, und die Software ermittelt die optimalen Reglerparameter basierend auf der Reaktion der Strecke. Simulink verwendet dafür numerische Algorithmen, die die Dynamik der Strecke analysieren und die PID-Parameter so anpassen, dass eine ausgewogene Balance zwischen Geschwindigkeit, Stabilität und Genauigkeit erreicht wird. Diese automatisierte Vorgehensweise ermöglicht eine schnelle und reproduzierbare Einstellung der Regler, ohne dass manuell aufwendige Experimente oder Näherungsverfahren erforderlich sind, und ist besonders für komplexe oder schwer messbare Systeme von Vorteil [\[4\]](#).

2.3.4 Limitierungen klassischer Tuningverfahren

Obwohl klassische PID-Tuning-Methoden wie Ziegler-Nichols, Chien-Hrones-Reswick oder Simulink-Autotuning in vielen industriellen Anwendungen erfolgreich eingesetzt werden, stoßen sie in der Praxis auf Grenzen. Sie gehen typischerweise von linearisierten oder gut modellierbaren Systemen aus und liefern Näherungswerte für die Reglerparameter. Nichtlineare, zeitvariante oder stark gestörte Prozesse werden dadurch oft nur unzureichend erfasst. Hier können moderne, datenbasierte Methoden wie maschinelles Lernen deutlich besser performen, da sie komplexe Zusammenhänge aus Mess- und Simulationsdaten direkt lernen und somit auch nichtlineare Dynamiken berücksichtigen [\[5\]\[6\]](#).

Ein weiteres Problem klassischer Verfahren ist ihre begrenzte Flexibilität. ZN oder CHR liefern feste Parameter, die manuell nachjustiert werden müssen und keine einfache Möglichkeit bieten, unterschiedliche Anforderungen an Geschwindigkeit, Überschwingen oder andere Performance-Metriken gezielt zu gewichten. Maschinelles Lernen hingegen erlaubt eine dynamische, multi-kriterielle Optimierung, bei der verschiedene Gütemaße in die Kostenfunktion einfließen und vom Nutzer individuell angepasst werden können [\[5\]\[6\]](#).

Darüber hinaus reagieren klassische Methoden empfindlich auf Streckendaten und Messfehler. Abweichungen in der Sprungantwort oder verrauschte Signale können die berechneten PID-Parameter stark verändern. ML-basierte Modelle können durch Training auf umfangreiche, diverse Datensätze robuster gegenüber solchen Unsicherheiten werden und so zuverlässigere Regelparameter liefern [5][6].

Schließlich sind klassische Methoden statisch ausgelegt. Änderungen im Prozess, wie Lastschwankungen, Materialänderungen oder Temperaturvariationen, erfordern ein erneutes Tuning. ML-gestützte Ansätze können hingegen anpassungsfähiger gestaltet werden und ermöglichen eine automatische, kontinuierliche Optimierung der Reglerparameter [5][6].

Insgesamt zeigen diese Limitationen, dass klassische Optimierungsstrategien zwar einfach und weit verbreitet sind, ihre Anwendbarkeit jedoch bei komplexen, dynamischen oder stark gestörten Prozessen eingeschränkt ist. Maschinelle Lernverfahren bieten hier die Möglichkeit, durch datenbasierte, flexible und multi-kriterielle Optimierung deutlich bessere Regelergebnisse zu erzielen [5][6].

2.4 Performanzmetriken in der Regelungstechnik

Die Bewertung der Qualität eines Regelkreises erfolgt über verschiedene Performanzmetriken, die Dynamik, Genauigkeit und Stabilität des Systems quantifizieren. Eine zentrale Größe ist der **integrierte quadratische Fehler** (ISE), der die Summe der quadrierten Abweichungen zwischen Soll- und Istwert über die Zeit darstellt. Für ein kontinuierliches System wird er durch

$$(1): ISE = \int_0^{\infty} e^2(t) dt$$

definiert, wobei $e(t) = y_{\text{Soll}}(t) - y(t)$ den momentanen Regelabweichung beschreibt. Der ISE gewichtet größere Fehler stärker und liefert ein Maß für die Gesamtgenauigkeit des Regelkreises [7].

Die **Überschwingweite** M_p oder auch *OS* beschreibt den maximalen Ausschlag des Ausgangs über den Sollwert, normiert auf den Endwert:

$$(2): M_p = \frac{y_{\text{max}} - y_{\text{end}}}{y_{\text{end}}} \cdot 100\%$$

Sie gibt Aufschluss über die Neigung des Systems zu Nachschwingern und ist ein Indikator für die Dämpfung [7].

Die **Anstiegszeit** t_r oder auch RT bezeichnet die Zeit, die der Ausgang benötigt, um von einem definierten unteren auf einen oberen Anteil des Sollwerts (z. B. 10 % bis 90 %) anzusteigen, und charakterisiert die Reaktionsgeschwindigkeit des Systems [7].

$$(3): t_r = t_{90\%} - t_{10\%}$$

Die **Einschwingzeit** t_s oder auch ST beschreibt die Dauer, bis der Ausgang nach einer Störung oder Führungsänderung dauerhaft innerhalb eines vorgegebenen Toleranzbereichs (häufig $\pm 2\%$ oder $\pm 5\%$ des Sollwerts) liegt [7].

$$(4): t_s = |y(t) - y_{\text{end}}| \leq \epsilon \cdot y_{\text{end}}, \quad \forall t \geq t_s$$

Der **stationäre Fehler** e_{stat} oder auch SSE gibt an, wie groß die verbleibende Abweichung zwischen Soll- und Istwert nach Erreichen des eingeschwungenen Zustands ist:

$$(5): e_{\text{stat}} = \lim_{t \rightarrow \infty} e(t)$$

Durch die Kombination dieser Metriken lässt sich die Regelgüte umfassend bewerten und unterschiedliche Regelstrategien klassische Tuning-Methoden wie ZN oder CHR sowie moderne, datenbasierte Optimierungen objektiv vergleichen [7].

3 Grundlagen des Machine Learning

Im Kapitel 3 „Grundlagen des Machine Learning“ werden die grundlegenden Konzepte und Paradigmen des maschinellen Lernens eingeführt und deren Relevanz für die Regelungstechnik aufgezeigt. Es behandelt insbesondere Supervised-Learning-Ansätze für die PID-Optimierung, erläutert die Auswahl geeigneter Modelle von Entscheidungsbaum-basierten Methoden über Symbolic Regression bis hin zu Gaussian Processes und stellt Verfahren zur Evaluierung der Modellgüte anhand statistischer Kennzahlen und Visualisierungen vor.

3.1 Einführung in Machine Learning

3.1.1 Grundprinzipien und Paradigmen

Machine Learning stellt ein Teilgebiet der Künstlichen Intelligenz (KI/AI) dar, das darauf abzielt, Algorithmen aus empirischen oder synthetischen Daten lernen zu lassen, anstatt Regeln ausschließlich analytisch oder heuristisch vorzugeben. Ein wesentliches Ziel besteht darin, Muster und Zusammenhänge in Daten zu erkennen, die für den Menschen nicht unmittelbar ersichtlich sind. Im Unterschied zu klassischen, modellbasierten Verfahren, die auf expliziten Gleichungen beruhen, basiert Machine Learning auf der Fähigkeit, statistische Abhängigkeiten zu erfassen und diese für Vorhersagen oder Entscheidungen zu nutzen. Ein trainiertes Modell kann die erlernten Zusammenhänge anschließend auf neue, zuvor unbekannte Daten anwenden und damit eine Generalisierungsfähigkeit zeigen [\[6\]\[8\]](#).

Grundsätzlich werden drei Lernansätze hierfür unterschieden, Supervised Learning, Unsupervised Learning und Reinforced Learning. Im Supervised Learning erfolgt das Lernen anhand von Trainingsdaten, die sowohl Eingaben als auch zugehörige Zielgrößen enthalten. Ziel ist es, eine Abbildung zwischen Eingabe- und Ausgaberaum zu approximieren. Ein klassisches Beispiel ist die Vorhersage des Preises von Immobilien auf Basis von Merkmalen wie Lage, Größe oder Baujahr. Das Modell lernt, welche Faktoren den Preis beeinflussen, und kann für neue Immobilien Vorhersagen treffen. Dieses Verfahren kommt vorallem dann zum Einsatz, wenn es bereits Belastbare empirische Vergleichsdaten in diesem Bereich gibt, die für das ursprüngliche Training genutzt werden können [\[6\]](#).

Beim Unsupervised Learning liegen keine Zielwerte vor. Stattdessen wird versucht, verborgene Strukturen oder Gruppierungen in den Daten zu identifizieren. Typische Methoden sind Clustering oder Dimensionsreduktion. Ein Beispiel ist die Kundensegmentierung im Marketing, bei der Kunden anhand ihres Kaufverhaltens oder demografischer Daten in Gruppen mit ähnlichen Eigenschaften eingeteilt werden, ohne dass vorher definiert ist, welche Gruppe „richtig“ ist. Diese Methode wird meist genutzt, wenn es keine klare Abgrenzung zwischen „richtig“ und „falsch“ gibt und die Gruppierungsübergänge fließend sind [\[7\]](#).

Das dritte Paradigma, Reinforcement Learning, beruht auf Interaktion mit einer Umgebung. Ein Agent wählt sukzessive Aktionen aus und erhält Belohnungen oder Strafen, wodurch er eine Strategie zur Maximierung der langfristigen Belohnung entwickelt. Ein praxisnahes Beispiel besteht darin, dass ein Programm in einem Computerspiel durch Versuch und Irrtum lernt, wie es ein Level effizient meistert, indem es für gute Spielzüge Punkte erhält und für Fehler bestraft wird [\[7\]](#).

Für die vorliegende Arbeit ist insbesondere Supervised Learning von Bedeutung, da es eine direkte Abbildung von Prozessparametern und Reglerkoeffizienten auf Leistungsmetriken wie Überspringen oder ISE erlaubt und somit eine Grundlage für die datengetriebene Optimierung von PID-Reglern bildet. Dieser Lernansatz wird im [Abschnitt 3.2 Supervised Learning für Regelungstechnik](#) ausführlicher behandelt.

3.1.2 Relevanz für die Regelungstechnik

Die zunehmende Digitalisierung und Vernetzung von Produktionsanlagen im Rahmen von Industrie 4.0 führt zu einer erheblichen Zunahme an Prozessdaten, die traditionelle Regelungstechniken nur begrenzt ausnutzen können [\[9\]](#). Machine Learning bietet hier die Möglichkeit, diese Daten systematisch auszuwerten und daraus Optimierungen der Reglerparameter abzuleiten, ohne dass explizite Modellgleichungen für jede Anlage notwendig sind. ML-gestützte Verfahren können beispielsweise eingesetzt werden, um die PID-Regler komplexer chemischer, energie- oder verfahrenstechnischer Anlagen automatisiert zu optimieren, in denen zahlreiche parallele Prozesse gleichzeitig gesteuert werden müssen. Dabei werden Reglerparameter kontinuierlich angepasst, wodurch Energie- und Personalkosten gesenkt sowie potenzielle Fehler oder Ermüdungserscheinungen frühzeitig erkannt werden können, was einen Beitrag zur vorausschauenden Wartung leistet (Predictive

Maintenance) leistet [\[9\]\[10\]](#), sowie den menschlichen Fehler reduziert. Ein wesentlicher Vorteil von ML liegt in der Fähigkeit zum Live-Tuning. Modelle können während des Betriebs große Anlagen überwachen und Reglerparameter dynamisch anpassen, um eine gewünschte Reaktionsgeschwindigkeit, minimalen Überschwingverhalten oder kurze Einschwingzeiten zu erreichen [\[6\]](#). Dies erlaubt eine kontinuierliche Verbesserung der Regelperformance, ohne dass ein Ingenieur jede Anpassung manuell vornehmen muss.

Darüber hinaus bieten ML-Methoden eine hohe Flexibilität und Retrainierbarkeit. Modelle können leicht auf firmeneigene Datensätze angepasst oder retrainiert werden, wodurch sie spezifische Anforderungen einzelner Prozesse berücksichtigen können. Beispielsweise lässt sich der Fokus gezielt auf bestimmte Leistungsmetriken legen, wie Überschwingen, Rise Time oder Settling Time, je nach Priorität des Betreibers. Hierdurch entsteht auch eine große Transparenz hinsichtlich des Trainingsverfahrens, während jedoch einzelne Vorhersagen nach dem Blackbox Prinzip entstehen [\[8\]\[10\]](#).

Im Vergleich zu klassischen PID-Tuning-Methoden wie Ziegler-Nichols, CHR oder Autotune zeichnen sich ML-Ansätze durch Automatisierung, Geschwindigkeit und geringe Kosten aus. Viele Algorithmen sind frei verfügbar, benötigen keine teuren Lizenzen im Vergleich zu Matlab/Simulink und können automatisch große Mengen synthetischer oder realer Daten auswerten im Vergleich zu händischer Zn/Chr Einstellung, wodurch die Optimierung in kurzer Zeit erfolgt [\[7\]](#).

Schließlich zeigt sich die praktische Relevanz von ML auch in verwandten Anwendungen außerhalb der direkten PID-Optimierung. So können Prozesse hinsichtlich Qualitätskennzahlen vorhergesagt, Anlagen nach dynamischem Verhalten gruppiert oder Wartungsmaßnahmen durch Datenanalysen effizient geplant werden [\[8\]](#). Diese Beispiele verdeutlichen, dass Machine Learning nicht nur ein theoretisches Konzept, sondern ein leistungsfähiges Werkzeug für die moderne Regelungstechnik darstellt, das klassische Methoden sinnvoll ergänzt [\[9\]\[10\]](#).

3.2 Supervised Learning für Regelungstechnik

3.2.1 Lernprinzip

Im Supervised Learning wird ein Modell anhand eines Datensatzes trainiert, in dem Eingaben und die zugehörigen Zielgrößen bekannt sind. Für die PID-Optimierung besteht der Datensatz typischerweise aus Streckenparametern (z. B. Zeitkonstanten, Verstärkung) und den zugehörigen PID-Koeffizienten, ergänzt durch die daraus resultierenden Leistungsmetriken wie Überschwingen, ISE oder Einschwingzeit. Ziel des Trainings ist es, eine Abbildung von Prozess- und Reglerparametern auf die Performance zu lernen, sodass das Modell Vorhersagen für zuvor unbekannte Strecken treffen kann [\[6\]](#).

Die verfügbaren Daten werden üblicherweise nach festen Prozentsätzen in Trainings-, Test- und Evaluationsdatensätze aufgeteilt, typischerweise 80 % für das Training, 15 % für Test und 5 % für Evaluation. Das Modell erhält die Trainingsdaten, bestehend aus Streckenparametern und den zugehörigen PID-Koeffizienten, um die zugrunde liegenden Zusammenhänge zu erkennen. Die Testdaten werden während des Trainings genutzt, um die Performance auf bisher ungesesehenen Daten zu überwachen. In jedem Trainingszyklus (Epoch) werden Vorhersagen für die Testdaten erzeugt und mit den tatsächlichen Zielgrößen verglichen. Diese Rückmeldung ermöglicht es, die Hyperparameter oder das Trainingsverhalten zu steuern, beispielsweise durch Anpassung der Lernrate oder Early Stopping, um Überanpassung (Overfitting) zu vermeiden [\[7\]\[10\]](#).

Mit zunehmender Anzahl der Trainingszyklen lernt das Modell schrittweise, die Abbildung zwischen Eingaben und Zielgrößen genauer zu approximieren. Zunächst verbessert sich die Performance auf den Trainingsdaten stark, während die Testdaten nur moderat profitieren. Durch die kontinuierliche Überprüfung der Testperformance wird sichergestellt, dass das Modell nicht nur die Trainingsdaten auswendig lernt, sondern auch auf unbekannte Strecken generalisiert. Eine zu geringe Anzahl an Zyklen kann zu unteranpassung (Underfitting) führen, während eine zu hohe Zahl ohne geeignete Regularisierung zu Overfitting führen kann [\[7\]\[10\]](#).

Unter- oder Überanpassung beeinflusst die Modellleistung erheblich und ist eine große Herausforderung im Machine Learning Bereich. Underfitting tritt auf, wenn das Modell die zugrunde liegenden Zusammenhänge nicht ausreichend erlernt und folglich weder auf Trainings- noch auf Testdaten genaue Vorhersagen liefert. Hierbei sind die Vorhersagen des

Algorithmus oft Schlechter oder vergleichbar mit blindem Raten. Overfitting tritt auf, wenn das Modell die Trainingsdaten zu stark auswendig lernt und dadurch die Generalisierungsfähigkeit auf unbekannte Strecken stark eingeschränkt ist, häufig liegt in diesem Fall eine gute Performance in den Training/Testdaten vor, jedoch bei Evaluierungsdaten oder realen Daten treten große Unstimmigkeiten auf. In beiden Fällen kann die PID-Optimierung unzuverlässige oder suboptimale Reglerparameter liefern, was zu ineffizientem Regelverhalten oder instabilen Systemen führt [\[6\]\[7\]\[10\]](#). Nach Abschluss des Trainings werden die Evaluationsdaten verwendet, um die objektive Leistung des endgültigen Modells unabhängig von Trainings- und Testdaten zu bewerten. Evaluationsdaten sind dabei Kombinationen an Daten, die dem Modell absichtlich vorenthalten wurden und die der Algorithmus daher nicht kennt. Dieser Schritt ist entscheidend, um sicherzustellen, dass das Modell zuverlässig Vorhersagen auf neuen, unbekannten Strecken treffen kann und somit eine robuste Grundlage für die Optimierung von PID-Reglern bietet und nicht einfach nur Kombinationen auswendig lernt [\[7\]\[10\]](#).

Durch diesen iterativen Ansatz kann das Modell neue Streckenparameter vorhersagen und die entsprechenden PID-Reglerwerte schätzen, ohne dass für jede neue Strecke eine vollständige Simulation erforderlich ist. Dies ermöglicht eine schnelle, datengetriebene Optimierung und die gezielte Priorisierung bestimmter Leistungsmetriken, wie Reaktionsgeschwindigkeit oder Überschwingen, nach den Anforderungen des Betreibers [\[10\]](#).

3.2.2 Anwendungsfälle in der PID-Optimierung

Machine Learning gestützte Verfahren zur Optimierung von PID-Reglern finden in verschiedenen industriellen Anwendungen zunehmend Einsatz. In der Öltrennungsprozesskontrolle bei Tengizchevroil wurden beispielsweise Algorithmen wie Cuckoo Search, Firefly Algorithm, Particle Swarm Optimization und Support Vector Machines genutzt, um die PID-Parameter zu optimieren. Die Implementierung dieser Methoden führte zu einer deutlich verbesserten Dynamik des Systems, einschließlich reduzierten Überschwingens und schnelleren Einschwingzeiten im Vergleich zu klassischen Abstimmungsverfahren [\[11\]](#).

Ein weiteres Beispiel ist die Temperaturregelung in Glasöfen, bei der ein modelbasierter PID-Regler eingesetzt wurde. Durch die präzisere Steuerung der Temperatur konnte der

Energieverbrauch um etwa 50 % gesenkt und gleichzeitig die Produktqualität signifikant verbessert werden [\[12\]](#).

Darüber hinaus zeigt sich der Nutzen von Machine Learning auch bei hochspezialisierten Anwendungen, etwa bei optischen Schaltern. Hier ermöglichte ein Deep Reinforcement Learning-Modell eine fünffache Verbesserung der Schaltgeschwindigkeit und eine Verringerung der Leistungsinkonsistenz um 75 %, wobei das Modell die PID-Parameter in Echtzeit generieren konnte [\[13\]](#).

Der Einsatz von Machine Learning wächst derzeit in nahezu allen Industriezweigen kontinuierlich, wobei insbesondere in der Regelungstechnik und im Kontext von Industrie 4.0 zunehmend datengetriebene Ansätze zur Optimierung von Prozessabläufen und zur automatischen Anpassung von Reglern implementiert werden [\[9\]](#).

3.3 Auswahl relevanter Modelle

3.3.1 Decision Tree Methoden (Random Forrest / XGBoost)

Decision Trees gehören zu den klassischen Verfahren des Supervised Learning und stellen eine vergleichsweise einfache, aber dennoch sehr leistungsfähige Methode dar, die in vielen praktischen Anwendungen als Stand der Technik etabliert ist. Ein einzelner Decision Tree zerlegt den Eingaberaum schrittweise in Teilbereiche, indem Entscheidungsregeln („Splits“) auf Basis derjenigen Variablen gewählt werden, die die größte Reduktion einer Gütefunktion (z. B. Varianzreduktion oder Informationsgewinn) bewirken, siehe Abbildung 1: Decision Tree. Das Resultat ist ein hierarchisches Modell, das Eingaben zu Vorhersagen abbildet und dadurch auch nichtlineare Zusammenhänge erfassen kann [\[7\]](#). Vorteile sind insbesondere die Interpretierbarkeit sowie die Eignung für tabellarische Daten, wie sie in Form von numerischen Prozessparametern vorliegen. Nachteile bestehen hingegen in der hohen Varianz und der Anfälligkeit für Overfitting, wenn einzelne Bäume zu tief wachsen [\[14\]](#).

Um diese Schwächen auszugleichen, wurden Ensemble-Methoden entwickelt, die viele Decision Trees kombinieren. Random Forest ist hierbei ein prominenter Ansatz, bei dem zahlreiche Bäume unabhängig voneinander trainiert werden. Jeder Baum erhält eine zufällige Stichprobe der Trainingsdaten sowie eine zufällige Auswahl von Eingangsvariablen (Bagging). Die endgültige Vorhersage ergibt sich als Mittelwert (Regression) oder Mehrheitsentscheidung

(Klassifikation) aller Bäume. Durch diese Aggregation sinkt die Varianz deutlich, wodurch ein robustes Modell entsteht, das in vielen industriellen Anwendungen zuverlässig eingesetzt wird [\[14\]](#).

Einen anderen Ansatz verfolgt das Gradient Boosting, das mit Frameworks wie XGBoost inzwischen als einer der leistungsfähigsten Algorithmen für tabellarische Daten gilt. Im Unterschied zum Random Forest werden die Bäume nicht parallel, sondern sequentiell trainiert. Jeder neue Baum wird darauf ausgerichtet, die Fehler („Residuals“) der bisherigen Vorhersage zu korrigieren. Konkret bedeutet dies, dass der neue Baum die Differenz zwischen den tatsächlichen Zielwerten und den bisherigen Modellvorhersagen approximiert. Durch die sukzessive Addition dieser Korrekturen entsteht ein Ensemble, das mit jedem weiteren Baum eine bessere Approximation der Zielfunktion erreicht. Ergänzt wird dieser Ansatz durch Regularisierungstechniken, effiziente Speicherverwaltung und Parallelisierbarkeit, die XGBoost zu einem hochskalierbaren Verfahren machen [\[15\]](#).

Insgesamt zeigen baumbasierte Methoden eine bemerkenswerte Leistungsfähigkeit bei numerischen und tabellarischen Datensätzen, die im Gegensatz zu komplexeren Deep-Learning-Verfahren oft einfacher zu trainieren und zu interpretieren sind [\[14\]\[15\]](#). In der vorliegenden Arbeit sind sie daher besonders relevant, da die Trainingsdaten in Form von Streckenparametern und Reglerkoeffizienten in tabellarischer Struktur vorliegen und eine präzise, robuste Vorhersage erforderlich ist.

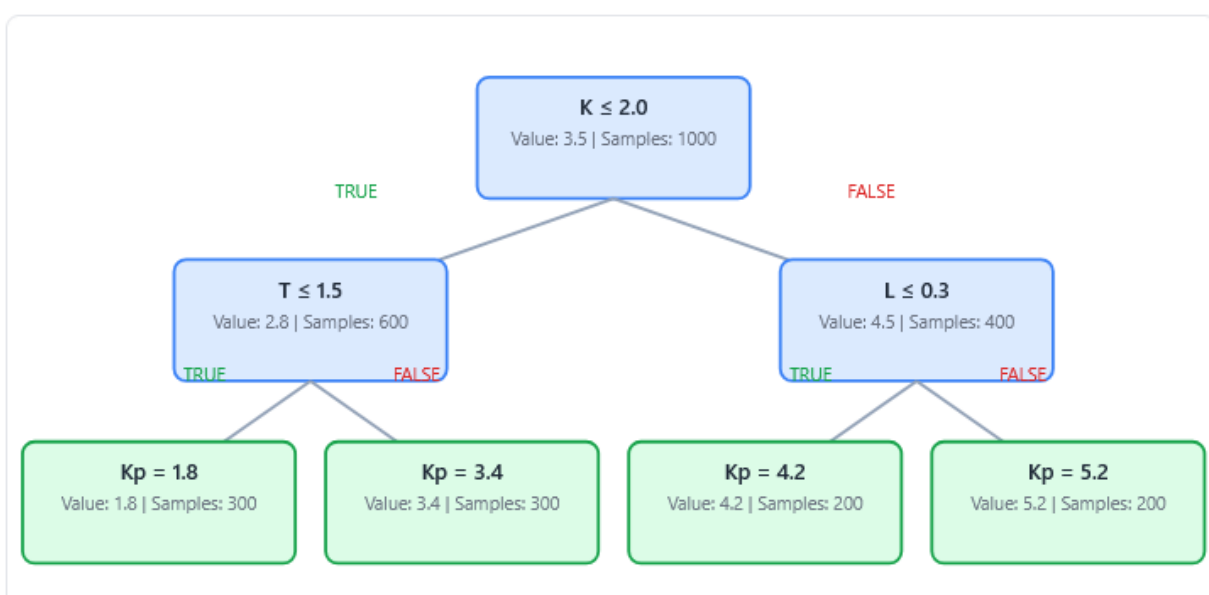


Abbildung 1: Decision Tree (Eigendarstellung)

3.3.2 Symbolic Regression

Symbolic Regression ist eine spezielle Form des maschinellen Lernens, die darauf abzielt, nicht nur Vorhersagen für Datenpunkte zu liefern, sondern eine explizite mathematische Funktion zu identifizieren, welche die zugrunde liegenden Zusammenhänge beschreibt. Im Gegensatz zu klassischen Regressionsmethoden, bei denen die Funktionsform (linear, polynomial) vorab definiert werden muss, sucht Symbolic Regression frei im Raum möglicher mathematischer Ausdrücke nach derjenigen Struktur, die die Trainingsdaten am besten erklärt [\[16\]](#).

Technisch wird dies häufig durch Genetic Programming (GP) umgesetzt. Dabei werden mathematische Ausdrücke wie Bäume kodiert, deren Knoten Operatoren (+, -, *, /, sin, exp, etc) enthalten, während die Blätter Variablen oder Konstanten repräsentieren. Ein initialer Population von Kandidatengleichungen wird erzeugt und im Verlauf vieler Iterationen mittels evolutionärer Operatoren (Mutation, Crossover, Selektion) verbessert. Als Fitnessmaß dient meist der Vorhersagefehler (z. B. mittlere quadratische Abweichung). Auf diese Weise entstehen Modelle, die nicht nur hohe Vorhersagegüte erreichen können, sondern zugleich eine explizite und interpretierbare Form besitzen [\[16\]](#).

Die wesentlichen Vorteile von Symbolic Regression sind die hohe Interpretierbarkeit der resultierenden Modelle sowie die Möglichkeit, physikalisch sinnvolle Strukturen und Zusammenhänge zu entdecken, die über reine Black-Box-Verfahren hinausgehen. Damit bietet sich ein Potenzial für wissenschaftliche und ingenieurtechnische Anwendungen, in denen nicht nur Prognose, sondern auch Modellverständnis entscheidend ist. Nachteile sind jedoch der im Vergleich zu anderen Verfahren hohe Rechenaufwand und die Gefahr, in sehr großen Suchräumen ineffizient zu explorieren. Moderne Implementierungen wie Eureqa, gplearn oder PySR nutzen daher heuristische Beschränkungen und Parallelisierung, um die Suche praktikabel zu machen [\[16\]](#).

Für die PID-Optimierung ist Symbolic Regression insofern interessant, als dass sich damit kompakte analytische Beziehungen zwischen Prozessparametern (z. B. Verstärkung, Zeitkonstanten, Totzeiten) und optimalen Reglerkoeffizienten (K_p , K_i , K_d) aufstellen lassen, siehe Tabelle 1: Symbolic Regression Gleichungen. Während datengetriebene Modelle wie Random Forest oder XGBoost primär für Prognosen eingesetzt werden, bietet Symbolic Regression zusätzlich die Chance, neue explizite Tuning-Regeln zu gewinnen, die über klassische Heuristiken wie Ziegler–Nichols hinausgehen.

Tabelle 1: Symbolic Regression Gleichungen

Parameter	Gefundene Gleichung (vereinfacht)
K_p	$= -0.4346 \cdot \ln(K) - 0.0637 \cdot \log \left(\log \left(\frac{0.982}{T_1/T_2} + T_1/T_2 \right) \right)$
K_i	$= 0.4235 - 0.4364 \cdot \ln(K) \left(K + \frac{K}{\sqrt{T_1/T_2 + 1.1168}} \right)$
K_d	$= -1.343 - \frac{3.260}{T_1(K + (T_1/T_2) \cdot T_2)}$

3.3.3 Gaussian Processes und Deep Gaussian Processes

Gaussian Processes (GP) stellen ein nichtparametrisches, probabilistisches Modell dar, das darauf abzielt, eine zugrunde liegende Funktion aus Trainingsdaten zu lernen. Im Gegensatz zu klassischen, deterministischen Modellen liefert ein GP nicht nur eine einzelne Vorhersage, sondern eine Verteilung der möglichen Werte, die Mittelwert und Unsicherheit umfasst. Diese Eigenschaft erlaubt, die Verlässlichkeit von Vorhersagen für neue, bislang unbekannte Streckenparameter abzuschätzen. In einem typischen Setting werden Eingaben wie Prozessparameter in die GP-Regression eingespeist, während die Zielgrößen die optimalen PID-Koeffizienten K_p , K_i und K_d darstellen. Die Kovarianzfunktion (Kernel) des GP definiert, wie stark verschiedene Eingaben miteinander verknüpft sind, und gewichtet die Trainingspunkte für die Vorhersage neuer Daten entsprechend ihrer Ähnlichkeit [\[17\]](#).

Für komplexere, stark nichtlineare Zusammenhänge, wie sie bei verfahrenstechnischen Prozessen häufig auftreten, werden Deep Gaussian Processes (DGP) eingesetzt. Diese erweitern das GP-Prinzip um mehrere Schichten, vergleichbar mit neuronalen Netzen,

wodurch das Modell in der Lage ist, hochkomplexe Abbildungen zwischen Prozessparametern und Reglerkoeffizienten zu approximieren. Dabei wird jeder Layer als GP modelliert, dessen Output als Input für die nächste Schicht dient. Dies ermöglicht die Erfassung von nichtlinearen Kopplungen, die mit einfachen GPs oder klassischen Regressionsmethoden schwer abzubilden wären [\[18\]](#).

3.4 Evaluierung von ML-Modellen

3.4.1 Statistische Gütemaße (R^2 , MAE, RMSE)

Die Güte von Machine-Learning-Modellen kann mit verschiedenen statistischen Kennzahlen beschrieben und bewertet werden. Für regressionale Vorhersagen, wie sie bei der PID-Optimierung vorkommen, sind vorallem R^2 , Mean Absolute Error (MAE) und Root Mean Squared Error (RMSE) gebräuchlich [\[6\]\[7\]\[8\]](#):

R^2 – Bestimmtheitsmaß:

$$(6): R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

R^2 , das sogenannte Bestimmtheitsmaß, gibt an, welcher Anteil der Varianz der Zielgröße durch das Modell erklärt wird. Ein R^2 -Wert von 1 bedeutet, dass das Modell alle Variationen der Daten perfekt vorhersagt, während ein Wert von 0 zeigt, dass das Modell keine Varianz erklären kann. Für die PID-Optimierung zeigt R^2 , wie gut die Modellvorhersagen der Reglerkoeffizienten K_p , K_i und K_d mit den tatsächlichen, optimalen Werten übereinstimmen [\[6\]\[7\]\[8\]](#).

MAE – Mean Absolute Error:

$$(7): MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Der MAE gibt die durchschnittliche absolute Abweichung der Vorhersagen von den tatsächlichen Werten an. Er ist robust gegenüber einzelnen Ausreißern und liefert eine leicht interpretierbare Fehlergröße in den Einheiten der Zielvariable [\[6\]\[7\]\[8\]](#).

RMSE – Root Mean Squared Error:

$$(8): RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Der RMSE gewichtet größere Abweichungen stärker als der MAE, was besonders hilfreich ist, wenn größere Fehler in der Regelung kritischer sind, z. B. bei Über- oder Unterschwingungen von Prozessen [\[6\]\[7\]\[8\]](#).

3.4.2 Visualisierung durch Vorhersage-vs.-Wahrheit-Analysen

Die Visualisierung von Modellvorhersagen im Vergleich zu den tatsächlichen Zielwerten ist ein zentrales Instrument zur qualitativen Beurteilung der Leistungsfähigkeit von Machine-Learning-Modellen. Bei der PID-Optimierung werden beispielsweise die vorhergesagten Reglerkoeffizienten K_p , K_i und K_d gegen die bekannten optimalen Werte aus dem Trainingsdatensatz aufgetragen. Ein gängiges Format ist das Scatter-Plot-Diagramm, in dem jede Punktwolke die Vorhersage eines Modells für einen bestimmten Datensatz darstellt. Idealerweise liegen die Punkte entlang der Diagonalen, was eine exakte Übereinstimmung zwischen Modellvorhersage und Realität signalisiert [\[6\]\[7\]](#).

Abweichungen von der Diagonale zeigen systematische oder zufällige Fehler auf, die Rückschlüsse auf Unter- oder Überanpassung (Under- bzw. Overfitting) zulassen. Dabei können unterschiedliche Farbcodierungen oder Markerstile verwendet werden, um Trainings-, Test- und Evaluationsdaten zu unterscheiden. Zusätzlich lassen sich durch Regressionslinien, Fehlerbänder oder Konfidenzintervalle die Genauigkeit und Unsicherheit der Vorhersagen visuell erfassen [\[6\]\[7\]](#).

Ein weiteres nützliches Werkzeug ist die Residuenanalyse, bei der die Differenz zwischen Vorhersage und tatsächlichem Wert gegen die Eingangsgrößen aufgetragen wird. Solche Diagramme helfen, nichtlineare Fehlerstrukturen oder Bias im Modell zu identifizieren und sind besonders wertvoll, wenn mehrere Modelle hinsichtlich ihrer Vorhersagequalität verglichen werden sollen [\[6\]\[7\]](#).

In der Praxis zeigt sich, dass die Kombination aus Scatter-Plots und Residuenplots eine intuitive, schnelle und leicht interpretierbare Übersicht über die Modellqualität bietet. Für die vorliegende Arbeit werden diese Methoden verwendet, um die Leistungsfähigkeit verschiedener ML-Modelle auf synthetischen und realen Strecken quantitativ und qualitativ zu evaluieren.

Abbildung 2: Vorhersage vs Wahrheit ideal zeigt ein ideales Vorhersage-vs.-Wahrheit-Diagramm für die PID-Koeffizienten K_p , K_i und K_d . Jeder Punkt repräsentiert eine Vorhersage des trainierten Surrogatmodells für eine bestimmte Prozessstrecke gegenüber dem tatsächlichen, bekannten optimalen Wert. Die Punkte liegen nahezu vollständig auf der Diagonalen, was eine exakte Übereinstimmung zwischen Modellvorhersage und Realität signalisiert. Diese Darstellung illustriert die Fähigkeit des Modells, die Zusammenhänge zwischen Streckenparametern und optimalen Reglerkoeffizienten präzise zu erfassen, und zeigt die hohe Güte der Regression sowohl für Trainings- als auch für Testdaten.

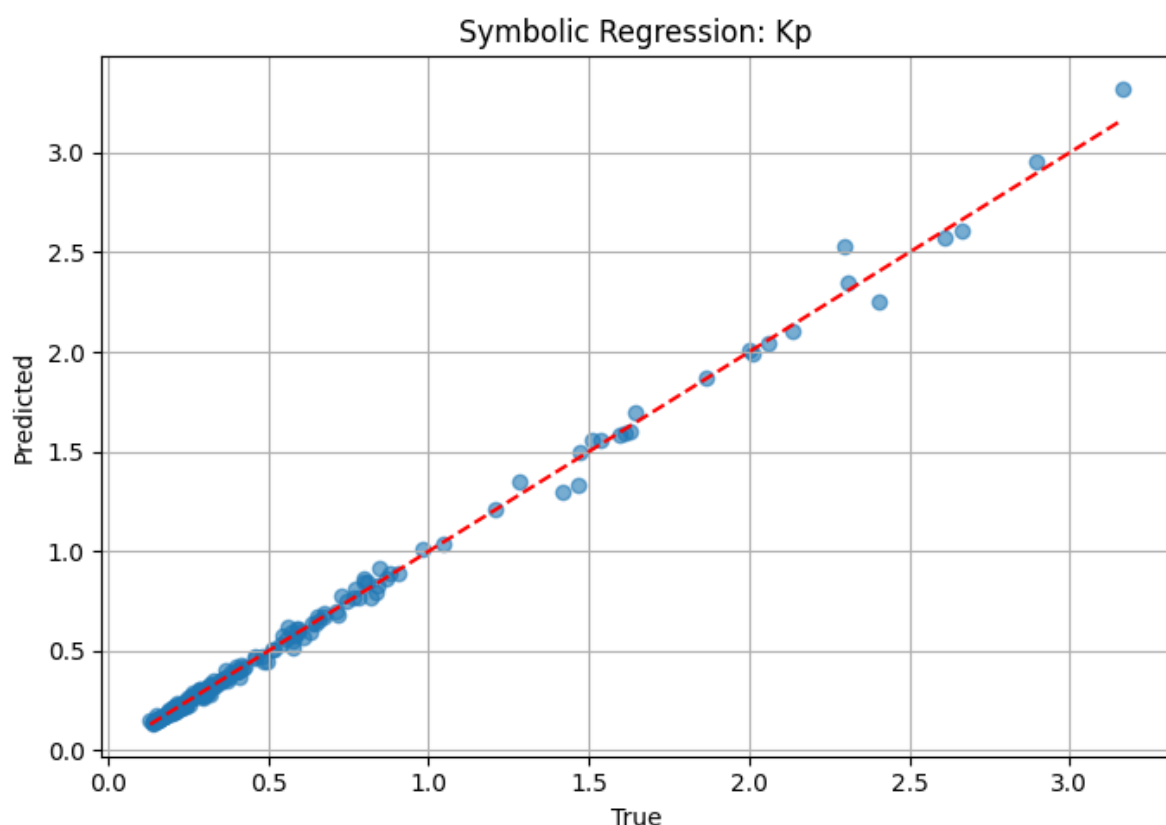


Abbildung 2: Vorhersage vs Wahrheit ideal (Eigendarstellung)

Abbildung 3: Vorhersage vs Wahrheit große Streuung zeigt ebenfalls ein Vorhersage-vs.-Wahrheit-Diagramm für ein Modell mit größerer Streuung. Hier weichen die Punkte deutlich von der Diagonalen ab, was systematische oder zufällige Vorhersagefehler verdeutlicht. Solche Abweichungen können auf Unter- oder Überanpassung des Modells, unzureichende Trainingsdaten oder stark nichtlineare Kopplungen hinweisen, die das Modell nicht vollständig erfassen konnte. Diese Visualisierung ist besonders nützlich, um Schwächen des Modells zu identifizieren und gezielte Optimierungen, etwa durch Hyperparameteranpassungen oder Datenerweiterung, einzuleiten.

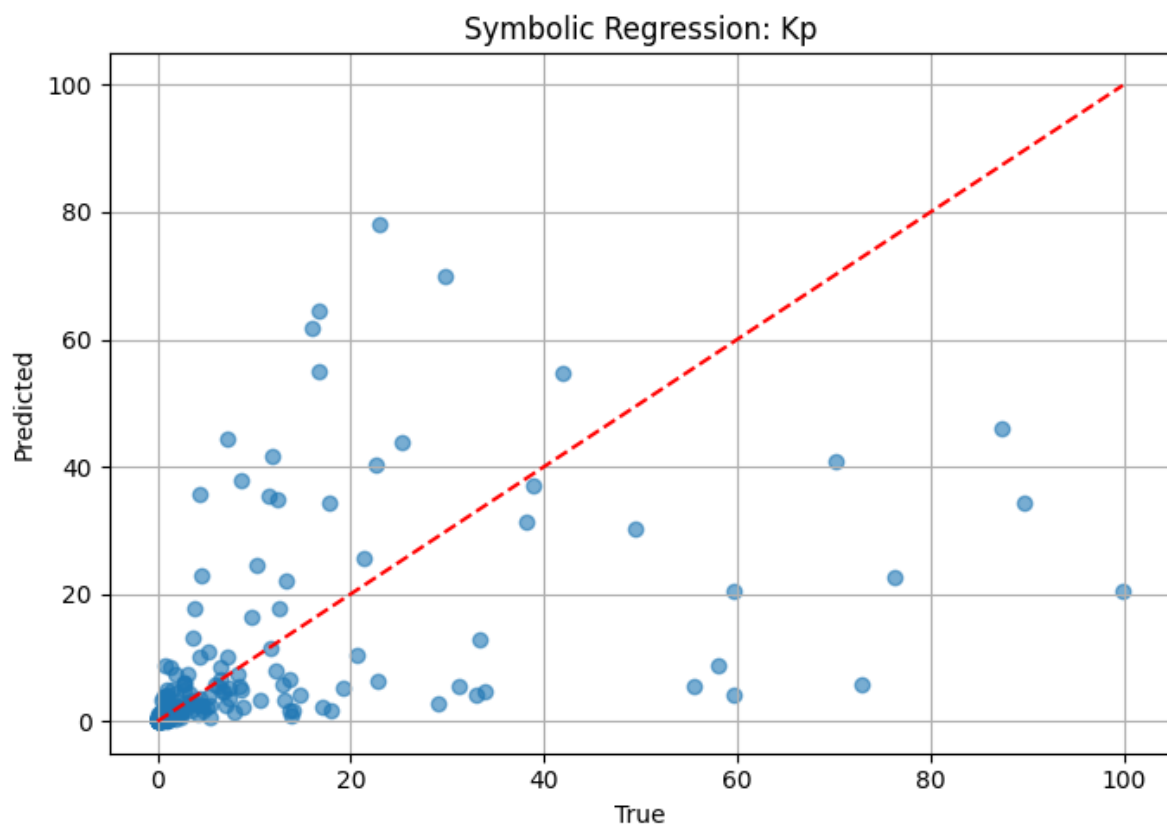


Abbildung 3: Vorhersage vs Wahrheit große Streuung (Eigendarstellung)

Abbildung 4: Vorhersage vs Wahrheit mit Konfidenzintervall zeigt ein Vorhersage-vs.-Wahrheit-Diagramm der PID-Koeffizienten mit vergleichsweise engen Konfidenzintervallen. Die Punkte liegen nicht nur nahe der Diagonalen, sondern auch die Unsicherheitsbereiche um die Vorhersagen sind klein. Dies deutet auf ein Modell hin, das nicht nur präzise, sondern auch konsistent in seinen Schätzungen ist. Die geringe Streuung der Konfidenzintervalle erhöht das Vertrauen in die Stabilität der Vorhersagen und die Robustheit des zugrundeliegenden Surrogatmodells.

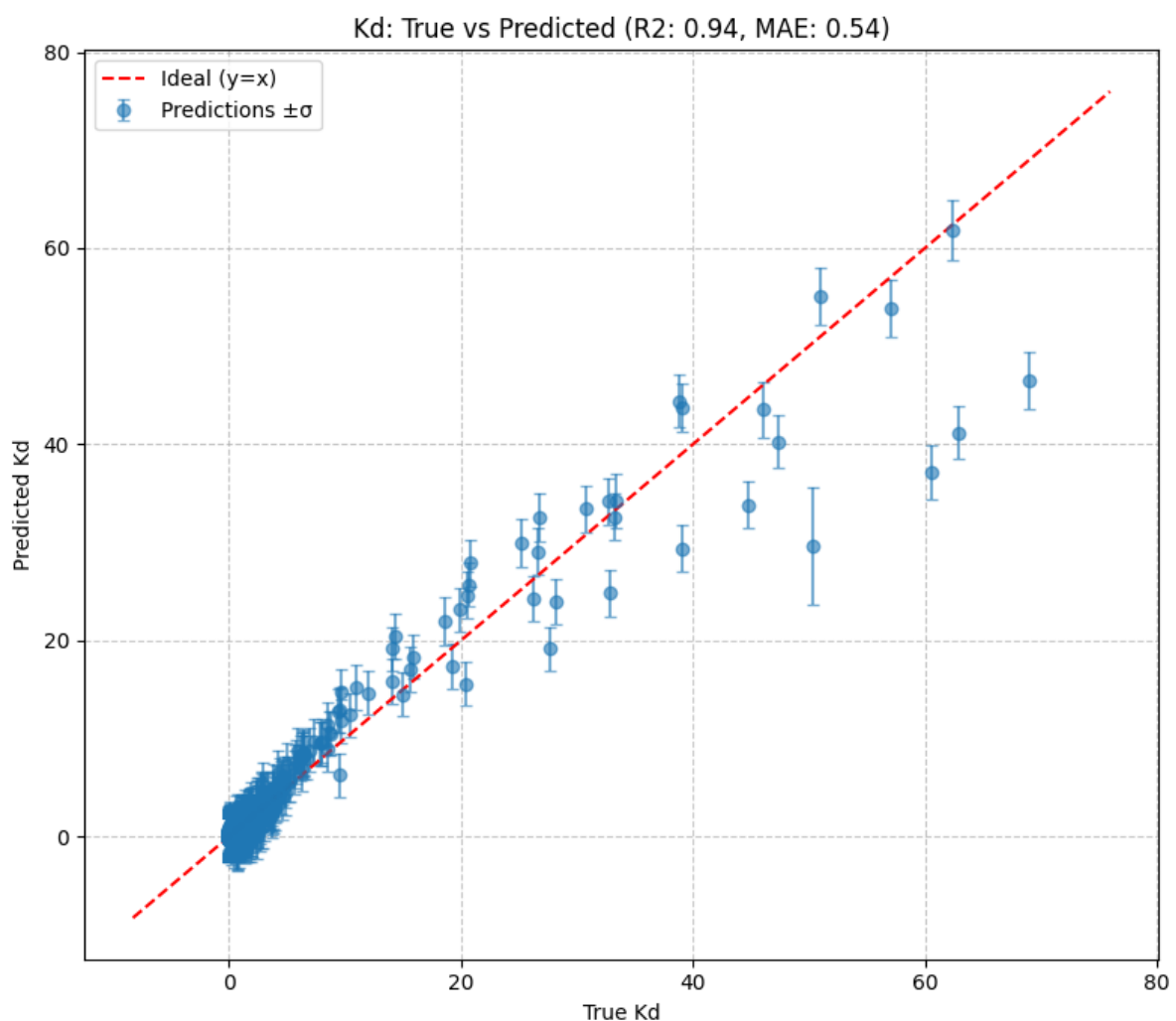


Abbildung 4: Vorhersage vs Wahrheit mit Konfidenzintervall (Eigendarstellung)

Abbildung 5: Vorhersage vs Wahrheit mit Konfidenzintervall große Streuung zeigt ein ähnliches Vorhersage-vs.-Wahrheit-Diagramm, allerdings mit deutlich größeren Konfidenzintervallen. Obwohl die Punkte auf den ersten Blick ebenfalls nahe an der Diagonalen liegen und damit eine gute Übereinstimmung mit den tatsächlichen Werten suggerieren, verdeutlicht die Breite der Unsicherheitsbereiche eine geringere Zuverlässigkeit der Modellvorhersagen. Dies macht deutlich, dass die alleinige Betrachtung der Punktvorhersagen nicht ausreicht, sondern auch die Konfidenzintervalle als Maß für die Modellgüte berücksichtigt werden müssen. Sie sind ein entscheidender Indikator, um Modelle mit scheinbar ähnlicher Vorhersagequalität differenziert zu bewerten.

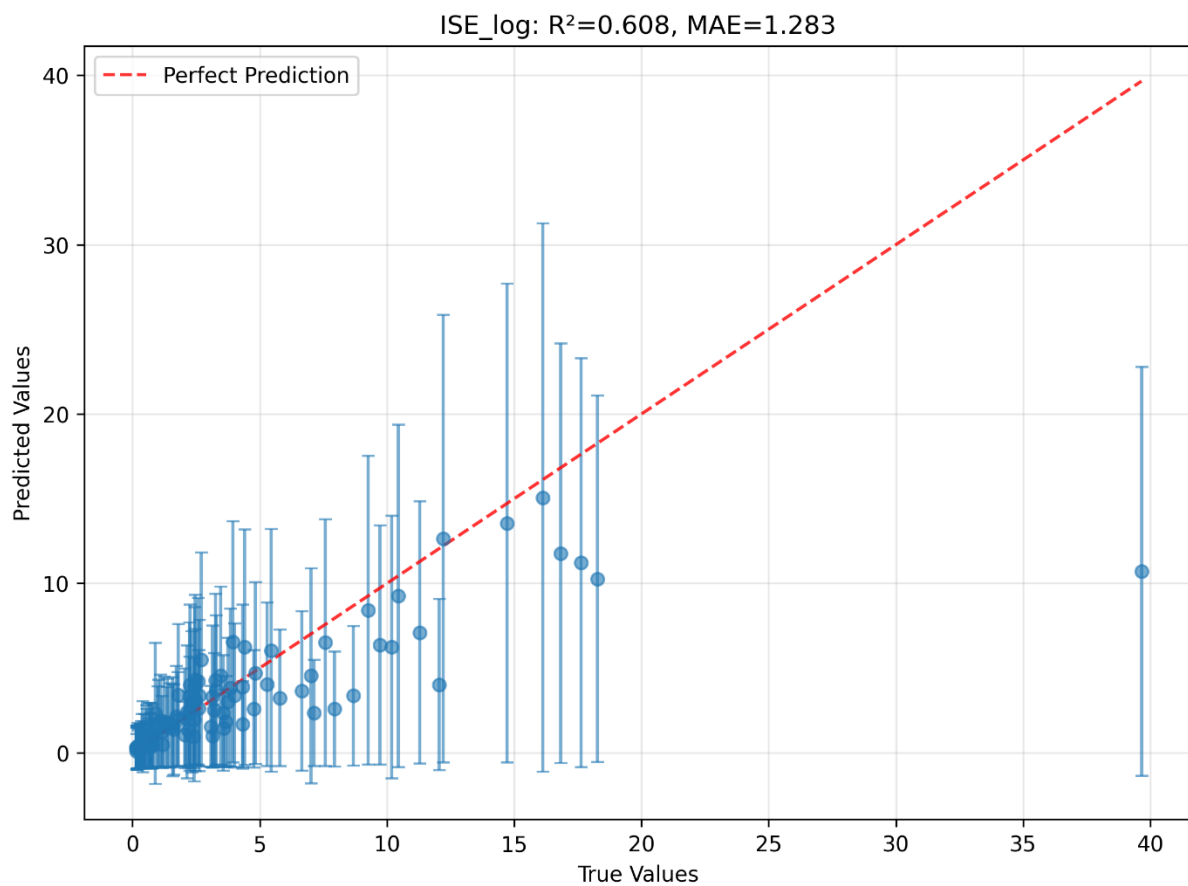


Abbildung 5: Vorhersage vs Wahrheit mit Konfidenzintervall große Streuung (Eigendarstellung)

Abbildung 6: Vorhersage vs Wahrheit nach Regelparameter zeigt Vorhersage-vs.-Wahrheit-Diagramme für die drei Reglerparameter K_p , K_i und K_d . Jeder Plot stellt die Modellvorhersagen für den jeweiligen Parameter den tatsächlichen Referenzwerten gegenüber. Während die Schätzungen für manche Parameter (z. B. K_p und K_d) eine hohe Genauigkeit mit geringer Abweichung von der Diagonalen aufweisen, können sich bei anderen Parametern (z. B. K_i) größere Streuungen und damit geringere Vorhersagegüten zeigen. Die Darstellung verdeutlicht, dass sich die Modellgenauigkeit innerhalb der einzelnen Reglerkomponenten unterscheiden kann und eine getrennte Analyse notwendig ist.

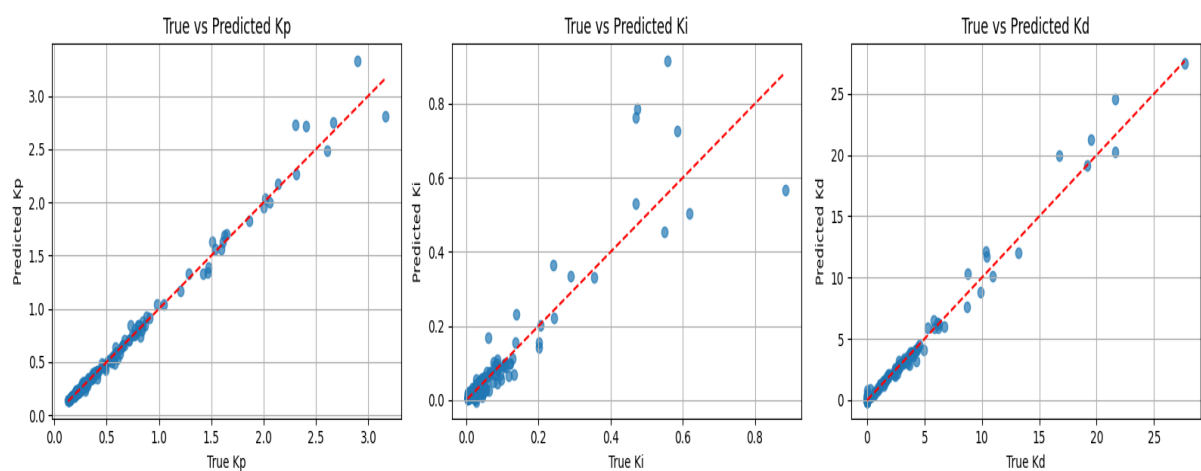


Abbildung 6: Vorhersage vs Wahrheit nach Regelparameter (Eigendarstellung)

Abbildung 7: Confusion Matrix zeigt eine Matrix, die die Klassifikation von Closed-Loop-Regelstrecken in vier Qualitätskategorien darstellt: good, hoher Overshoot, langsame Regelung und instabil. Jede Zelle der Matrix gibt die Anzahl der Instanzen an, die vom Klassifikationsmodell einer bestimmten Klasse zugeordnet wurden (Vorhersage, Spalten) im Vergleich zu ihrer tatsächlichen Zugehörigkeit (Wahrheit, Zeilen). Eine starke Besetzung der Diagonale weist auf eine hohe Klassifikationsgenauigkeit hin, während Einträge außerhalb der Diagonale Fehleinschätzungen des Modells darstellen.

Diese Darstellung erlaubt eine differenzierte Analyse der Modellleistung: So wird beispielsweise sichtbar, ob das Modell instabile Strecken zuverlässig erkennt oder ob langsames Regelverhalten fälschlicherweise als good klassifiziert wird und unpassende Regler aussortiert.

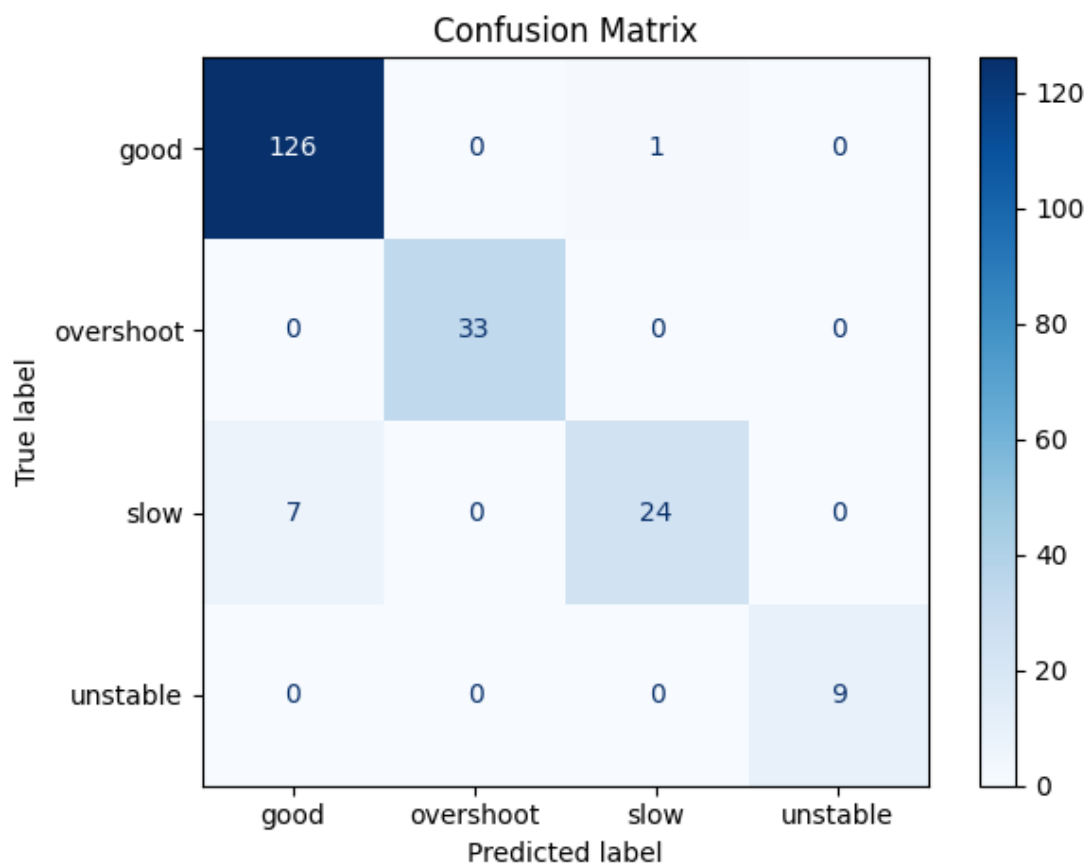


Abbildung 7: Confusion Matrix (Eigendarstellung)

4 Methoden und Implementierung

Im Kapitel 4 „Methoden und Implementierung“ wird die konkrete Vorgehensweise zur Optimierung von PID-Reglern mittels Machine Learning beschrieben. Dazu gehören die Gesamtarchitektur der ML-Pipeline, die Generierung und Aufbereitung synthetischer Datensätze, der Lernprozess sowie die Entwicklung von Surrogat- und Klassifikationsmodellen. Außerdem wird die Umsetzung eines Optimierungsmodells erläutert, das es ermöglicht, Reglerparameter unter Berücksichtigung nutzerdefinierter Präferenzen systematisch zu bestimmen.

4.1 Gesamtarchitektur der Machine Learning pipeline

Die entwickelte Optimierungspipeline folgt einem modularen Aufbau, der eine systematische Abbildung von Prozessstrecken auf geeignete Reglerparameter ermöglicht. Da in industriellen Anwendungen typischerweise keine ausreichend großen, frei verfügbaren Datensätze zur Verfügung stehen, wurde im Rahmen dieser Arbeit zunächst auf die Generierung synthetischer Daten zurückgegriffen. Hierzu wurden mit MATLAB und der integrierten pidtune-Funktion automatisiert verschiedene Prozessstrecken parametrisiert (z. B. durch Variation von Zeitkonstanten, Verstärkungen und Totzeiten) und die jeweils zugehörigen PID-Koeffizienten ermittelt [\[4\]](#). Das Ergebnis sind strukturierte Datensätze im CSV-Format, die als Grundlage für das anschließende Modelltraining dienen.

Im zweiten Schritt erfolgt das Training von Surrogatmodellen mit unterschiedlichen Verfahren (Symbolic Regression, Gradient-Boosting-Modelle wie XGBoost, sowie Gaussian und Deep Gaussian Processes). Ziel ist es, die Abbildung zwischen Streckenparametern und Reglerparametern beziehungsweise Leistungsmetriken wie Überschwingen oder ISE zu erlernen.

Anschließend wird die Modellgüte systematisch bewertet. Dazu werden die trainierten Modelle mit statistischen Kennzahlen (R^2 , MAE, RMSE) sowie Vorhersage-vs.-Wahrheit-Analysen überprüft. Zeigt sich, dass ein Modell unzureichend generalisiert oder deutliche Vorhersagefehler auftreten, erfolgt ein Retraining. Dies kann beispielsweise durch die Erweiterung der Datenbasis, die Transformation der Eingangsgrößen (z. B. logarithmische Skalierung) oder die Anpassung von Modellhyperparametern geschehen.

Sobald ein Modell hinreichend gute Ergebnisse liefert, wird es in der Produktionspipeline eingesetzt. Dort dient es als Vorhersageinstrument, um für neue Streckenbedingungen effizient geeignete Reglerparameter zu identifizieren. Der iterative Aufbau ermöglicht es, den gesamten Prozess von der Datengenerierung bis zur produktiven Nutzung kontinuierlich zu verbessern.

Abbildung 8: Pipeline visualisiert die Gesamtarchitektur der entwickelten Machine-Learning-Pipeline. Der Prozess startet mit der Generierung synthetischer Datensätze, da reale Prozessdaten oft nicht in ausreichender Menge zur Verfügung stehen. Diese Daten durchlaufen eine mehrstufige Datenvorbereitung, die Bereinigung, Feature Engineering und Skalierung beinhaltet.

Anschließend wird ein einfaches Basismodell erstellt und evaluiert. Erweist sich dieses als vielversprechend ("ausbaufähig"), folgt das Training der finalen, komplexeren Algorithmen (Symbolic Regression, XGBoost, Deep Gaussian Process) inklusive Hyperparameter-Tuning. Das wesentliche Merkmal der Pipeline ist ihr iterativer Charakter: Eine Rückkopplungsschleife ermöglicht die ständige Verfeinerung des gesamten Prozesses, bis eine zufriedenstellende Modellperformance für die PID-Optimierung erreicht ist.

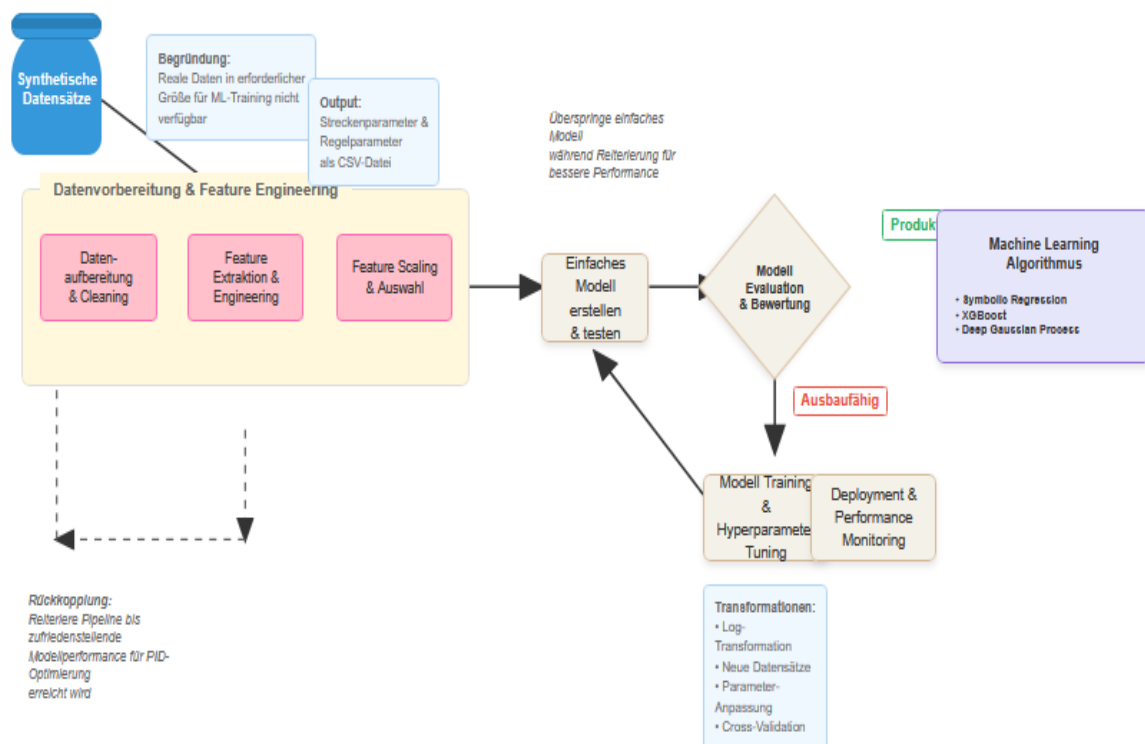


Abbildung 8: Pipeline (Eigendarstellung)

4.2 Generierung synthetischer Datensätze

4.2.1 Modellierung von Regelstrecken in MATLAB/Simulink

Die Generierung synthetischer Datensätze setzt eine geeignete Modellierung von Regelstrecken voraus, die eine hinreichend realistische Abbildung industrieller Prozesse ermöglicht. Für die vorliegende Arbeit wurde MATLAB/Simulink gewählt, da es sich um ein in der Regelungstechnik etabliertes Werkzeug handelt, das speziell für die Simulation dynamischer Systeme und die Analyse von Reglerentwürfen entwickelt wurde [\[4\]](#). Durch die enge Integration von Simulationsumgebung und Control System Toolbox lassen sich lineare zeitinvariante (LTI) Systeme effizient abbilden und mit standardisierten Methoden wie der automatisierten PID-Synthese über die Funktion `piddtune` koppeln.

Die Regelstrecken wurden in Form von Übertragungsfunktionen modelliert, die verschiedene Charakteristika industrieller Prozesse repräsentieren. Hierbei kamen insbesondere Systeme erster und zweiter Ordnung (PT1- und PT2-Glieder) sowie erweiterte Modelle mit zusätzlicher Totzeit zum Einsatz. Der Schwerpunkt der nachfolgenden Evaluation liegt auf der Analyse von PT2-Strecken, da diese eine verbreitete und relevante Klasse dynamischer Systeme in der Praxis darstellen und im Labor der Hochschule verfügbar sind. Diese Strukturen bilden eine große Bandbreite praxisrelevanter Prozesse ab, wie sie etwa in chemischen Reaktoren, thermischen Anlagen oder hydraulischen Systemen auftreten. Durch die modulare Struktur in Simulink können darüber hinaus komplexere Strecken zusammengesetzt werden, beispielsweise durch die Kaskadierung mehrerer Verzögerungsglieder oder die Kombination mit reinen Integratoren [\[1\]\[2\]](#).

$$(9): G_{IT1}(s) = \frac{K}{s(Ts + 1)}$$

$$(10): G_P(s) = Kp$$

$$(11): G_{PT1}(s) = \frac{K}{Ts + 1}$$

$$(12): G_{PT2}(s) = \frac{K}{(T_1s + 1)(T_2s + 1)}$$

Die Parametervariationen, wurden automatisiert durch MATLAB-Skripte realisiert. Auf diese Weise konnten zahlreiche Streckenkonfigurationen systematisch generiert werden, die gemeinsam eine breite Abdeckung des relevanten Parameterraums sicherstellen. Die jeweiligen Strecken wurden anschließend mithilfe der pidtune-Funktion eingestellt, wodurch selbstständig zugehörige Reglerparameter bestimmt werden konnten. Die automatisierte Erzeugung Abbildung 9: Simulink Modell Synthetische Generierung stellt dabei sicher, dass die Datengrundlage sowohl konsistent als auch reproduzierbar ist, was eine wesentliche Voraussetzung für die nachfolgende Anwendung maschineller Lernverfahren darstellt [4].

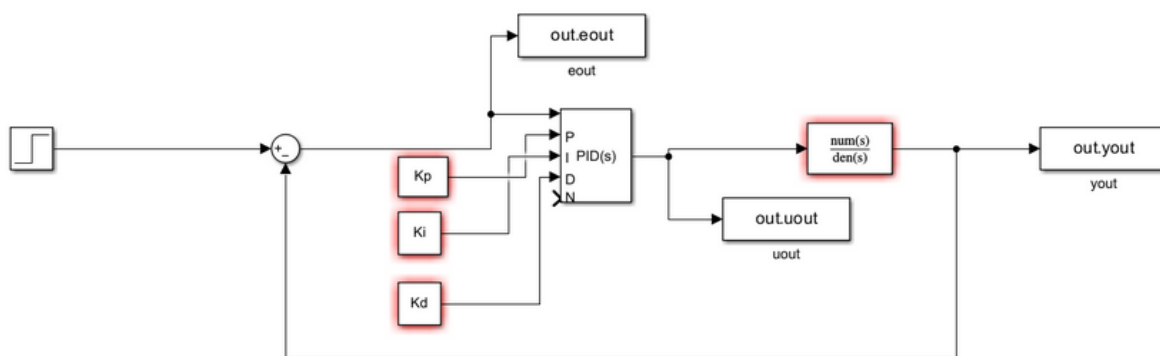


Abbildung 9: Simulink Modell Synthetische Generierung (Eigendarstellung)

4.2.2 Definition der Parameterbereiche

Zur Generierung der synthetischen Datensätze war es notwendig, geeignete Parameterbereiche für die Modellierung der Regelstrecken festzulegen. Zu Beginn wurde mit relativ engen Grenzen gearbeitet, um eine stabile und kontrollierte Datengrundlage zu schaffen. So wurde die Verstärkung K zunächst im Bereich von 0,5 bis 5 variiert, während die Zeitkonstanten T_1 und T_2 Werte zwischen 0 und 10 annahmen.

Im Verlauf der Arbeit wurden die Parameterbereiche schrittweise erweitert, um auch komplexere und realitätsnähere Dynamiken abzubilden. Die Verstärkung K wurde dabei auf einen Bereich von 0,1 bis 100 ausgeweitet, während die Zeitkonstanten T_1 und T_2 Werte zwischen 0,01 und 150 umfassen konnten. Zusätzlich wurde eine mögliche Totzeit T_d berücksichtigt, deren Variation zwischen 0 und 5 erfolgte.

4.2.3 Automatisierte Berechnung von Leistungsmetriken

Zur Bewertung der erzeugten Reglerparameter wurden die geschlossenen Regelkreise nach der Simulation in MATLAB/Simulink automatisch analysiert. Die Simulationsergebnisse wurden direkt aus dem Simulink-Modell in den MATLAB-Workspace exportiert [\[4\]](#).

Im Zentrum der Analyse standen sowohl klassische zeitdiskrete Metriken als auch integrale Gütemaße. Mit der Funktion `stepinfo` wurden zentrale Kenngrößen wie Anstiegszeit, Einschwingzeit und Überspringen berechnet. Zusätzlich wurden das integrierte quadratische Fehlermaß (ISE) sowie der stationäre Fehler (SSE) aus dem simulierten Sprungantwortsignal abgeleitet. Dabei wurde der Fehler $e(t)$ als Differenz zwischen Soll- und Ist-Ausgang definiert, wobei im Fall eines Einheitssprungs $e(t) = 1 - y(t)$ gilt. Das ISE wurde über die Zeitachse mithilfe einer numerischen Integration approximiert, während der SSE direkt aus dem Endwert des Fehlers bestimmt wurde [\[4\]](#).

Alle berechneten Größen, darunter die Streckenparameter (K , T_1 , T_2 , T_d), die Reglerparameter (K_p , K_i , K_d , N) sowie die Leistungsmetriken (ISE, SSE, Anstiegszeit, Einschwingzeit und Überspringen), wurden automatisiert in einer CSV Datei gespeichert.

4.3 Datenaufbereitung und Lernprozess

Die im Simulationsprozess generierten Rohdaten mussten vor dem eigentlichen Training des Machine Learning Verfahren sorgfältig aufbereitet werden, um eine robuste und konsistente Modellbildung zu gewährleisten. Zunächst wurden ausschließlich stabile Regelkreise in den Datensatz aufgenommen. Dies erfolgte durch die Anwendung der `isstable`-Funktion in MATLAB, die instabile Strecken und Reglerkombinationen zuverlässig identifiziert und ausschließt [\[4\]](#). Darüber hinaus wurden Simulationsergebnisse mit stark unzureichender Regelgüte beispielsweise sehr hohen Werten für das Integral of Squared Error (ISE) oder extrem langen Einschwingzeiten als „schlecht“ klassifiziert und aus den Trainingsdaten entfernt.

Zur weiteren Verbesserung der Modellierbarkeit wurden ausgewählte Eingangsgrößen einer Transformation unterzogen. Insbesondere logarithmische Transformationen kamen zum

Einsatz, um stark streuende oder nichtlinear verteilte Größen zu glätten und deren Einfluss auf den Trainingsprozess systematisch zu untersuchen.

Nach dieser Bereinigung und Transformation erfolgte die Aufteilung der Daten in Trainings-, Validierungs- und Testmengen. Dadurch wurde sichergestellt, dass die Modelle nicht nur auf die vorhandenen Daten angepasst sind, sondern auch eine gute Generalisierungsfähigkeit gegenüber unbekannten Prozessstrecken aufweisen. Insgesamt bildete diese Phase der Datenaufbereitung die Grundlage für einen stabilen und reproduzierbaren Lernprozess, bei dem die gewählten Algorithmen (Symbolic Regression, XGBoost, Deep Gaussian Processes) effizient trainiert und evaluiert werden konnten.

4.4 Surrogat- und Klassifikationsmodelle

4.4.1 Vorhersage von Reglerparametern aus Streckenparametern

Im ersten Ansatz erfolgt die Modellierung ausschließlich auf Basis der Streckenparameter wie Prozessverstärkung K , Zeitkonstanten T_1 und T_2 sowie Totzeit T_d . Ziel ist es, die daraus resultierenden optimalen Reglerkoeffizienten K_p , K_i und K_d vorherzusagen. Dieses Verfahren entspricht einer datengetriebenen Alternative zu klassischen Tuningmethoden wie den Verfahren nach Ziegler–Nichols oder Chien–Hrones–Reswick [\[1\]\[2\]](#).

Der wesentliche Vorteil liegt darin, dass das Modell für neue, bislang unbekannte Prozesse unmittelbar Vorschläge für geeignete Reglerparameter liefern kann, ohne dass eine erneute Simulation oder ein analytisches Tuning erforderlich ist [\[13\]](#). Für diese Aufgabe eignen sich insbesondere Verfahren des Supervised Learning wie Entscheidungsbaum-basierte Methoden (Random Forest, XGBoost), Symbolic Regression, die zusätzlich interpretierbare Gleichungen liefert, sowie Gaussian Processes, die neben der Vorhersage auch eine Quantifizierung der Unsicherheit ermöglichen. Die Hauptherausforderung liegt in der Generalisierungsfähigkeit: Nur wenn der Trainingsdatensatz eine ausreichend große Vielfalt an Streckenparametern abdeckt, lassen sich robuste Vorhersagen auch für komplexe oder seltene Prozessdynamiken erzielen [\[14\]\[15\]\[16\]\[17\]\[18\]](#).

Basierend auf den Parametern einer vom Nutzer definierten Übertragungsfunktion ermittelt das ML-Modell die optimalen Reglerkoeffizienten und stellt die resultierende Sprungantwort in einer vergleichenden Simulation den klassischen Verfahren nach Ziegler-Nichols und CHR gegenüber (Abbildung 10: Vorhersage von Reglerparametern aus Streckenparametern).

Key Performance Metrics (All Controllers)

	Controller	Rise Time [s]	Settling Time [s]	Overshoot [%]	ISE
0	ML	0.88	5.19	21.94	0.600
1	ZN	0.40	17.56	77.53	1.185
2	CHR 0%	0.72	7.43	26.26	0.535
3	CHR 20%	0.56	8.09	34.39	0.464

Closed-Loop Step Response

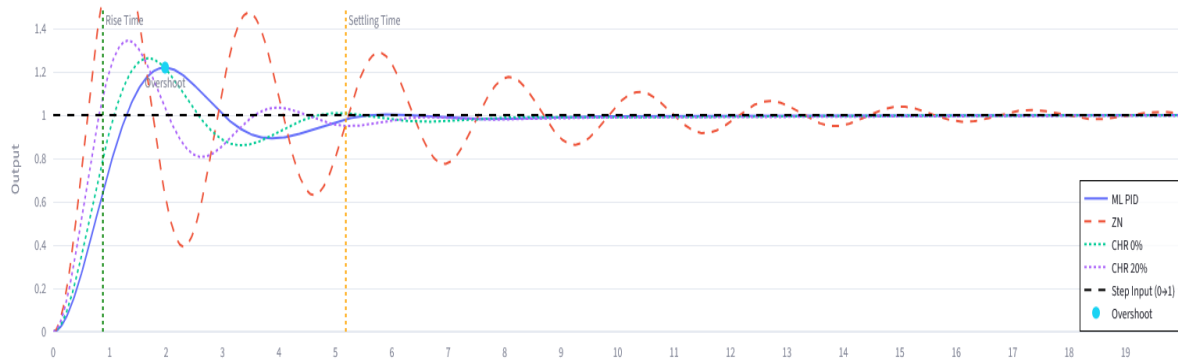


Abbildung 10: Vorhersage von Reglerparametern aus Streckenparametern (Eigendarstellung)

4.4.2 Vorhersage von Leistungskenngrößen aus Strecken- und Reglerparametern

Der zweite Ansatz erweitert den Eingaberaum um die Reglerparameter K_p , K_i und K_d , sodass das Modell neben den Streckenparametern auch die aktuelle Reglerauslegung berücksichtigt. Die Ausgaben sind hier nicht die Reglerparameter selbst, sondern relevante Leistungsmetriken wie Integral of Squared Error (ISE), Überschwingen, Einschwingzeit, Ausregelzeit oder stationärer Fehler (SSE) [7]. Auf diese Weise wird ein Surrogatmodell aufgebaut, das das dynamische Verhalten der geschlossenen Regelstrecke approximiert. Der Vorteil dieses Ansatzes besteht darin, dass er schnelle Simulationen ermöglicht: Anstatt jedes Regler-Strecken-Paar vollständig in MATLAB/Simulink zu berechnen, können Vorhersagen direkt über das trainierte Modell erfolgen. Dies reduziert den Rechenaufwand erheblich und erlaubt die Einbindung von Optimierungsverfahren, beispielsweise genetischen Algorithmen, zur

gezielten Suche nach optimalen Reglerparametern. Darüber hinaus ermöglichen Surrogatmodelle Unsicherheitsanalysen, etwa durch die Breite von Konfidenzintervallen bei Gaussian Processes oder Deep Gaussian Processes, was für die Bewertung der Verlässlichkeit der Ergebnisse entscheidend ist. Die größte Herausforderung in diesem Ansatz liegt in der hohen Dimensionalität der Eingabeparameter und der Komplexität der zugrunde liegenden nichtlinearen Zusammenhänge, die besonders leistungsfähige Regressionsmethoden erfordern [16][18].

Zusammenfassend lässt sich festhalten, dass beide Ansätze komplementär eingesetzt werden können: Ein Modell zur direkten Vorhersage von Reglerparametern liefert eine schnelle Initialisierung für neue Prozesse, während Surrogatmodelle eine detaillierte Bewertung und Optimierung der Reglerleistung erlauben.

Das in Abbildung 11: Vorhersage von Leistungskenngrößen dargestellte Modell dient als schneller Simulator (Surrogat), indem es die vollständige Eingabe von Strecken- und Reglerparametern nutzt, um die dynamische Systemantwort in Form von Leistungskenngrößen unmittelbar vorherzusagen.

Performance: Surrogate vs Simulation

	Metric	Predicted	Simulated
0	ISE	-0,1591	0,5102
1	SSE	0,01886	0,00000
2	Overshoot [%]	4,95	9,07
3	Settling Time [s]	0,22	5,80
4	Rise Time [s]	-1,10	1,15

Closed-Loop Step Response

Step Response of $G(s)$ + PID

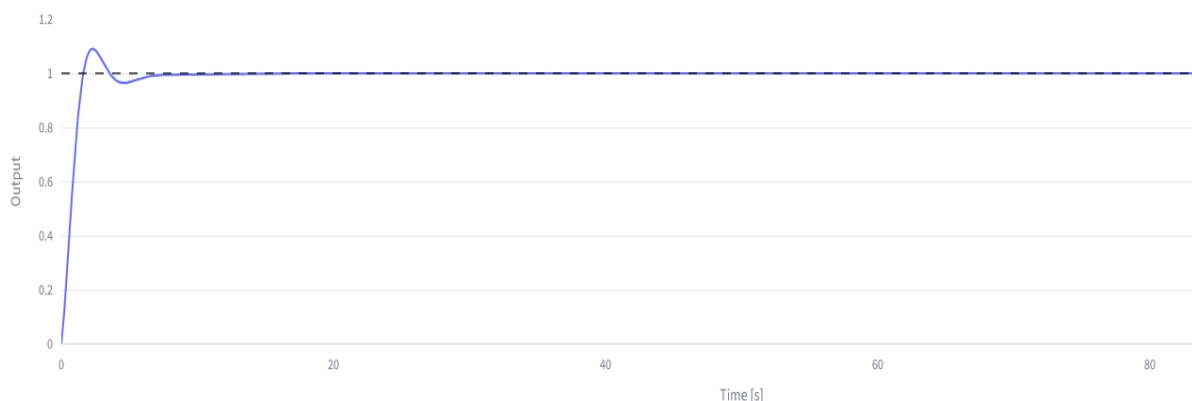


Abbildung 11: Vorhersage von Leistungskenngrößen (Eigendarstellung)

4.5 Optimierungsmodell mit nutzerdefinierten Präferenzen

Klassische Methoden der PID-Parametrierung, wie Ziegler-Nichols oder Chien-Hrones-Reswick, liefern standardisierte Einstellungen, die auf allgemeine Gütekriterien ausgerichtet sind, berücksichtigen jedoch nicht individuelle Zielsetzungen. Das vorgestellte Optimierungsmodell ermöglicht daher eine flexible Anpassung der PID-Parameter an die Präferenzen des Nutzers [\[1\]\[2\]](#).

Über interaktive Schieberegler kann der Anwender die Gewichtung einzelner Leistungsmetriken etwa ISE, Überschwingen, Einschwingzeit oder Sprungantwortgeschwindigkeit direkt auf die Kostenfunktion einwirken lassen. Zusätzlich lassen sich Grenzwerte für einzelne Kennzahlen festlegen, wodurch PID-Regler, die diese überschreiten, automatisch aussortiert werden. So werden nicht nur optimale Parameter vorgeschlagen, sondern auch konkrete Mindestanforderungen an die Regelgüte eingehalten.

Die Umsetzung erfolgt über eine interaktive Weboberfläche (vgl. Abbildung 12: Optimierungsmodell Nutzereingabe, Abbildung 13: Optimierungsmodell Output). Diese zeigt im Optimierungs-Tab die verfügbaren Slider zur Gewichtung der Metriken, Eingabefelder für Grenzwerte, die Performance-Kennzahlen in tabellarischer Form sowie die Schrittantworten der fünf besten Controller. Dadurch erhält der Nutzer sofortige Rückmeldung über die Auswirkungen seiner Einstellungen. Schrittantwortplots, Fehlerverläufe und Performance-Tabelle visualisieren die Ergebnisse anschaulich und machen die Optimierung transparent.

Define Optimization Weights



Define Performance Constraints

Define Performance Constraints:

- Max ISE: 25.00
- Max Overshoot (%): 50.00
- Max Settling Time: 100.00
- Max Rise Time: 50.00
- Max SSE: 0.50

Abbildung 12: Optimierungsmodell Nutzereingabe (Eigendarstellung)

	Kp	Ki	Kd	ISE_sim	Overshoot_sim	SettlingTime_sim	RiseTime_sim	SSE_sim	Cost_true
0	2.0357	0.199	5.2862	1.036	1.635	2.903	1.502	0	1.576
1	1.8897	0.1585	4.7383	1.096	0.588	3.704	1.802	0	1.586
2	2.6248	0.2131	6.0532	0.963	5.312	3.804	1.101	0	2.415
3	3.1472	0.2194	6.3748	0.948	11.04	4.204	0.901	0	3.613
4	1.9452	0.1744	3.779	1.215	5.528	9.409	1.902	0	3.785

Step Responses of Top 5 Controllers

Step Response of Top 5 Controllers

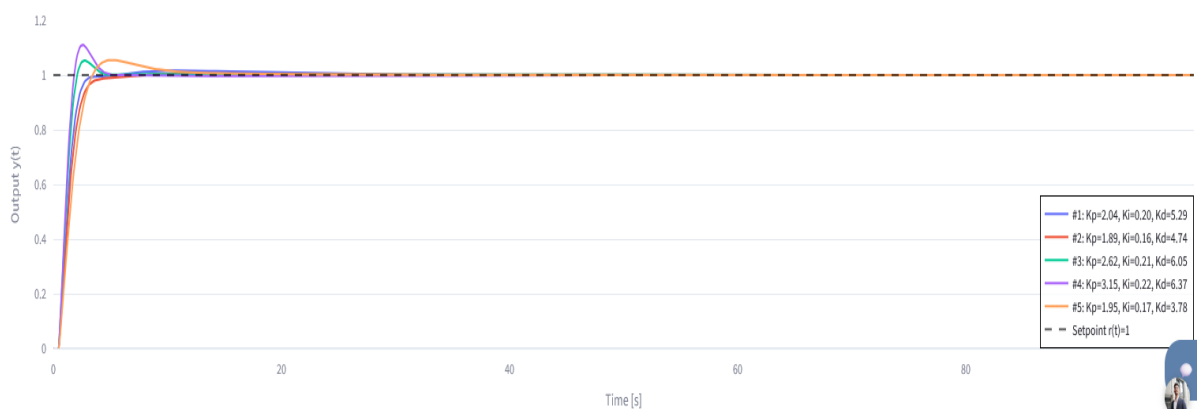


Abbildung 13: Optimierungsmodell Output (Eigendarstellung)

Das Optimierungsmodell kombiniert maschinelles Lernen mit heuristischer Optimierung, um PID-Reglerparameter automatisiert und effizient zu bestimmen. Ausgangspunkt ist die Eingabe der Streckenparameter durch den Nutzer, wie Verstärkung, Zeitkonstanten oder Totzeit, die die zu regelnde Übertragungsfunktion definieren, sowie Gewichtungen (weights w) der Leistungsmerkmale und Obergrenzen (Constraints C). Anschließend wird ein zuvor trainiertes Modell geladen, in diesem Fall Deep Gaussian Process (DGP). Dieses Modell dient als Surrogatmodell, also eine vereinfachte, lernbasierte Näherung der realen Strecke, die es ermöglicht, die erwarteten Leistungskennwerte eines Reglers schnell vorherzusagen, ohne eine vollständige Simulation durchzuführen.

Zur Optimierung der PID-Parameter wird ein differenzielles Evolutionsverfahren eingesetzt. Dieses Verfahren startet mit einer Ursprungspopulation zufällig gewählter Reglerkonfigurationen innerhalb festgelegter Grenzen. In iterativen Schritten werden neue Kandidaten durch Mutation und Rekombination der existierenden Population erzeugt. Jeder neue Kandidat wird anhand einer Kostenfunktion (Cost) bewertet, die die einzelnen Gütekriterien der Regelung wie Anstiegszeit, Überschwingen, Einschwingzeit und Integral des quadratischen Regelabweichungsfehlers gewichtet und eventuelle Nebenbedingungen wie maximale Überschreitungen oder Obergrenzen für die Einschwingzeit berücksichtigt. Die besten Lösungen jeder Generation werden in die nächste Generation übernommen, wodurch die Population schrittweise auf optimale Parameterkonfigurationen hin konvergiert.

Die Kostenfunktion ist in der Gleichung (13) dargestellt; ein möglichst niedriger Wert entspricht hierbei der optimalen Regelgüte. Über interaktive Schieberegler kann der Anwender die Gewichtung der einzelnen Leistungskriterien innerhalb der Kostenfunktion anpassen und somit individuelle Prioritäten setzen. Die in Gleichung (14) exemplarisch dargestellten oberen Grenzen, hier am Beispiel des ISE, dienen der Filterung von Reglerparametern: Kandidaten, deren simulierte Kennwerte die definierten Grenzwerte C überschreiten, werden aus dem Suchraum ausgeschlossen, wodurch nur zulässige Controller in die weitere Evaluierung aufgenommen werden.

$$(13) \text{ Cost} = w_{\text{ISE}} \cdot \text{ISE} + w_{\text{OS}} \cdot \text{OS} + w_{\text{ST}} \cdot \text{ST} + w_{\text{RT}} \cdot \text{RT}$$

$$(14) \text{ ISE}_{\text{SIM}} \leq \text{ISE}_C$$

Nach Abschluss der Optimierung werden die Top-25-Regler auf Basis ihrer prognostizierten Leistung ausgewählt. Um die Vorhersagegenauigkeit des Surrogatmodells zu validieren, werden diese Kandidaten anschließend in einer realen oder simulierten Umgebung getestet, um die tatsächlichen Leistungskennwerte zu bestimmen. Nur die Top-5-Regler, die zusätzlich alle vom Nutzer vorgegebenen Grenzen erfüllen, werden schließlich bereitgestellt. Für diese Regler werden die Schrittantworten visualisiert, sodass der Nutzer die Dynamik und Qualität der Regelung direkt beurteilen kann.

Damit kombiniert das Modell automatisierte, multikriterielle Optimierung mit einer intuitiven, nutzerzentrierten Steuerung und erhöht sowohl Flexibilität als auch Praxisrelevanz. Idealerweise würde eine perfekte Regelung alle Leistungsmetriken wie ISE, Anstiegszeit, Einschwingzeit oder Überschwingen auf null reduzieren. In der Realität existiert eine solche „perfekte“ Regelung jedoch praktisch nicht, da stets Kompromisse zwischen den verschiedenen Kennwerten bestehen. Durch die interaktiven Slider kann der Nutzer nun selbst die Gewichtung der Metriken anpassen und sein individuelles Optimum definieren, wodurch unterschiedliche Trade-offs schnell und anschaulich bewertet werden können.

5 Ergebnisse

Im Kapitel 5 „Ergebnisse“ werden die Resultate der PID-Optimierung mittels Machine Learning präsentiert und analysiert. Es erfolgt zunächst ein Vergleich verschiedener ML-Modelle hinsichtlich ihrer Vorhersagegüte, gefolgt von einem Vergleich mit klassischen Tuning-Methoden. Die Evaluierung erfolgt sowohl anhand synthetischer Regelstrecken in Simulink als auch durch Validierung an realen Regelstrecken, um die praktische Anwendbarkeit der Ansätze zu prüfen.

5.1 Vergleich der Machine Learning Modelle

Die Beurteilung der Modellgüte erfolgte über die drei zentralen Kennzahlen Bestimmtheitsmaß (R^2), mittlerer absoluter Fehler (MAE) sowie Root Mean Squared Error (RMSE). Diese Kombination erlaubt sowohl eine Einschätzung der Erklärungsleistung der Modelle als auch eine differenzierte Betrachtung der mittleren und stärker gewichteten Abweichungen [\[6\]](#)[\[7\]](#)[\[8\]](#).

Im direkten Vergleich konnte das XGBoost-Modell die besten Ergebnisse erzielen. Mit hohen R^2 -Werten nahe 0,98 und gleichzeitig sehr niedrigen MAE- und RMSE-Werten zeigte es eine robuste Anpassungsfähigkeit an unterschiedliche Streckentypen und eine konsistente Generalisierung auch auf Testdaten. Dieses Ergebnis war erwartbar, da XGBoost für tabellarische Daten mit gemischten Skalen optimiert ist und in der Lage ist, nichtlineare Zusammenhänge sowie Interaktionen zwischen den Eingangsparametern (z. B. Systemverstärkung, Zeitkonstanten, Totzeiten) effizient abzubilden.

Das Deep Gaussian Process (DGP)-Modell folgte in der Gesamtbewertung auf Platz zwei. Auch wenn die Genauigkeit etwas geringer ausfiel, konnte das Modell zusätzlich Konfidenzintervalle für seine Vorhersagen bereitstellen. Diese Eigenschaft ist insbesondere für die Regelungstechnik von großem Vorteil, da sich damit nicht nur Punktvorhersagen, sondern auch Unsicherheiten in den Parametern quantifizieren lassen. In praktischen Anwendungen erlaubt dies eine Einschätzung, ob eine vorgeschlagene Parametrierung mit hoher Wahrscheinlichkeit stabil ist oder ob zusätzliche Optimierungsschritte notwendig erscheinen.

Das symbolische Regressionsmodell blieb dagegen in allen Kennzahlen deutlich hinter den anderen Verfahren zurück. Mit spürbar niedrigeren R^2 -Werten sowie höheren MAE- und RMSE-Werten zeigte sich, dass die methodische Beschränkung auf vergleichsweise einfache funktionale Abbildungen die komplexen Zusammenhänge in der PID-Parametrierung nicht ausreichend erfassen konnte. Hinzu kam, dass dieses Modell in einzelnen Szenarien sogar unplausible oder instabile Reglerparameter (z. B. negative Verstärkungswerte) vorgeschlagen hat.

Besonders auffällig war, dass XGBoost und DGP über die betrachteten Streckentypen (P, PT1, PT2 und IT1) hinweg eine konsistente Leistungsfähigkeit zeigten, während die symbolische Regression stark schwankte und in einzelnen Fällen vollständig versagte. Diese Resultate legen nahe, dass sich insbesondere XGBoost aufgrund seiner hohen Genauigkeit und DGP aufgrund seiner Unsicherheitsbewertung für eine robuste Surrogatmodellierung der PID-Parametrierung eignen, während die symbolische Regression in diesem Kontext nur eingeschränkt nutzbar ist.

Die entsprechenden Ergebnisse sind in Tabelle 2: Vergleich der Modellgüte (R^2 , MAE, RMSE) von XGBoost, DGP und Symbolic Regression über verschiedene Streckentypen zusammengefasst. Dabei zeigte sich, dass die Modellgüte deutlich vom zugrunde liegenden Streckentyp abhängt. Über alle Verfahren hinweg lieferten die PT2-Strecken die besten Vorhersagewerte, während IT1-Strecken die schwächste Performance aufwiesen.

Tabelle 2: Vergleich der Modellgüte (R^2 , MAE, RMSE) von XGBoost, DGP und Symbolic Regression über verschiedene Streckentypen

Modell	PT1 (K_p , K_i , K_d)	PT2 (K_p , K_i , K_d)	IT1 (K_p , K_i , K_d)
XGBoost	$R^2 > 0.92$, MAE < 0.02, RMSE < 0.05	$R^2 \approx 0.98$, MAE < 0.05, RMSE < 0.1	$R^2 \approx 0.77$, MAE < 0.06, RMSE < 0.12
Deep Gaussian Process	$R^2 \approx 0.87$, MAE $\approx$ 0.04, RMSE $\approx$ 0.1	$R^2 \approx 0.92$, MAE $\approx$ 0.06, RMSE $\approx$ 0.15	$R^2 \approx 0.68$, MAE $\approx$ 0.07, RMSE $\approx$ 0.2
Symbolic Regression	$R^2 \approx 0.85$, MAE $\approx$ 0.15, RMSE $\approx$ 0.25	$R^2 \approx 0.80$, MAE $\approx$ 0.18, RMSE $\approx$ 0.3	$R^2 \approx 0.66$, MAE $\approx$ 0.2, RMSE $\approx$ 0.35

Um die Generalisierungsfähigkeit des Deep Gaussian Process (DGP)-Modells über unterschiedliche Streckenparameter hinweg zu evaluieren, wurde die Modellgüte systematisch in verschiedenen Parameterregionen untersucht. Dabei wurden Strecken mit unterschiedlichen Verstärkungen K sowie Zeitkonstanten T_1 , T_2 betrachtet, um sowohl typische als auch extremere Betriebsbedingungen abzudecken. Die Vorhersagegüte für die drei Regleranteile K_p , K_i und K_d wurde anhand von R^2 und MAE beurteilt [6][7][8].

Die Ergebnisse, zusammengefasst in Tabelle 3: DGP-Performance in unterschiedlichen Parameterregionen, zeigen, dass das DGP-Modell insbesondere in Regionen mit mittleren bis hohen Verstärkungen und moderaten Zeitkonstanten die besten Vorhersagewerte liefert.

Tabelle 3: DGP-Performance in unterschiedlichen Parameterregionen

Region	Parameter	Regleranteil	R^2	MAE	Samples
K 1–5	T1 10–150	K_d	0.991	0.172	680
K 5–10	T2 1–10	K_d	0.933	0.125	279
K 5–10	$T_1 < 1$	K_d	0.917	0.142	65
K 1–5	T1 1–10	K_d	0.968	0.190	484
K 1–5	$T_2 < 1$	K_d	0.930	0.211	77
K 1–5	T1 10–30	K_i	0.884	0.083	68
K 5–10	$T_1 < 1$	K_i	0.857	0.075	65
K 5–10	T1 10–150	K_i	0.773	0.078	279
K 1–5	T1 1–10	K_i	0.734	0.074	148
K 1–5	$T_1 < 1$	K_i	0.638	0.136	37
K 1–5	T1 10–100	K_p	0.961	0.223	360
K 5–10	T1 0.1–10	K_p	0.951	0.191	279
K 5–10	$T_1 < 1$	K_p	0.945	0.201	650
K 1–5	T1 1–150	K_p	0.836	0.218	484

Für den Differentialanteil (K_d) werden R^2 -Werte nahe 0,98 erreicht, während der Integralanteil (K_i) in Regionen mit sehr kleinen Zeitkonstanten deutlich höhere Fehler aufweist. Regionen, in denen viele Trainingsdaten sowohl innerhalb des Bereichs selbst als auch in angrenzenden Parameterräumen vorliegen, zeigen die beste Modellperformance, da das Deep Gaussian Process-Modell dort ausreichend Informationen besitzt, um die zugrunde liegenden Zusammenhänge zuverlässig zu lernen und generalisieren zu können. Insgesamt verdeutlichen die Daten, dass die Performance von DGP stark vom betrachteten Parameterbereich und der Datenmenge abhängt: Strecken mit sehr kleinen Zeitkonstanten ($T_1 < 1$) oder niedriger Verstärkung zeigen eine geringere Modellgenauigkeit, während typische Betriebsbereiche eine sehr hohe Vorhersagegüte ermöglichen.

5.2 Vergleich mit klassischen Tuning-Methoden

5.2.1 Evaluierung anhand synthetischer Strecken in Simulink

Um die Leistungsfähigkeit des entwickelten ML-basierten PID-Optimierungsmodells zu überprüfen, erfolgt ein systematischer Vergleich mit den klassischen Tuning-Methoden Ziegler-Nichols (ZN) und Chien-Hrones-Reswick (CHR) anhand synthetischer Teststrecken in Simulink [\[1\]\[2\]\[4\]](#). Die klassischen PID-Tuning-Methoden Ziegler-Nichols (ZN) und Chien-Hrones-Reswick (CHR) wurden in diesem Vergleich automatisiert am Computer umgesetzt. Bei der Ziegler-Nichols-Methode kommen sowohl Open-Loop- als auch Closed-Loop-Ansätze zum Einsatz. Im Open-Loop-Verfahren wird zunächst ein Sprung am Prozessinput appliziert, und die resultierende Ausgangsreaktion wird analysiert. Wesentliche Kennwerte wie Verstärkung, Zeitkonstante und Totzeit werden aus der Sprungantwort extrahiert und anschließend zur Berechnung der PID-Parameter genutzt. Für die Closed-Loop-Variante wird der Regler so eingestellt, dass das System schwingend arbeitet: Die proportionale Verstärkung K_p wird schrittweise erhöht, während Integrations- und Differenzanteile zunächst auf null gesetzt sind. Der Wert der Verstärkung, bei dem das System konstante Oszillationen zeigt (ultimative Verstärkung K_u) sowie die Periode der Schwingungen (ultimative Periode P_u) dienen als Grundlage zur automatischen Berechnung der PID-Parameter [\[19\]\[20\]](#).

Die CHR-Methode hingegen nutzt das Wendetangenten-Verfahren, um die Prozessparameter aus der Sprungantwort zu bestimmen. Auf dieser Basis werden die PID-Parameter für unterschiedliche Regelungsaufgaben berechnet, wobei zwischen Sollwertverfolgung (Servo)

und Störgrößenunterdrückung (Regler) sowie zwischen Varianten mit oder ohne Überschwingen unterschieden wird. Dieses Vorgehen ermöglicht eine gezielte Anpassung der Reglerparameter an gewünschte Systemverhalten und wurde für jede synthetische Teststrecke automatisiert am Computer durchgeführt [\[19\]\[20\]](#).

Die Teststrecken umfassen sowohl Parameterwerte innerhalb des Trainingsbereichs des Modells als auch gezielt Strecken leicht unter und über diesen Bereichs. Auf diese Weise kann nicht nur die Genauigkeit der ML-PID-Parameter für bekannte Referenzsysteme evaluiert werden, sondern auch die Robustheit und Grenzen des Modells gegenüber unbekannten oder extrapolierten Systemen analysiert werden.

Für jede Teststrecke werden die Schrittantworten der ML-optimierten PID-Regler sowie der ZN- und CHR-Regler simuliert. Die resultierenden Kennzahlen wie Integral des quadratischen Fehlers (ISE), Überschwingen, Einschwingzeit und weitere Performance-Metriken werden erfasst und miteinander verglichen. Diese Vorgehensweise ermöglicht eine objektive und reproduzierbare Bewertung der Optimierungsleistung und gibt Aufschluss darüber, in welchen Paramaterbereichen das ML-Modell zuverlässig arbeitet und wo die klassische Tuning-Methodik noch Vorteile bieten kann.

Zur Analyse der Performance des ML-basierten PID-Modells wurden die Regelparameter für eine Reihe PT1,PT2, IT1-Strecken getestet. Als Stresstest wurden Strecken mit den Parametern $K=1, 10, 100$ und $T_1=T_2=0.1$ gewählt. Das Ziel war, die Robustheit des Modells gegenüber stark variierenden Systemverstärkungen und Zeitkonstanten zu prüfen.

Die Simulationsergebnisse zeigen, dass das ML-Modell in den meisten Fällen stabile Regelungen liefert. Im Vergleich dazu liefern die klassischen Tuning-Methoden Ziegler-Nichols (ZN) und Chien-Hrones-Reswick (CHR) zu teils schlechteren Ergebnissen: ZN führt zu sehr langen Einschwingzeiten, während CHR eine geringe Überschwingung erreicht, jedoch langsamer reagiert.

Ein exemplarischer Vergleich für eine PT1-Strecke mit $K=100$, $T_1=0.1$ ist in Tabelle 4: Vergleich Leistungparameter1 zusammengefasst, die Sprungantworten der verschiedenen Regler sind zusätzlich in Abbildung 14: Stepresponse Synthetische Strecke1 dargestellt.

Tabelle 4: Vergleich Leistungparameter1

Controller	Rise Time [s]	Settling Time [s]	Overshoot [%]	ISE
ML	0.03	0.30	0.70	0.100
ZN	1.50	3.30	0.00	0.287
CHR 0%	5.71	10.51	0.00	0.941
CHR 20%	1.20	2.40	0.00	0.199

Key Performance Metrics (All Controllers)

	Controller	Rise Time [s]	Settling Time [s]	Overshoot [%]	ISE
0	ML	0.00	0.02	0.00	0.007
1	ZN	0.46	0.86	0.00	0.103
2	CHR 0%	1.72	3.08	0.00	0.308
3	CHR 20%	0.34	0.62	0.00	0.074

Closed-Loop Step Response

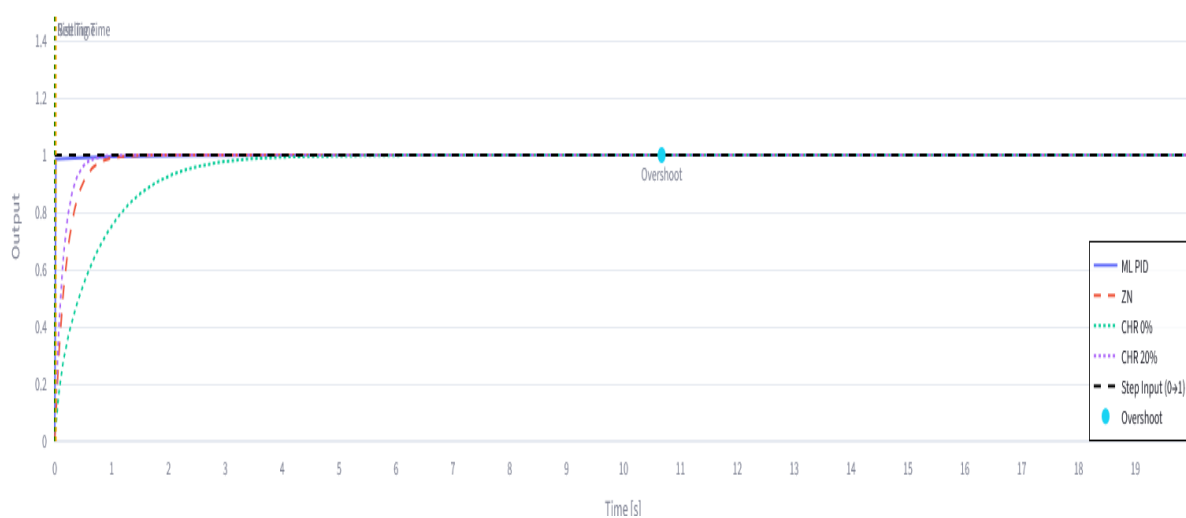


Abbildung 14: Stepresponse Synthetische Strecke1 (Eigendarstellung)

Neben der Untersuchung von extrem parametrisierten PT2-Strecken wurde die Leistungsfähigkeit des ML-basierten PID-Modells auch systematisch innerhalb des definierten Trainingsbereichs evaluiert. Ziel war es, die Konsistenz der Reglerqualität unter praxisnahen Bedingungen sowie die Robustheit gegenüber unterschiedlichen Streckentypen zu überprüfen. Dazu wurden PT1-, PT2- und IT1-Strecken mit einer Vielzahl an Parameterkombinationen herangezogen. Der Parameterraum umfasste Verstärkungsfaktoren K im Bereich von 0,5 bis 10, Zeitkonstanten $T1$ beziehungsweise $T2$ zwischen 0,1 und 100 sowie Totzeiten Td zwischen 0 und 3. Auf diese Weise konnte die Generalisierungsfähigkeit des Modells in Bezug auf unterschiedliche Streckendynamiken und Verzögerungscharakteristika umfassend analysiert werden.

Ein repräsentatives Beispiel für eine Strecke innerhalb dieses Bereichs ist ein PT2-Glied mit den Parametern $K = 1$, $T1 = 1$, $T2 = 2$. Die zugehörigen Kennzahlen der verschiedenen Reglerkonzepte sind in Tabelle 5:Vergleich Leistungsparameter2 dargestellt, während Abbildung 15: Stepresponse Synthetische Strecke2 die resultierenden Schrittantworten visualisiert.

Tabelle 5:Vergleich Leistungsparameter2

Controller	Rise Time [s]	Settling Time [s]	Overshoot [%]	ISE
ML	0.67	8.63	17.58	0.512
ZN	0.40	17.57	77.51	1.185
CHR 0%	0.75	7.44	26.25	0.535
CHR 20%	0.53	8.10	34.39	0.464

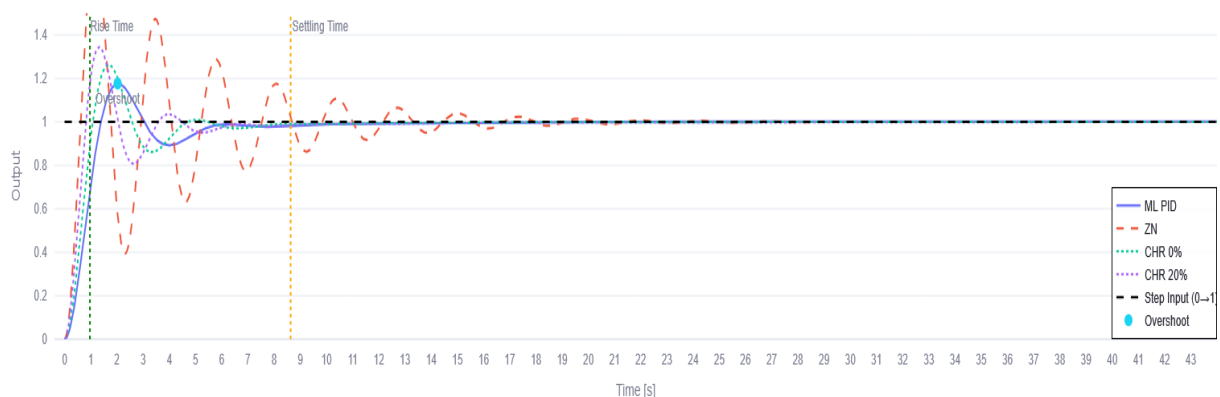


Abbildung 15: Stepresponse Synthetische Strecke2 (Eigendarstellung)

Diese Ergebnisse zeigen exemplarisch, wie sich der ML-basierte Ansatz im Vergleich zu klassischen Methoden verhält: Während der Ziegler-Nichols-Regler eine besonders kurze Anstiegszeit, jedoch verbunden mit einem sehr hohen Überschwingen und einer langen Einschwingzeit erzeugt, liefert der CHR-Regler ein moderateres Verhalten mit geringeren Überschwingern, aber weiterhin spürbaren Kompromissen bei der Genauigkeit. Der ML-Regler hingegen weist ein insgesamt ausgewogeneres Verhalten auf. Auch wenn die erzielte Anstiegszeit nicht die kürzeste ist, überzeugt jedoch durch eine im Vergleich geringere Überschwingweite, eine verkürzte Einschwingzeit gegenüber Ziegler-Nichols sowie einen reduzierten quadratischen Fehler.

Damit positioniert sich die ML-Reglung als praxistaugliche Alternative, die vor allem bei streckeninternen Parametrierungen eine robuste und stabile Regelgüte gewährleistet. Die Ergebnisse zeigen, dass das ML-Modell die Zielgrößen ausgewogen erreicht: Die Regelung bleibt stabil, die Einschwingzeit ist moderat und der Overshoot ist vergleichsweise gering. Im Vergleich dazu liefert Ziegler-Nichols eine sehr schnelle Reaktion, geht jedoch mit starkem Überschwingen einher. CHR-Regler erreichen ein geringeres Overshooting, benötigen jedoch teilweise längere Einschwingzeiten.

Um die Robustheit des ML-basierten PID-Modells weiter zu untersuchen, wurden Teststrecken an der oberen Grenze des Trainingsbereichs simuliert. Dabei zeigte sich, dass das Modell auch bei Werten oberhalb des bekannten Trainingsbereichs teilweise noch stabile Regelungen erzeugt. Allerdings geht dies mit deutlich erhöhtem Überschwingen und längeren Einschwingzeiten einher.

Ein exemplarischer Vergleich für eine PT2-Strecke mit stark von den Trainingsdaten abweichenden Parametern ist in *Tabelle 6: Vergleich Leistungsparameter3* dargestellt. Als Testfall wurde eine Strecke mit einer hohen Verstärkung von 15, einer sehr großen ersten Zeitkonstante von 250 und einer zweiten Zeitkonstante von 50 untersucht. Diese Parametrisierung liegt deutlich außerhalb des in der Datengenerierung vorgesehenen Trainingsbereichs und stellt somit eine besondere Herausforderung für die Generalisierungsfähigkeit der Optimierungsansätze dar.

Die in *Tabelle 6: Vergleich Leistungsparameter3* zusammengefassten Kennzahlen sowie die in *Abbildung 16: Stepresponse Synthetische Strecke3* dargestellten Schrittantworten verdeutlichen, dass die klassischen Heuristiken auch in diesem stark extrapolierten Bereich noch eine grundsätzlich stabile, wenn auch nicht optimale Regelgüte ermöglichen.

Tabelle 6: Vergleich Leistungsparameter3

Controller	Rise Time [s]	Settling Time [s]	Overshoot [%]	ISE
ML	8.69	259.31	76.99	22.627
ZN	6.06	94.51	47.80	6.803
CHR 0%	9.74	116	26.87	8.462
CHR 20%	7.37	109	37.61	7.393

Closed-Loop Step Response

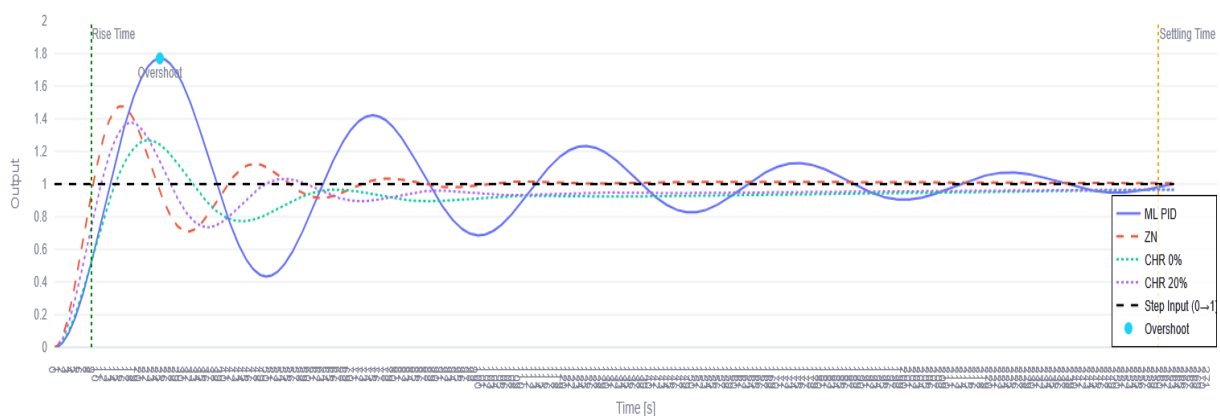


Abbildung 16: Stepresponse Synthetische Strecke3 (Eigendarstellung)

Die maschinellen Lernverfahren hingegen zeigten ein differenziertes Verhalten im Vergleich zu den herkömmlichen Tuningeinstellungen: Während einige Modelle auch unter diesen Bedingungen noch brauchbare Reglereinstellungen liefern konnten, erzeugten andere Verfahren instabile Regler (Abbildung 17: Stepresponse Synthetische Strecke instabil) oder lieferten unplausible Ergebnisse. Dazu zählten unter anderem negative Werte für den Proportionalfaktor K_p sowie extrem hohe Integrations- oder Differenzieranteile, die in der praktischen Anwendung nicht sinnvoll einsetzbar sind.

Closed-Loop Step Response

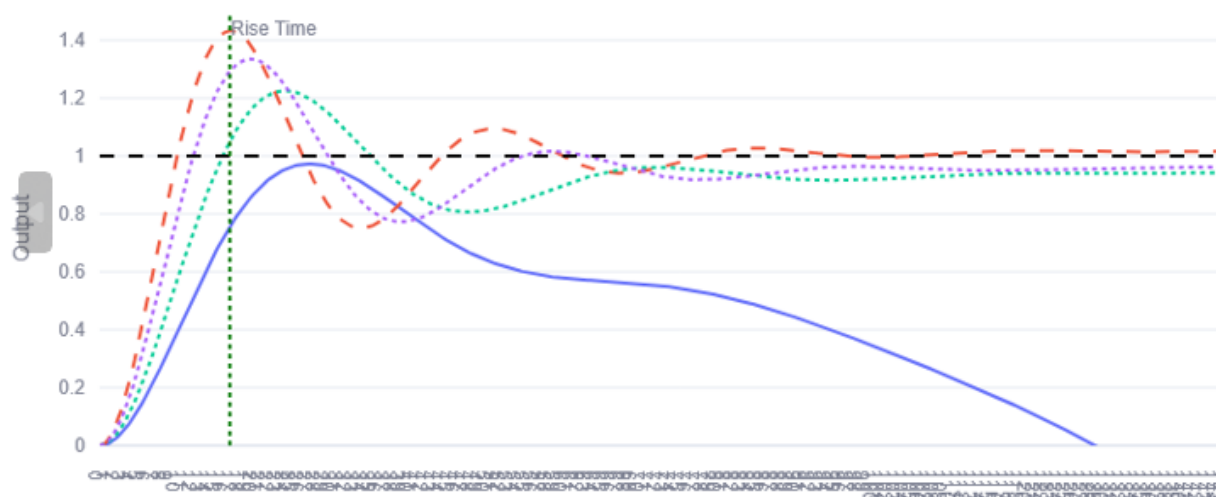


Abbildung 17: Stepresponse Synthetische Strecke instabil (Eigendarstellung)

Diese Ergebnisse machen deutlich, dass die Übertragbarkeit von datengetriebenen Ansätzen auf stark abweichende Streckenparameter nicht selbstverständlich ist und in der Regel nur innerhalb des definierten Gültigkeitsbereichs zuverlässig funktioniert. Gleichzeitig zeigen die Resultate, dass eine robuste Einschätzung der Vorhersagequalität sowie Mechanismen zur Erkennung von nicht einsetzbaren Reglerparametern essenziell für den praktischen Einsatz solcher Methoden sind.

5.2.2 Validierung an realen Regelstrecken

Neben der Simulation auf synthetischen Teststrecken ist die Validierung an realen Regelstrecken besonders aussagekräftig, da diese die tatsächlichen Dynamiken, Messrauschen und nichtlinearen Effekte eines physikalischen Prozesses berücksichtigen. Dadurch lassen sich praxisnahe Aussagen über die Leistungsfähigkeit der PID-Optimierung treffen.

Die untersuchte reale Teststrecke verfügte über einstellbare Verstärkung K , sowie Zeitkonstanten T_1 und T_2 , wobei die Verstärkung K zu Beginn auf 1 gesetzt wurde. Die Tests

begannen mit den Parametern $K=1$, $T_1=1$ und $T_2=1$, um eine Baseline zu erhalten. Anschließend wurde gezielt T_1 erhöht, um die Auswirkungen einer veränderten Systemdynamik auf die Regelung zu analysieren.

Die aufgenommenen Schrittantworten der verschiedenen PID-Regler ML-basiert, Ziegler-Nichols (ZN) und Chien-Hrones-Reswick (CHR) sind in Abbildung 18: Stepresponse reale Strecke1 und Abbildung 19: Stepresponse Plot ML Strecke 1 dargestellt. Dabei zeigt sich, dass die ML-Regelung eine langsamere Rise Time aufweist, dafür jedoch einen deutlich geringeren Overshoot und einen niedrigeren ISE erzielt. Die Settling Time liegt auf ähnlichem Niveau wie bei CHR, wodurch die ML-Regelung ein ausgewogenes Verhalten zwischen Geschwindigkeit und Stabilität bietet. Die klassischen Methoden liefern teilweise sehr schnelle Reaktionen oder geringe Überschwinger, erreichen jedoch nicht das gleiche ausgewogene Verhältnis wie der ML-basierte Ansatz.

Diese Experimente bestätigen, dass das ML-Modell nicht nur auf synthetischen Teststrecken, sondern auch auf realen Regelstrecken zuverlässig einsetzbar ist und die Vorteile einer flexiblen, datenbasierten Optimierung gegenüber klassischen Tuning-Methoden demonstriert.

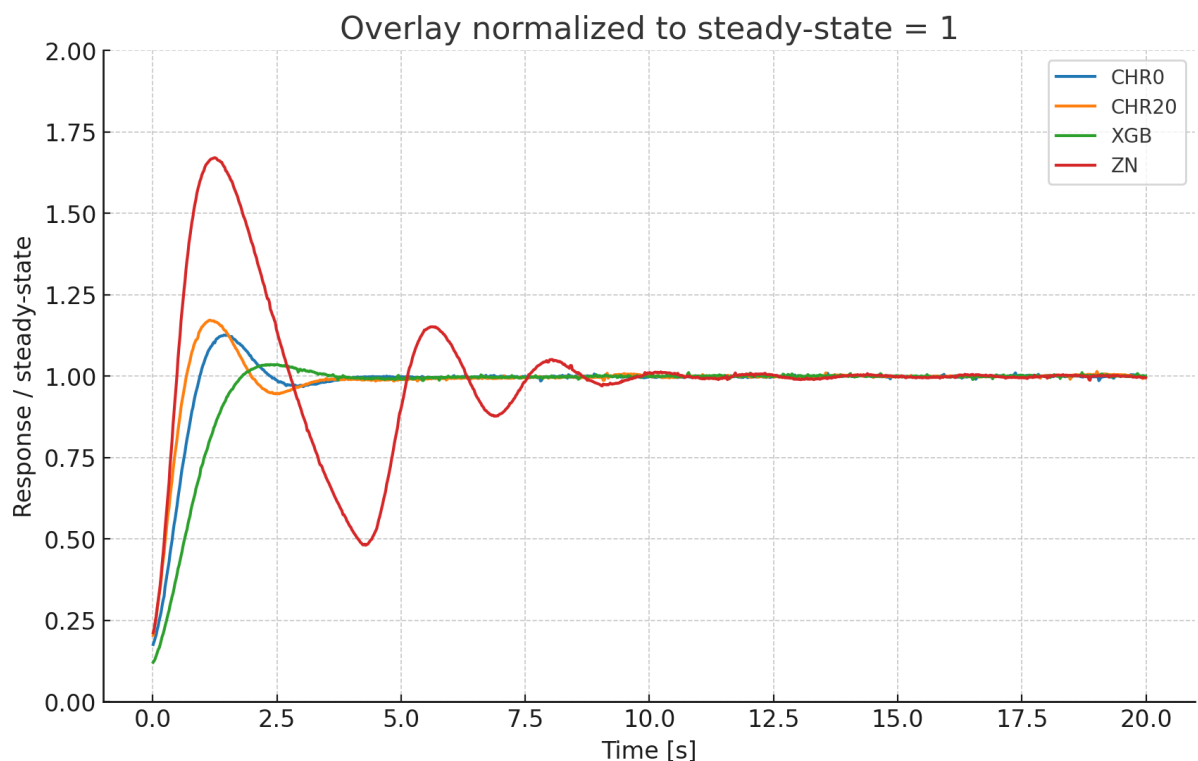


Abbildung 18: Stepresponse reale Strecke1 (Eigendarstellung)

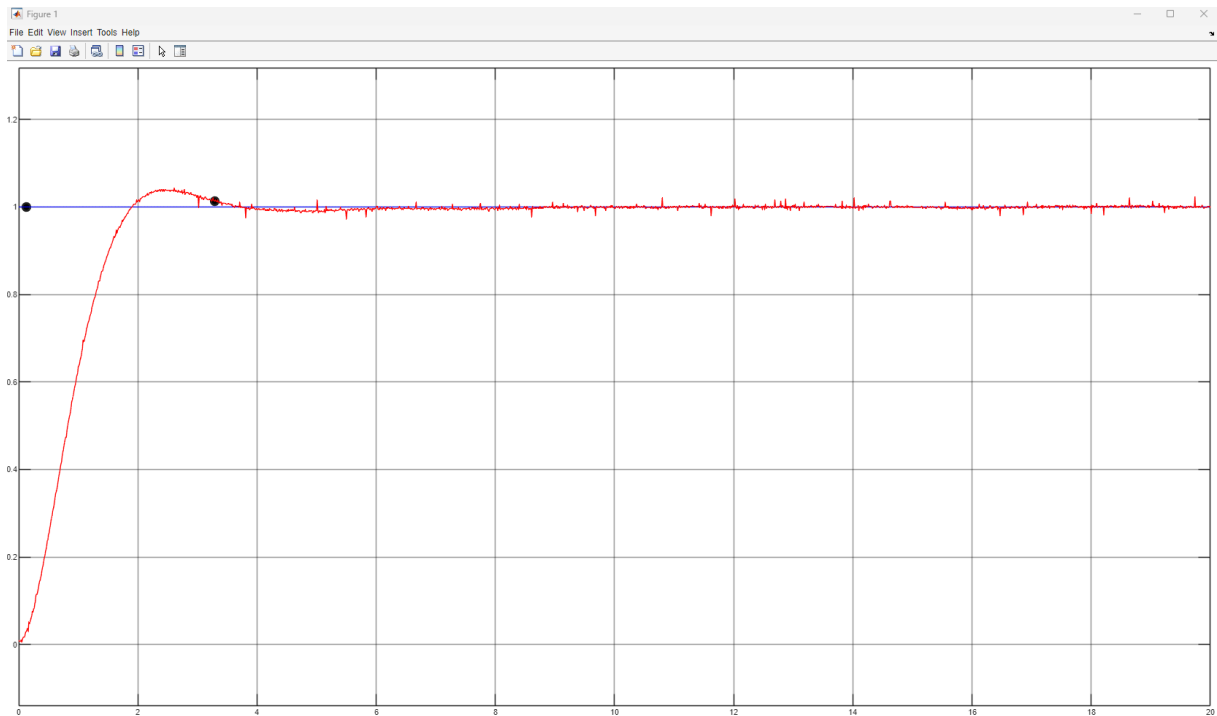


Abbildung 19: Stepresponse Plot ML Strecke 1 (Eigendarstellung)

Die in Tabelle 7: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 1 dargestellten Ergebnisse zeigen den Vergleich der PID-Tuning-Methoden für eine PT2-Strecke mit den Parametern $K=1$, $T_1=1$ und $T_2=1$. Dabei lassen sich deutliche Unterschiede in der Regelperformance zwischen den klassischen Verfahren und den Machine-Learning-basierten Ansätzen beobachten.

Tabelle 7: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 1

Tuning Methode	Kp	Ki	Kd	Status	Overshoot [%]	SSE	Rise Time [s]	Settling Time [s]
DGP	6,52	2,15	6,02	✓	15,70	0,3	1,56	8,42
Symbolic	4,28	2,02	3,64	✓	13,6	0,6	1,63	6,37
XGB	2,94	1,17	0,73	✓	0,38	0	1,28	3,14
ZN	11,56	20,50	10,3	✓	76,7	0,34	0,4	9,71
CHR 20	9,17	2,69	1,37	✓	20,4	0,17	0,75	3,37
CHR 0	5,79	2,13	0,81	✓	17,1	0	0,93	3,98

Die ML-Modelle DGP, Symbolic Regression und XGBoost liefern alle stabile Regler, wobei XGBoost die beste Gesamtperformance aufweist. Dies zeigt sich in einem minimalen Überschwingen, sehr niedrigem Fehler (SSE) sowie kurzen Anstiegs- und Einschwingzeiten. Symbolic Regression und DGP erreichen ebenfalls gute Ergebnisse, weisen jedoch leicht höhere Werte für Überschwingen und Einschwingzeit auf, insbesondere DGP, das in diesem Fall ein moderates Überschwingen zeigt, dafür aber einen insgesamt stabilen Verlauf mit niedrigerem SSE bietet.

Die klassischen Tuning-Methoden Ziegler-Nichols und Chien-Hrones-Reswick zeigen hingegen deutlich höhere Überschwingwerte, insbesondere Ziegler-Nichols, das ein starkes Überschwingen mit gleichzeitig langen Einschwingzeiten erzeugt. Die CHR-Variante mit 20% Dämpfung zeigt eine Verbesserung, liegt jedoch weiterhin hinter den ML-Methoden. Die CHR-Variante ohne Dämpfung erreicht moderate Werte, kann aber nicht die gleiche Effizienz wie die ML-Ansätze erreichen.

Die Ergebnisse in Tabelle 8: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 2 zeigen den Vergleich der PID-Tuning-Methoden für eine PT2-Strecke mit den Parametern $K=1$, $T_1=3$ und $T_2=5$ (Abbildung 20: Stepresponse reale Strecke3), also innerhalb des Trainingsbereichs. Die Tabelle verdeutlicht deutliche Unterschiede zwischen den klassischen Tuning-Methoden und den ML-basierten Ansätzen.

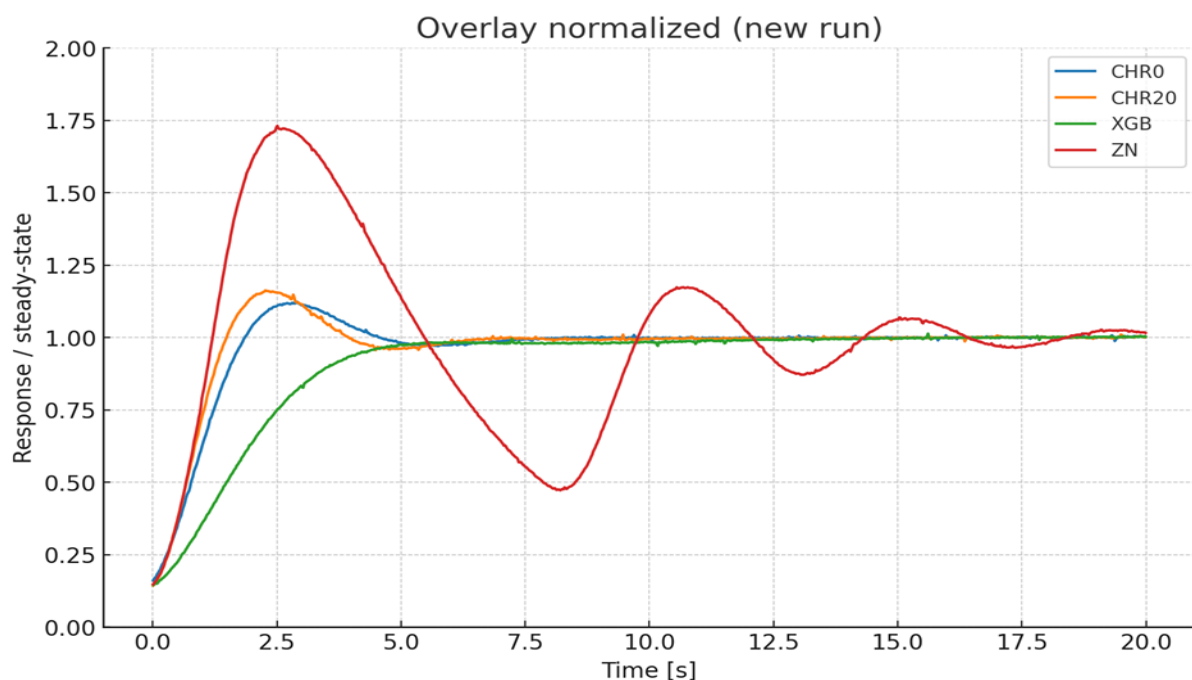


Abbildung 20: Stepresponse reale Strecke3 (Eigendarstellung)

Tabelle 8: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 2

Tuning Methode	Kp	Ki	Kd	Status	Overshoot [%]	SSE	Rise Time [s]	Settling Time [s]
DGP	2,72	2,10	14,85	Oszillierend	64,7	6,24	4,24	98,6
Symbolic	2,86	2,01	11,79	✓	56,02	0,84	4,1	61,27
XGB	4,18	1,85	3,72	✓✓✓	28,2	0,23	3,46	28,2
ZN	12,25	5,45	6,45	✓	36,63	0,50	2,03	36,89
CHR 20	9,52	0,68	4,6	✓✓✓	8,91	0,06	2,91	8,63
CHR 0	6,04	0,58	3,72	✓	10,74	0	3,79	10,36

Der DGP-Regler erzeugt in diesem Fall einen oszillierenden Verlauf mit sehr hohem Überspringen und verlängerten Einschwingzeiten. Symbolic Regression liefert stabile Regler, die jedoch ein vergleichsweise hohes Überspringen und lange Einschwingzeiten aufweisen.

XGBoost zeigt erneut die beste Performance: Die Regler sind stabil, der Überspringwert ist deutlich reduziert, der SSE ist minimal, und Anstiegs- sowie Einschwingzeiten sind vergleichsweise kurz. Dies unterstreicht die Stärke von XGBoost bei tabellarischen Daten, da es die Zusammenhänge zwischen Streckenparametern und Reglerkennwerten besonders effektiv erfasst.

Bei den klassischen Methoden zeigt Ziegler-Nichols moderate Überspringwerte, jedoch lange Einschwingzeiten. Die CHR-Variante mit 20% Dämpfung erreicht eine sehr gute Regelperformance, die in Bezug auf Überspringen und Einschwingzeit fast mit XGBoost vergleichbar ist. Die CHR-Variante ohne Dämpfung liefert stabile Regelung, bleibt aber insgesamt hinter den ML-Methoden zurück.

Die in Tabelle 9: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 3 dargestellten Ergebnisse zeigen den Vergleich der PID-Tuning-Methoden für eine PT2-Strecke mit den Parametern $K=0,8$, $T_1=1$ und $T_2=0,8$, also innerhalb des unteren Trainingsbereichs. Die Performance der verschiedenen Tuning-Methoden unterscheidet sich deutlich, insbesondere zwischen den klassischen Verfahren und dem Machine-Learning-basierten Ansatz. In Übereinstimmung mit der statistischen Analyse aus 5.1 Vergleich der Machine Learning Modelle, in der XGBoost die höchste Modellgüte aufwies, wurde für die nachfolgende Untersuchung ausschließlich dieses Verfahren zum Vergleich mit den klassischen Tuningverfahren herangezogen.

Tabelle 9: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 3

Tuning Methode	Kp	Ki	Kd	Status	Overshoot [%]	SSE	Rise Time [s]	Settling Time [s]
XGB	2,94	1,16	0,77	✓	0	0	1,84	2,34
ZN	14,58	29,03	1,83	✓	82,4	0,08	0,62	7,62
CHR 20	11,54	3,51	1,36	✓	17,8	0,01	0,69	1,78
CHR 0	7,29	2,99	0,97	✓	13,1	0	0,91	2,055

Der XGBoost-Regler liefert in diesem Fall die beste Gesamtleistung: Er erzeugt einen stabilen Regelverlauf ohne Überschwingen, mit minimalem SSE und kurzen Anstiegs- sowie Einschwingzeiten. Die klassischen Methoden Ziegler-Nichols und CHR zeigen teilweise erhebliche Unterschiede. Ziegler-Nichols erreicht zwar einen stabilen Regler, erzeugt jedoch ein sehr starkes Überschwingen und eine lange Einschwingzeit, was die Regelgüte deutlich einschränkt. Die CHR-Variante mit 20% Dämpfung liefert eine deutlich bessere Performance, reduziert Überschwingen und verkürzt die Einschwingzeit deutlich. Die CHR-Variante ohne Dämpfung zeigt stabile, aber insgesamt weniger effiziente Regelung im Vergleich zu XGBoost.

Zusätzlich wurde die ML-Regelung an einer realen PT2-Temperaturstrecke getestet, deren Parameter deutlich außerhalb des Trainingsbereichs lagen: $K=13,31$, $T_1=420$ und $T_2=54,47$. Die Übertragungsfunktionen wurde mithilfe des Simulink System Identification Tools bestimmt (Abbildung 21: Simulink Darstellung Temperaturreglung).

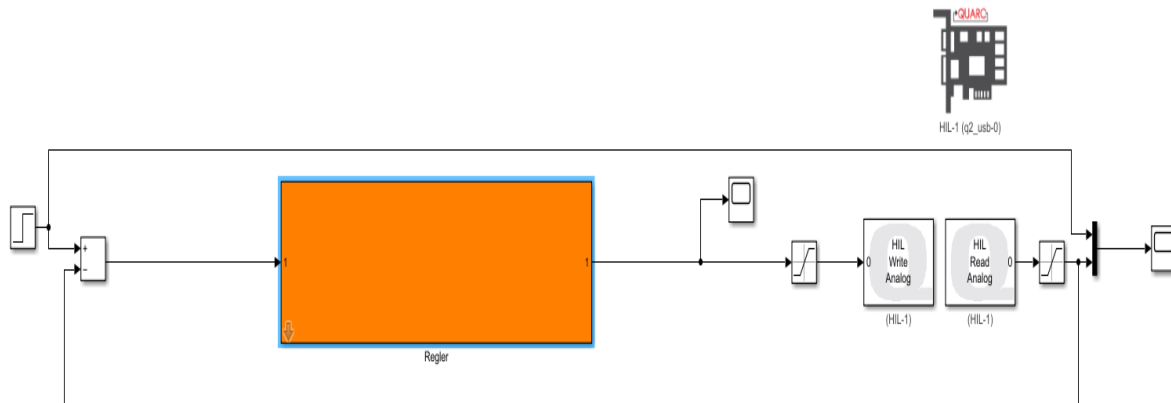


Abbildung 21: Simulink Darstellung Temperaturreglung (Eigendarstellung mithilfe von Dipl.-Ing. Sameh Ben Nablia)

Nach dem Erreichen des Steady State wurde eine Störung Z1 eingeführt, indem die Last des Motors von 1 auf 5 erhöht wurde. Nach erneuter Stabilisierung wurde eine zweite Störung Z2 appliziert, bei der die Last von 5 auf 10 angehoben wurde. Zur Sicherheit der Versuchsanlage wurde die Regelungsleistung reduziert, sodass die Maschine innerhalb sicherer Betriebsgrenzen betrieben werden konnte, ohne dass die Integrität der Komponenten gefährdet wurde, weshalb eine bleibende Regeldifferenz bei Störung Z2 verblieb.

Die Analyse der Sprungantworten zeigt, dass die ML-Regelung (Abbildung 22: ML Temperaturreglung mit Störung Z1 Z2) nach Einführung der ersten Störung Z1 und der zweiten Störung Z2 die schnellste Reaktion auf die veränderten Systembedingungen liefert. Nach einem hohen Überschwingens von 11 °C über der Solltemperatur zeigt die ML-basierte Regelung eine sehr kurze Anstiegs- und moderate Einschwingzeit. Die verbleibenden Oszillationen führen dazu, dass im Zeitraum ein schwankender SSE bestehen bleibt. Die ML-Regelung erholt sich am schnellsten von der Störung Z1 und beginnt auch mit der

Kompensation von Z2, wird durch eine bewusste Leistungsrosselung des Systems begrenzt.

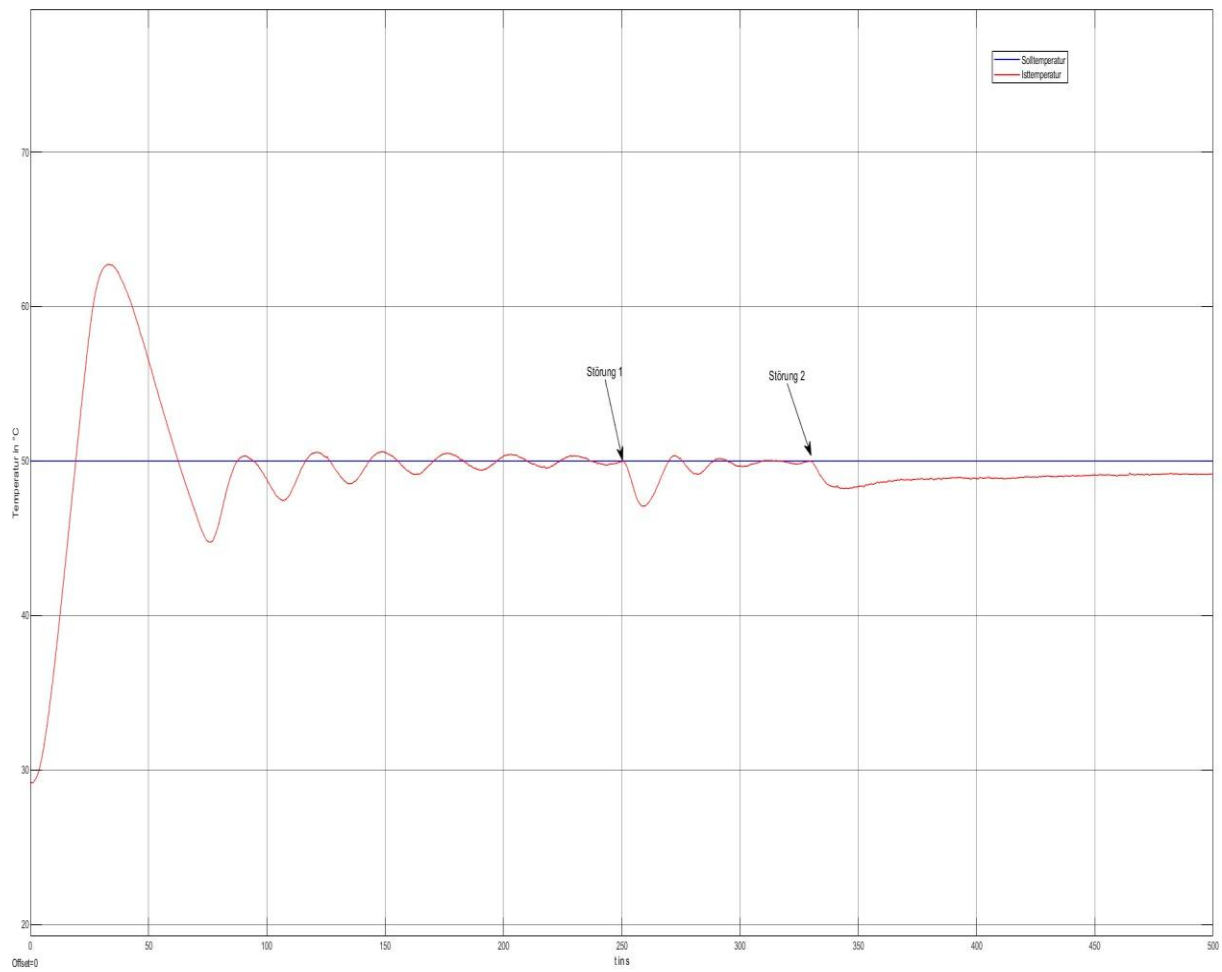


Abbildung 22: ML Temperaturreglung mit Störung Z1 Z2 (Eigendarstellung)

Die Ziegler-Nichols-Regler (Abbildung 23: ZN Temperaturreglung mit Störung Z1 Z2) zeigen im Vergleich ein moderates Überschwingen bei vergleichsweise kurzen Anstiegs- und Einschwingzeiten. Nach der Störung Z1 kann die Führungsgröße weitgehend erreicht werden, und auch auf eine zweite Störung wie Z2 würde das System noch reagieren. Insgesamt erfolgt die Anpassung jedoch langsamer als bei der ML-Regelung, was sich in längeren Rise- und Settling-Times sowie einem etwas höheren SSE widerspiegelt. Diese Ergebnisse zeigen, dass klassische Methoden wie ZN zwar stabile Regelung ermöglichen, jedoch in Extrembereichen der Prozessparameter nicht die gleiche Dynamik und Schnelligkeit in der Störungsbewältigung erreichen wie ML-basierte Ansätze.

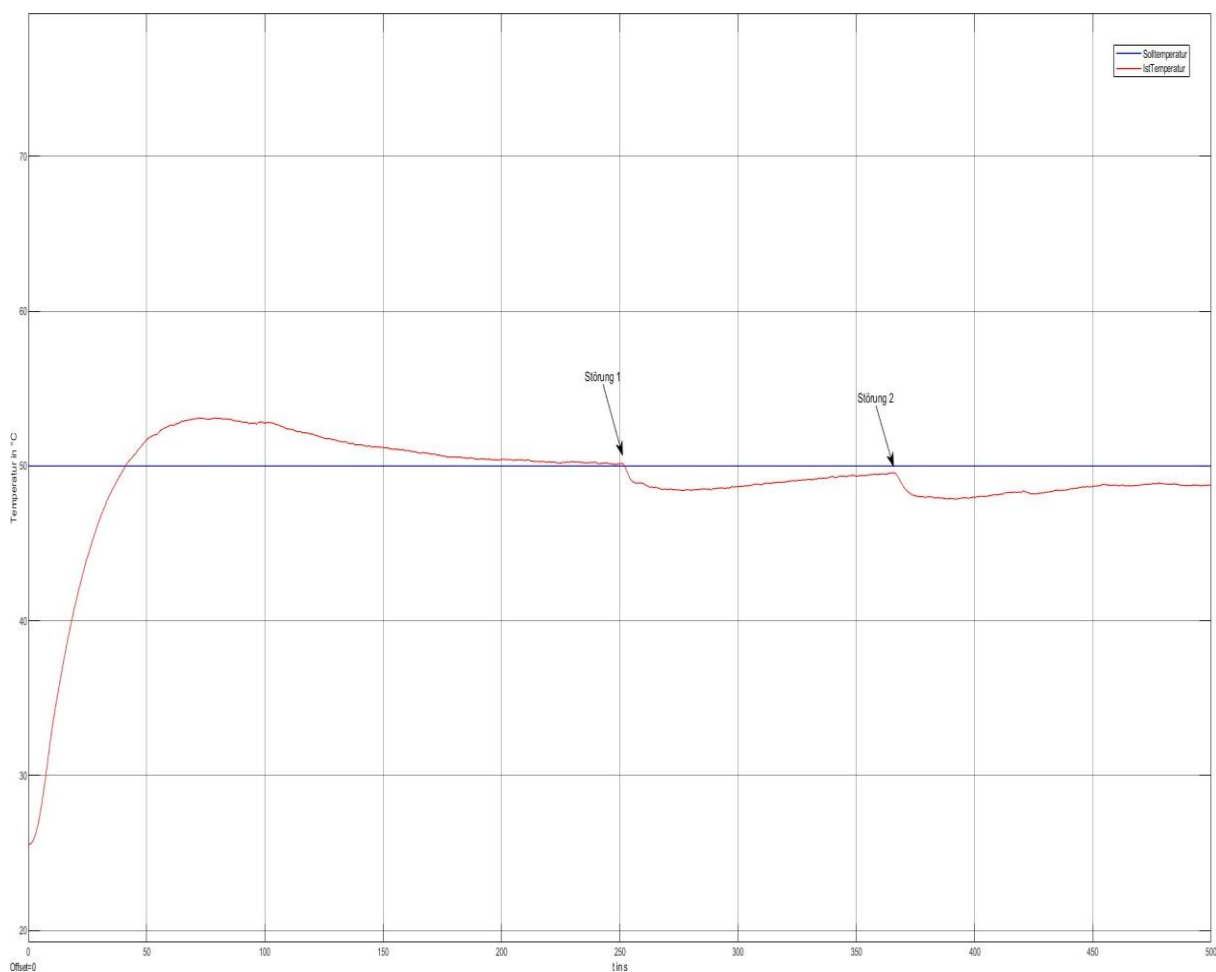


Abbildung 23: ZN Temperaturreglung mit Störung Z1 Z2 (Eigendarstellung)

Im Gegensatz dazu zeigen die CHR-Regler sowohl mit 20% Dämpfung (Abbildung 24: Chr20% Temperaturreglung mit Störung Z1 Z2) als auch ohne Dämpfung (Abbildung 25: Chr0% Temperaturreglung mit Störung Z1 Z2) eine deutlich trägere Reaktion auf die eingeführten Störungen. Die Anstiegszeiten sind verlängert, und die Einschwingzeiten fallen deutlich länger aus, wodurch die Führungsgröße nach der ersten Störung Z1 nur unvollständig erreicht wird. Obwohl kaum Überschwingen auftritt, resultiert diese verzögerte Anpassung in einem höheren integrierten quadratischen Fehler (SSE) und damit einer insgesamt geringeren Regelgüte. Aufgrund der unvollständig erreichten stationären Zustände konnte die zweite Störung Z2 nicht sinnvoll appliziert werden, da das System noch nicht stabilisiert war. Dieses Verhalten verdeutlicht, dass klassische CHR-Regler in stark abweichenden Prozessparametern ihre Anpassungsfähigkeit einschränken und die dynamische Leistungsfähigkeit gegenüber ML-basierten Regelungen deutlich reduziert ist.

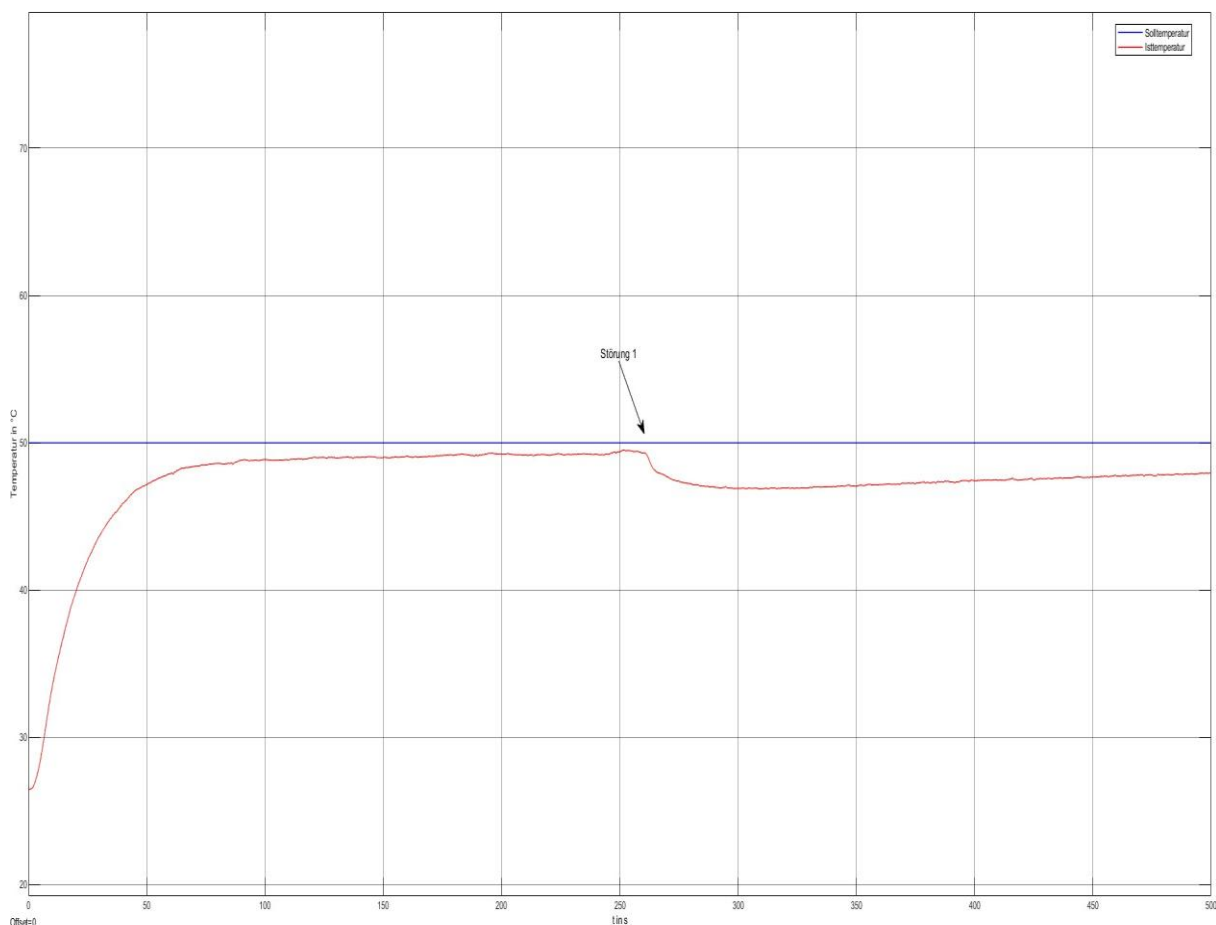


Abbildung 24: Chr20% Temperaturreglung mit Störung Z1 Z2 (Eigendarstellung)

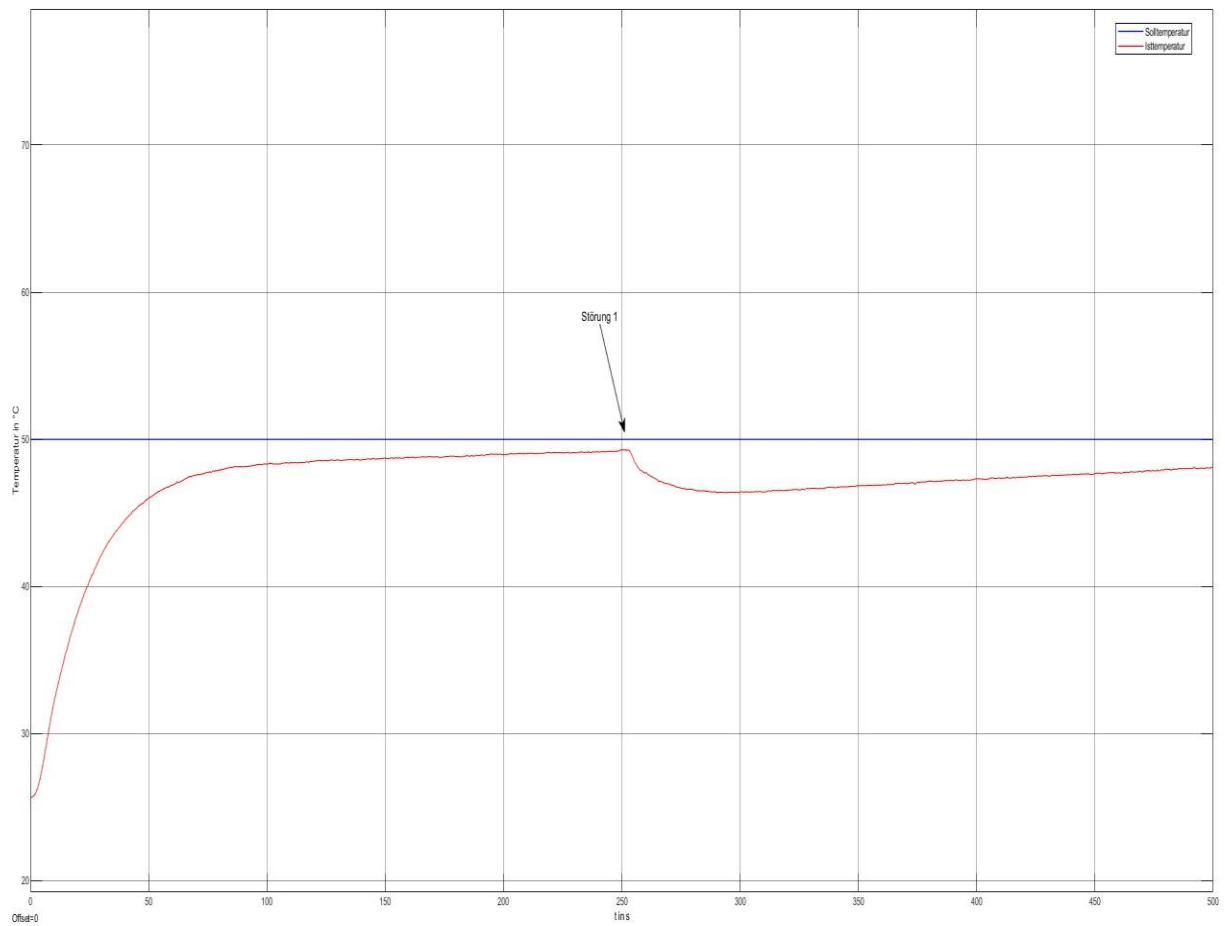


Abbildung 25: Chr0% Temperaturreglung mit Störung Z1 Z2 (Eigendarstellung)

6 Diskussion

Im Kapitel 6 „Diskussion“ werden die erzielten Ergebnisse kritisch analysiert. Es erfolgt eine Interpretation im Vergleich zu klassischen Tuning-Methoden, eine Bewertung der Stärken und Schwächen des Machine-Learning-Ansatzes sowie eine Betrachtung der Grenzen der Generalisierbarkeit auf unterschiedliche Regelstrecken und Einsatzszenarien.

6.1 Interpretation der Ergebnisse im Vergleich zu klassischen Verfahren

Die Analyse der simulierten Strecken verdeutlicht die Leistungsfähigkeit der ML-basierten PID-Regelung im Vergleich zu klassischen Methoden, insbesondere den CHR-Reglern, unter verschiedenen Parametrierungen. Für die Teststrecke aus Tabelle 4: Vergleich Leistungsparameter1, die durch hohe Verstärkungsanteile, niedrige Zeitkonstanten sowie K-Werte oberhalb des Trainingsbereichs und T-Werte unterhalb des Trainingsbereichs charakterisiert ist, zeigt die ML-Regelung in jeder betrachteten Metrik überlegene Werte. Mit einer Rise Time von 0,3 Sekunden, einer Einschwingzeit von 2,1 Sekunden und nahezu null Overshoot reagiert der ML-Regler deutlich schneller und präziser als die CHR-Regler. Die CHR-Regler, sowohl ohne Dämpfung als auch mit 20 % Dämpfung, weisen zwar ebenfalls null Overshoot auf, erreichen aber signifikant längere Anstiegs- und Einschwingzeiten. Der ZN-Regler liegt hinsichtlich Rise Time zwischen ML und CHR, erreicht jedoch eine höhere Einschwingzeit, sodass der ML-Regler die effizienteste Dynamik ausnutzt und eine präzise Steuerung auch außerhalb des Trainingsbereichs gewährleistet.

Innerhalb des Trainingsbereichs Tabelle 5:Vergleich Leistungsparameter2) liefert die ML-Regelung in vielen Bereichen überlegene Performance, während in einzelnen Metriken leichte Nachteile gegenüber den CHR-Reglern auftreten. So zeigt der ML-Regler eine Rise Time von 0,67 Sekunden, einen Overshoot von 17,6 % und einen ISE von 0,512, während der CHR-Regler mit 0 % Dämpfung in einzelnen Kennzahlen etwas besser abschneidet, etwa bei der Rise Time. Der ZN-Regler weist einen deutlich höheren Overshoot von über 77 % auf und eine sehr lange Einschwingzeit, was die ML-Regelung im Vergleich zu klassischen Methoden klar überlegen macht.

Für Strecken, die deutlich oberhalb des Trainingsbereichs liegen (Tabelle 6: Vergleich Leistungsparameter³), kann die ML-Regelung zwar weiterhin stabile Regler liefern, jedoch geht dies mit einem stark erhöhten Overshoot und teilweise oszillierendem Verhalten einher. Die CHR-Regler zeigen in diesen Extrembereichen zwar geringeres Überspringen, reagieren jedoch deutlich langsamer und benötigen längere Einschwingzeiten.

Die in Tabelle 10: Vergleich CHR/ML Synthetische Strecken dargestellten prozentualen Abweichungen zwischen der ML-basierten PID-Regelung und den CHR-Tunern sind teilweise sehr hoch. Diese prozentualen Werte ergeben sich jedoch aus sehr kleinen absoluten Differenzen, insbesondere bei Kennwerten wie der Anstiegszeit auf Strecken mit sehr kurzen Reaktionszeiten (z. B. Strecke 1, ML RT $\approx$ 0,04 s).

$$(15) \text{ prozentuale Abweichung}_{\%} = \frac{\text{Wert}_{CHR} - \text{Wert}_{ML}}{\text{Wert}_{ML}} \cdot 100$$

$$(16) \Delta = \text{Wert}_{CHR} - \text{Wert}_{ML}$$

Tabelle 10: Vergleich CHR/ML Synthetische Strecken

Strecke	CHR	RT [%]	RTΔ [s]	ST [%]	STA [s]	OS [%]	OSA [%]	ISE [%]	ISEΔ
Strecke 1 (Tabelle4)	0%	+14275	5,67	+3433	3,60	X	-0,70	+841	0,84
	20%	+2900	1,16	+700	2,10	X	-0,70	+99	0,09
Strecke 2 (Tabelle5)	0%	+11,19	0,08	-13,82	-1,19	+48,98	8,67	+4,46	0,02
	20%	-20,90	-0,14	-6,10	-0,53	+95,46	16,81	-9,30	-0,04
Strecke 3 (Tabelle6)	0%	+12,12	1,05	-55,25	-143,31	-65,14	-50,12	-62,41	-14,1
	20%	-15,17	-1,32	-57,90	-150,31	-51,18	-39,38	-67,08	-15,2

In den Bereichen, die dem Trainingsbereich der ML-Modelle entsprechen oder leicht darunter liegen (Strecke 1 und 2), übertrifft die ML-Regelung häufig die CHR-Regler. Beispielsweise werden auf Strecke 1 die Einschwingzeit (ST) um 3,60 s und der ISE um 0,84 reduziert, während die Anstiegszeit trotz eines hohen prozentualen Unterschieds nur um 5,67 s abweicht. Auf Strecke 2 zeigt die ML-Regelung insbesondere bei ST (-1,19 s) und Overshoot (+8,67 %) Vorteile gegenüber CHR 0 %, wobei die ISE um nur 0,02 absolute Einheiten verbessert wird.

Im Bereich oberhalb des Trainingsbereichs, dargestellt durch Strecke 3, fällt die Performance der ML-Regelung hingegen deutlich ab. Hier liegen Anstiegszeit und Einschwingzeit deutlich über den Werten der CHR-Regler, und der Overshoot ist stark erhöht, während auch die ISE

deutlich höher ausfällt. Dies verdeutlicht, dass das Modell außerhalb seiner Trainingsregionen die Regelgüte nur eingeschränkt zuverlässig vorhersagen kann.

Tabelle 11: Vergleich CHR/ML reale Strecken zeigt die prozentuale Abweichung an realen Teststrecken. Für reale Strecke 1 übertrifft der ML-Regler (XGB) den CHR 0 % in fast allen Kennwerten: Die Einschwingzeit beträgt 3,14 s gegenüber 3,98 s, der Overshoot liegt bei 0,38 % statt 17,1 % bei vergleichbaren SSE. Beim CHR 20 % liegen die Werte ebenfalls höher (ST: 3,37 s, OS: 20,4 %, SSE: 0,17). Lediglich die Anstiegszeit ist mit 1,28 s beim ML-Regler etwas länger als 0,93 s bzw. 0,75 s bei den CHR-Reglern.

Für reale Strecke 3 zeigt sich ein ähnliches Bild: Der ML-Regler erreicht eine Einschwingzeit von 2,34 s, einen Overshoot von 0 % und eine Anstiegszeit von 1,84 s, während CHR 0 % bei ST = 2,055 s, OS = 0 % und RT = 0,91 s liegt, und CHR 20 % bei ST = 1,78 s, OS = 17,8 % und RT = 0,69 s. Auch hier outperforms der ML-Regler nahezu alle Kennwerte außer der Anstiegszeit.

Auf reale Strecke 2 liegt der CHR 20 % dagegen vorn: Anstiegszeit 0,06 s vs. 0,23 s (ML), Einschwingzeit 2,91 s vs. 3,46 s und Overshoot 8,63 % vs. 28,2 %, wodurch der CHR-Regler in allen relevanten Metriken bessere Regelqualität erzielt.

Tabelle 11: Vergleich CHR/ML reale Strecken

Strecke	CHR	RT [%]	RT Δ [s]	ST [%]	ST Δ [s]	OS [%]	OS Δ [%]	SSE [%]	SSE Δ
Strecke 1 (Tabelle7)	0 %	-27,34	-0,35	+26,75	+0,84	+4421	+20,02	0	0
	20 %	-41,41	-0,53	+7,29	+0,23	+357	+4,93	+100	+0,17
Strecke 2 (Tabelle8)	0 %	+44,50	+1,69	+9,57	+3,23	-62,02	-16,06	+4657	+0,23
	20 %	+139,90	+4,84	-15,73	-4,88	-50,71	-14,32	+387	+0,17
Strecke 3 (Tabelle9)	0 %	+24,73	+0,91	+11,41	+0,74	+1700	+0,91	0	0
	20 %	+527,40	+9,70	-70,51	-1,56	-100	0	X	+0,01

Zusammenfassend lässt sich feststellen, dass die ML-basierte Regelung, insbesondere im Vergleich zu den CHR-Reglern, eine höhere Anpassungsfähigkeit und Dynamik aufweist. Sie liefert in den meisten Fällen eine schnellere Settling Time, geringeren ISE und eine effiziente

Erholung nach transienten Zuständen, während CHR und ZN eher konservative Regelungen mit stabiler, aber langsam reagierender Dynamik bieten.

Die Validierung an realen PT2-Temperaturstrecken liefert praxisnahe Einblicke in die Leistungsfähigkeit der ML-basierten Regelung. Für die Teststrecken aus Tabelle 7 bis 9, die alle innerhalb des Trainingsbereichs parametrisiert sind, zeigt die ML-Regelung durchgehend eine schnelle Anstiegszeit, kurze Einschwingzeiten und kontrolliertes Überschwingen. In Tabelle 7: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 1, mit moderaten Streckenparametern, erreicht der ML-Regler die Führungsgröße schneller als die CHR-Regler, die zwar ein geringes Überschwingen aufweisen, jedoch deutlich längere Rise- und Settling Times besitzen. Der ZN-Regler reagiert ebenfalls schnell, generiert jedoch höheres Overshoot, wodurch die ML-Regelung in der Gesamtperformance überlegen bleibt.

Für die zweite und dritte Teststrecke (Tabelle 8: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 2 und Tabelle 9: Vergleich der PID-Tuning-Methoden und resultierenden Regelkennwerte Strecke 3) zeigt sich ein ähnliches Bild: Der ML-Regler liefert ausgewogene Ergebnisse, wobei in einigen Kennzahlen die CHR-Regler leicht bessere Werte erzielen, etwa bei sehr geringem Overshoot. Der ZN-Regler bleibt zwar bei der Rise Time konkurrenzfähig, erzeugt jedoch deutlich höhere Überschwinger. Die ML-Regelung zeigt insgesamt eine gute Balance zwischen schneller Reaktion, stabiler Einschwingzeit und minimalem ISE. Wobei die CHR-Regler in der zweiten Strecke ein besseres Gesamtergebnis liefern und eines der Machine Learning Modelle (DGP) ein stark oszillierende Regelung erzeugte.

Ein zusätzliches Experiment an einer PT2-Temperaturstrecke mit Parametern über des Trainingsbereichs, $K=13,31$, $T_1=420$ und $T_2=54,47$, ergänzt die Bewertung um die Reaktion auf Störungen. Nach Erreichen des Steady State wurde die Störung Z1 eingeführt, indem die Last des Motors von 1 auf 5 erhöht wurde, gefolgt von einer zweiten Störung Z2, bei der die Last von 5 auf 10 erhöht wurde. Die Analyse der Schrittantworten zeigt, dass der ML-Regler die schnellste Reaktion auf die Störungen liefert, die Rise Time bleibt kurz, die Einschwingzeit moderat, und trotz erhöhtem Overshoot wird der stationäre Zustand effizient erreicht. Die CHR-Regler mit 20 % Dämpfung und ohne Dämpfung reagieren deutlich langsamer, erreichen die Führungsgröße nach Z1 nur teilweise und zeigen verlängerte Einschwingzeiten bei sehr geringem Overshoot, weshalb eine zweite Störung Z2 nicht sinnvoll eingeführt werden konnte. Der ZN-Regler reagiert schnell auf Z1 und Z2 jedoch langsamer als die ML-Regelung, und weist

auch ein auch ein Überschwingen auf. Die ML-Regelung hierbei eine dynamische Anpassungsfähigkeit und Robustheit im Störungsfall.

Insgesamt verdeutlichen die realen Teststrecken, dass die ML-Regelung sowohl innerhalb des Trainingsbereichs als auch in stark abweichenden Situationen durch schnelle Reaktion, effiziente Erholung nach Störungen und gute Balance zwischen Rise Time, Settling Time, Overshoot und ISE überzeugt und anwendbar ist.

6.2 Bewertung der Stärken und Schwächen des ML-Ansatzes

Der Einsatz von Machine-Learning-Methoden zur automatisierten PID-Regelung eröffnet neue Möglichkeiten, die über die klassischen Tuning-Strategien hinausgehen. Die Ergebnisse der vorliegenden Arbeit zeigen, dass ML-basierte Regler in Bezug auf Reaktionsgeschwindigkeit, Anpassungsfähigkeit und Flexibilität klare Vorteile bieten. So ermöglichen trainierte Modelle wie XGBoost, Symbolic Regression oder Deep Gaussian Processes eine schnelle und automatisierte Vorhersage von Reglerparametern für unterschiedlichste Streckenprofile, ohne dass ein manuelles Abstimmen erforderlich ist. Hierbei zeigt XGBoost die best Performance. Dies reduziert den Aufwand für die Parametrierung erheblich und ermöglicht eine deutlich effizientere Integration in praxisnahe Regelungsaufgaben. Ein wesentlicher Vorteil liegt zudem in der Anpassbarkeit durch gezieltes Retraining auf konkrete Anwendungsfälle: Die Modelle können auf spezifische Prozessbedingungen kalibriert werden, sodass die Regelung gezielt auf die charakteristischen Dynamiken der Anlage zugeschnitten ist.

Im Vergleich zu klassischen Methoden wie Ziegler-Nichols oder Chien-Hrones-Reswick zeigen die ML-Regler in vielen Szenarien eine überlegene Performance. Bei den simulierten Strecken innerhalb des Trainingsbereichs konnte eine Reduktion der der Einschwingzeit beobachtet werden, während der Overshoot und der stationäre Fehler auf einem niedrigen Niveau blieben. Besonders deutlich wird der Vorteil bei hochparametrierten oder extrem schnellen Strecken, bei denen klassische Methoden entweder zu starkem Überschwingen neigen oder die gewünschte Dynamik nicht erreichen. Hier konnte der ML-Regler trotz Streckenparametern außerhalb des Trainingsbereichs oft stabile Regler erzeugen und eine schnelle Reaktion nach Störungen gewährleisten, wie dies beispielsweise bei der Temperaturregelung mit den Störungen Z1 und Z2 gezeigt wurde.

Darüber hinaus ist die Fähigkeit der ML-Modelle, multikriterielle Optimierungen zu berücksichtigen, ein entscheidender Vorteil. Durch die Integration von Nutzerpräferenzen hinsichtlich der Gewichtung verschiedener Leistungsmetriken wie Anstiegszeit, Einschwingzeit, Overshoot oder ISE können ML-Regler gezielt auf bestimmte Anforderungen optimiert werden. Dies erlaubt einen flexiblen Trade-off zwischen Geschwindigkeit und Stabilität, der mit klassischen Methoden nur eingeschränkt möglich ist. Insbesondere Modelle wie Deep Gaussian Processes bieten zusätzlich die Möglichkeit, Unsicherheiten in den Vorhersagen durch Konfidenzintervalle abzuschätzen, was für die Risikobewertung in kritischen Anwendungen von großem Vorteil ist.

Trotz dieser Vorteile existieren Limitationen. In Extrembereichen, also bei stark abweichenden Parametern oder sehr hohen Verstärkungen, können ML-Modelle instabile Regler oder unrealistische Parameterwerte wie negative K_p -Werte erzeugen. Die Qualität der Regelung hängt stark von der Menge, der Diversität und der Repräsentativität des Trainingsdatensatzes ab. Zudem ist die Interpretierbarkeit der Reglerparameter eingeschränkt; während klassische Methoden eine klare, nachvollziehbare Beziehung zwischen Parametern und Systemdynamik bieten, liefern ML-Modelle lediglich Vorhersagen, deren innerer Mechanismus oft komplex und nicht unmittelbar verständlich ist (Black Box). Während die einzelnen Vorhersagen nach diesem Prinzip schwer nachvollziehbar sind, sind die Trainingsdaten und Pipelines jedoch öffentlich zugänglich und beliebig anpassbar.

In der Gesamtsicht zeigt der ML-Ansatz ein ausgewogenes Leistungsprofil: Er bietet eine schnelle, flexible und robuste Regelung, die insbesondere in Szenarien mit variierenden Streckenparametern, Störungen oder individuellen Optimierungszielen Vorteile gegenüber klassischen Tuning-Methoden hat. Gleichzeitig müssen die Grenzen in extremen Parametrierungen sowie mögliche Unsicherheiten bei der Parameterbestimmung berücksichtigt werden. Insgesamt ergibt sich eine vielversprechende Perspektive für den Einsatz von Machine Learning in der PID-Regelung, insbesondere in praxisnahen Anwendungen, in denen Anpassungsfähigkeit, Geschwindigkeit und automatisierte Parametrierung entscheidend sind.

6.3 Grenzen der Generalisierbarkeit

Die Analyse der ML-basierten PID-Regler zeigt, dass innerhalb des Trainingsbereichs (Tabelle 5,7,8) konsistent gute Ergebnisse erzielt werden. Auch leicht unterhalb der Trainingsgrenzen liefern die Modelle stabile Regler mit akzeptablen Anstiegs- und Einschwingzeiten sowie Überschwung (Tabelle 4). Mit zunehmender Entfernung vom Trainingsbereich, insbesondere bei höheren Zeitkonstanten, treten jedoch vermehrt Probleme auf: Die Reglerparameter führen teilweise zu starkem Überspringen, verlängerten Einschwingzeiten oder sogar zu oszillierenden, instabilen Regelungen (Tabelle 6, Abbildung 16: Stepresponse Synthetische Strecke3, Abbildung 17: Stepresponse Synthetische Strecke instabil, Abbildung 21: Simulink Darstellung Temperaturreglung). In solchen Regionen ist die Generalisierbarkeit der trainierten Modelle eingeschränkt, da die zugrunde liegenden Strukturen der Strecken nicht mehr ausreichend durch die vorhandenen Trainingsdaten abgedeckt sind.

Die praktische Konsequenz besteht darin, dass ML-Modelle nicht ohne weiteres für Strecken verwendet werden sollten, deren Parameter deutlich außerhalb des Trainingsbereichs liegen. Stattdessen empfiehlt sich ein gezieltes Retraining der Modelle für diese speziellen Parameterbereiche, um die Vorhersagegenauigkeit zu erhöhen und sichere, stabile Regler zu gewährleisten. Diese Vorgehensweise stellt sicher, dass die Vorteile der ML-basierten Regelung schnelle, automatisierte Parametervorhersage und gute Performance innerhalb bekannter Regionen auch bei neuartigen oder stark abweichenden Prozessbedingungen genutzt werden können.

7 Ausblick

Ein zentraler Ansatz für weiterführende Arbeiten liegt in der Erweiterung des Trainingsbereichs der ML-Modelle, um auch Prozesse mit stark variierenden Parametern, höheren Zeitkonstanten oder größeren Verstärkungen zuverlässig abbilden zu können. Hierdurch könnten die Modelle auch für Strecken jenseits der bisher definierten Bereiche robuste Regelungen liefern. Darüber hinaus wäre die Integration weiterer Streckentypen, wie PTn-Glieder oder Verzögerungsglieder, sowie von hintereinander geschalteten, heterogenen Regelstrecken denkbar, um die Anwendbarkeit der Methode auf komplexere industrielle Prozesse zu erhöhen. Zukünftige Erweiterungen könnten darauf abzielen, dass die ML-Modelle

nicht mehr ausschließlich die Übertragungsfunktion als Eingabe nutzen, sondern direkt aus den gemessenen Ein- und Ausgangssignalen der Regelstrecke lernen, wodurch eine noch flexiblere und praxisnähere Parametrierung von PID-Reglern ermöglicht wird.

Ein vielversprechender Ansatz zur Erweiterung des Trainingsbereichs ist der Einsatz von Reinforcement Learning. Hierbei wird der Regler durch Interaktion mit dem System kontinuierlich verbessert, indem er für gute Regelaktionen belohnt und für schlechte bestraft wird. Reinforcement Learning ermöglicht es, nichtlineare und instabile Prozesse effizient zu handhaben, und kann so die Robustheit der ML-Modelle gegenüber stark variierenden Prozessparametern erhöhen [\[21\]](#).

Ein weiterer Ansatz ist die Integration von self-tuning PID-Reglern, bei denen die Reglerparameter in Echtzeit an die aktuellen Systembedingungen angepasst werden, statt als stare Werte vorgegeben. Dies erlaubt eine kontinuierliche Optimierung der Regelperformance während des Betriebs, insbesondere in dynamischen Umgebungen wie etwa selbststeuernden Autos oder Drohnen mit sich ändernden Prozessgrößen. Die Kombination solcher selbstadaptiven Methoden mit den bestehenden ML-Modellen könnte die Praxisrelevanz und Flexibilität der PID-Optimierung deutlich steigern [\[22\]](#).

Zur Steigerung der Genauigkeit der Vorhersagen bietet sich ein gezieltes Hyperparameter-Tuning der ML-Modelle an, kombiniert mit einer Aufteilung in spezialisierte Modelle für unterschiedliche Trainingsbereiche oder Parameterklassen. Durch diese Modularität könnten für spezifische Anwendungsfälle maßgeschneiderte Reglerparametrierungen erstellt werden, die sowohl die Performanz als auch die Robustheit verbessern.

8 Literaturverzeichnis

- [1] Loose, T: Applied Control and Automation Technology, Springer Vieweg 2022
- [2] Föllinger, Otto: Regelungstechnik, VDE VERLAG, 11. Auflage, Berlin, 2013.
- [3] Unbehauen, Heinz: Regelungstechnik I, Springer Vieweg, 15. Auflage, Berlin, 2008.
- [4] MathWorks: PID Autotuning Basics, <https://www.mathworks.com/help/slcontrol/ug/pid-autotuning-basics.html>, Aufruf am 22. Sep. 2025.
- [5] Kusiak, Andrew: Smart Manufacturing, International Journal of Production Research, Taylor & Francis Journals, vol. 56(1-2), pages 508-517, January.
- [6] Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron: Deep Learning, MIT Press, Cambridge, MA, 2016.
- [7] Hastie, Trevor; Tibshirani, Robert; Friedman, Jerome: The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Springer, 2nd edition, New York, 2009.
- [8] Bishop, Christopher M.: Pattern Recognition and Machine Learning, Springer, New York, 2006.
- [9] Lee, Jay; Bagheri, Behrad; Kao, Hung-An: A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems, In: Manufacturing Letters, 3, 2015, S. 18-23.
- [10] Bujgoi, G.; Sendrescu, D. Tuning of PID Controllers Using Reinforcement Learning for Nonlinear System Control. Processes 2025, 13, 735
- [11] Samigulina Z.I.; Amangaliyeva A.G.: Optimization of PID Controller Parameters Using Machine Learning Algorithms Based on Oil Separation Process Data, 2023.
- [12] Popa, D.; Precup, R.-E.; Roman, R.-C.; Petriu, E.M. Model-Based Control of a Glass Melting Furnace. Processes 2024, 12, 2085.
- [13] Bujgoi, G.; Sendrescu, D. One-Shot, Offline and Production-Scalable PID Optimisation with Deep Reinforcement Learning. arXiv 2022, arXiv:2210.13906.
- [14] Breiman, Leo: Random Forests, In: Machine Learning, 45(1), 2001, S. 5–32.
- [15] Chen, Tianqi; Guestrin, Carlos: XGBoost: A Scalable Tree Boosting System, In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13.-17. Aug. 2016, S. 785–794.
- [16] Cranmer, Miles et al.: PySR: High-Performance Symbolic Regression in Python and Julia, 2023, arXiv:2305.01582.
- [17] Rasmussen, Carl Edward; Williams, Christopher K. I.: Gaussian Processes for Machine Learning, MIT Press, Cambridge, MA, 2006.
- [18] Salimbeni, Hugh; Deisenroth, Marc: Doubly Stochastic Variational Inference for Deep Gaussian Processes, In: Advances in Neural Information Processing Systems 30 (NIPS), Long Beach, CA, USA, 4.-9. Dez. 2017.
- [19] Rodrigues, D.: Análise Comparativa de Métodos de Sintonia para Controladores PID: Ziegler-Nichols e Chien-Hrones-Reswick. Universidade Tecnológica Federal do Paraná (UTFPR). <https://github.com/rodriguesdeyson/pid-tuner>

- [20] Muresan, C. I.; De Keyser, R.: Revisiting Ziegler–Nichols: A Fractional Order Approach. ISA Transactions, 2022
- [21] van Niekerk, J.A.; le Roux, J.D.; Craig, I.K.: Reinforcement learning based automatic tuning of PID controllers in multivariable grinding mill circuits, Control Engineering Practice, 2025, 106522
- [22] Sharifi, I.; Alasty, A.: Self-Tuning PID Control via a Hybrid Actor-Critic-Based Neural Structure for Quadcopter Control, arXiv 2307.01312, 2023

9 Anhang

Die Ergebnisse meiner Arbeit sind über die Webanwendung unter <https://ml-based-pid-optimizer.streamlit.app/> abrufbar, während das zugehörige GitHub-Repository den vollständigen Quellcode sowie die verwendeten Datensätze enthält <https://github.com/NikKessel/PID-Controller-optimization-with-machine-learning/tree/main>

Im Rahmen der Erstellung dieser Prüfungsarbeit habe ich nach Rücksprache mit meinem Erstbetreuer KI-gestützte Schreibwerkzeuge und weitere KI-Anwendungen eingesetzt, wobei ich steuernd gearbeitet habe und mein eigener gestalterischer Einfluss überwiegt. Genutzt wurden ChatGPT zur Formulierung von Texten und Unterstützung beim Coding (<https://chat.openai.com>), GitHub Copilot zur Programmierung (<https://github.com/features/copilot>) sowie Claude zur Erstellung von Grafiken (<https://www.anthropic.com>)

Projektstruktur:

PID-Controller-optimization/

```
├── data/          # Synthetic and real-world system datasets
├── src/           # All core source code (models, logic, GUI)
├── models/        # Trained regressors, surrogates, classifiers
├── results/       # Optimization and benchmark outputs
├── plots/         # Step responses, radar charts, performance plots
├── streamlit/     # Streamlit app logic and interface
├── simulink/      # Simulink model + MATLAB interaction scripts
├── requirements.txt # Dependency list
└── README.md      # Project overview
```

Pipeline Selbsttest

Generate synthetic dataset

```
python src/data/generate_pid_dataset_full.m
```

Train ML models of choice

```
python src/models/train_dgp.py
```

Run PID optimization using surrogate + GA

```
python src/tuning/run_optimizer.py
```

Launch GUI

```
streamlit run streamlit/streamlit_app.py
```

Tech Stack

Python 3.12+

Libraries: NumPy, SciPy, Pandas, scikit-learn, XGBoost, PySR, DEAP, PyTorch

Surrogates: GPR, Deep GP, MLP

Optimization: Genetic Algorithm (DEAP), Bayesian Optimization (Scikit-Optimize/BoTorch)

Frontend: Streamlit

Simulink backend via MATLAB .mat interaction (subprocess)

Control Systems Toolbox (Python) for simulation

Auszug Trainingsdatenset

K	T1	T2	L	Kp	Ki	Kd	ISE	SSE	RiseTime	SettlingTime	Overshoot	SystemType
2.76	12.6400	4.179	4.82	1.63	10.02	4.31	20.31	0.076	5.4574204	35.50698002	0	PT1_MediumC
8.06	1.48808	6.485	3.58	0.14	0.00	0.17	20.03	0.294	27.114858	45.05707905	0	PT1_VeryHigh
6.31	2.11975	12.74	0.25	0.36	0.10	0.03	0.644	1.115	3.0422975	9.683248502	0	PT1_VeryHigh
4.94	3.19802	2.137	0.02	0.57	0.36	0	0.502	1.176	1.5292984	6.569963991	8.22410898	PT1_VeryHigh
7.27	3.89893	1.932	9.31	0.16	10.00	0.53	451.00	0.289	27.298874	45.07184854	0	PT1_HighGain
3.63	5.82909	2.796	2.37	0.13	0.00	3.93	18.62	0.595	0.3112450	42.65894485	229.553999	PT2_HighGain
6.74	0.64306	2.362	0.61	0.12	0.13	0.00	0.822	1.998	1.17801791	4.097810008	8.88178419	PT1_VeryHigh
5.59	1.64823	0	3.26	0.31	0.03	0.23	5.724	0.003	0.3659140	29.08155760	11.7105627	PT1_VeryHigh
3.60	3.72996	0	2.01	0.91	0.08	0.64	2.319	0.002	0.9122085	25.06300237	0	PT1_HighGain
6.78	4.36363	4.992	0.53	1.12	0.09	3.29	0.877	7.600	1.1152371	11.15565507	0	PT2_HighGain
2.72	6.16884	0	6.63	0.82	0.01	2.15	32.50	0.160	0.2885663	42.09095631	0	PT1_HighGain
2.27	18.4007	0	3.25	4.21	0.07	7.00	6.729	0.030	0.9426526	20.37385910	0	PT1_LowGain
2.38	1.46099	0	1.92	0.58	0.01	0.51	7.765	0.230	25.950428	44.65969218	0	PT1_VeryLowC
2.56	0.97055	1.763	0.66	0.18	0.00	1.44	14.87	0.423	0.2824086	47.52721191	42.1868637	PT2_VeryHigh
6.47	1.64813	1.138	2.08	0.20	0.02	0.42	3.600	0.004	0.6707838	32.94131410	22.6202699	PT2_VeryHigh
5.08	4.52462	0.860	3.46	0.04	0.00	1.20	30.93	0.774	0.1374154	46.97895828	466.441516	PT2_VeryHigh
2.72	4.17986	0	7.35	0.54	10.00	1.49	57.25	0.239	23.950305	44.24618704	0	PT1_VeryHigh
7.73	4.48556	2.265	2.50	0.07	0.00	2.11	17.84	0.565	0.3257907	44.34328996	230.091009	PT2_HighGain
1.20	12.7497	0	0.08	10.3	2.88	0	0.534	9.595	1.4565349	9.658363016	10.0652052	PT1_MediumC
4.35	2.13717	3.032	3.25	0.06	10.00	1.74	29.57	0.760	0.1777212	49.71786172	484.902458	PT2_HighGain
5.59	1.93283	0	1.60	0.38	0.09	0.16	1.665	3.277	0.9236948	11.03939496	0	PT1_VeryHigh
9.30	2.79625	0	0.43	0.29	0.12	0	0.779	4.758	1.0835402	4.881699114	5.51043661	PT1_VeryHigh
6.28	2.36249	1.033	2.32	0.37	0.05	0.22	2.731	0.000	0.6988116	17.22210729	6.56040562	PT1_VeryHigh
2.18	18.2786	2.609	1.95	4.92	0.08	6.91	3.044	0.027	3.3494345	18.03985418	0	PT1_VeryLowC
2.79	14.09870	8.256	1.49	1.53	0.17	0.86	1.613	0.000	0.8329353	16.90609195	0	PT1_HighGain
6.38	7.01561	19.611	2.61	0.91	0.09	0.73	3.707	0.000	0.5284424	7.276381581	32.2318041	PT1_HighGain
3.30	2.26165	2.934	1.09	0.68	0.02	0.59	4.038	0.106	21.048381	42.65996922	0	PT1_VeryHigh
3.90	13.4333	7.224	1.23	3.38	0.32	1.12	1.358	0.000	0.8251753	4.318114284	18.6364370	PT1_MediumC
3.31	2.52749	0	2.20	0.71	0.06	0.47	2.689	0.005	0.9024097	30.22962060	0	PT1_LowGain
7.19	2.41568	0	0.93	0.22	0.00	0.32	84.32	0.100	26.5239831	43.96272370	0	PT1_VeryHigh
5.26	7.43649	0	1.58	0.83	0.01	1.17	3.237	0.084	6.0000637	36.62727222	0	PT1_HighGain
2.66	6.31202	0	1.79	2.15	0.21	1.30	1.948	0.000	0.7694094	11.19591845	9.13940347	PT1_HighGain
1.54	15.8500	1.212	3.42	0.57	0.00	16.2	18.73	0.484	0.3238759	49.84784223	219.335051	PT2_VeryLowC
6.28	7.04729	0	3.32	0.58	0.01	0.98	7.639	0.100	5.2412438	38.35681812	0	PT1_HighGain
8.83	11.03319	0	0.08	0.77	0.93	0.03	0.114	0	0.2509768	1.017429200	2.33633237	PT1_VeryHigh
6.77	2.60924	0.753	0.25	0.83	0.36	0.47	0.469	8.881	0.6785286	4.697803169	7.15614814	PT2_VeryHigh
4.10	8.25638	6.582	1.42	0.48	10.00	13.6	5.242	0.272	0.5818474	18.69813876	80.6777910	PT2_MediumC
2.41	19.61103	0	0.88	4.04	10.40	0.59	1.070	1.890	0.9264272	3.760216374	12.6324218	PT1_LowGain
3.57	2.93444	8.125	1.66	0.94	0.03	7.13	3.380	0.052	0.7310642	41.33877949	45.0087390	PT2_LowGain
1.41	7.22411	0	1.97	4.51	10.43	2.98	2.255	0.000	0.6965291	7.834149733	14.1645901	PT1_MediumC
1.04	7.35891	0	0.80	7.14	0.95	0.59	1.059	1.380	0.9142793	3.760075623	14.1246037	PT1_VeryLowC
6.12	3.92371	0	4.12	0.18	0.00	0.63	2312	0.307	27.927791	45.22286904	0	PT1_VeryHigh